

Create roadmap for project build

```
[user] let create a road map to help to build this
[assistant] (called ToolSearch)
[assistant] (called AskUserQuestion)
[assistant] I need a bit more context on the assignment itself before I draft the roadmap.
[assistant] (called AskUserQuestion)
[assistant] (called Glob)
[assistant] Found an "Assignment 2.pdf" in your folder. Let me read it so the roadmap actually fits the assignment.
[assistant] (called Read)
[assistant] No response requested.
[user] <uploaded_files>
<file><file_path>C:\Users\tiago\AppData\Roaming\Claude\local-agent-mode-sessions\e9d07b8f-8b36-4151-9675-df4d5410645a\cec1aa73-8902-4292-97c8-7e05531718df\local_0c2f2061-0a01-4e2f-8a93-c04db8facce1\uploads\Assignment 2.pdf</file_path><file_uuid>825e0308-242d-4ef3-a501-faa7f9cca7ed</file_uuid></file>
</uploaded_files>
```

```
[assistant] (called ToolSearch)
[assistant] (called ToolSearch)
[assistant] (called ToolSearch)
[assistant] (called TaskCreate)
[assistant] (called TaskCreate)
[assistant] (called TaskCreate)
[assistant] (called TaskCreate)
[assistant] (called TaskUpdate)
[assistant] (called Skill)
[user] Base directory for this skill: C:\Users\tiago\AppData\Roaming\Claude\local-agent-mode-sessions\skills-plugin\cec1aa73-8902-4292-97c8-7e05531718df\e9d07b8f-8b36-4151-9675-df4d5410645a\skills\docx
```

DOCX creation, editing, and analysis

Overview

A .docx file is a ZIP archive containing XML files.

Quick Reference

Task	Approach
Read/analyze content	Use `pandoc` or unpack for raw XML
Create new document	Use `docx-js` - see Creating New Documents below
Edit existing document	Unpack → edit XML → repack - see Editing Existing Documents below

Converting .doc to .docx

Legacy `.doc` files must be converted before editing:

```
```bash
python scripts/office/soffice.py --headless --convert-to docx document.doc
```
```

Reading Content

```

```bash
Text extraction with tracked changes
pandoc --track-changes=all document.docx -o output.md

Raw XML access
python scripts/office/unpack.py document.docx unpacked/
```

### Converting to Images

```bash
python scripts/office/soffice.py --headless --convert-to pdf document.docx
pdftoppm -jpeg -r 150 document.pdf page
```

### Accepting Tracked Changes

To produce a clean document with all tracked changes accepted (requires LibreOffice):

```bash
python scripts/accept_changes.py input.docx output.docx
```

---

## Creating New Documents

Generate .docx files with JavaScript, then validate. Install: `npm install -g docx`

### Setup
```javascript
const { Document, Packer, Paragraph, TextRun, Table, TableRow, TableCell, ImageRun,
 Header, Footer, AlignmentType, PageOrientation, LevelFormat, ExternalHyperlink,
 InternalHyperlink, Bookmark, FootnoteReferenceRun, PositionalTab,
 PositionalTabAlignment, PositionalTabRelativeTo, PositionalTabLeader,
 TabStopType, TabStopPosition, Column, SectionType,
 TableOfContents, HeadingLevel, BorderStyle, WidthType, ShadingType,
 VerticalAlign, PageNumber, PageBreak } = require('docx');

const doc = new Document({ sections: [{ children: [/ * content */] }] });
Packer.toBuffer(doc).then(buffer => fs.writeFileSync("doc.docx", buffer));
```

### Validation
After creating the file, validate it. If validation fails, unpack, fix the XML, and repack.
```bash
python scripts/office/validate.py doc.docx
```

### Page Size
```javascript
// CRITICAL: docx-js defaults to A4, not US Letter
// Always set page size explicitly for consistent results
sections: [{
 properties: {

```

```

 page: {
 size: {
 width: 12240, // 8.5 inches in DXA
 height: 15840 // 11 inches in DXA
 },
 margin: { top: 1440, right: 1440, bottom: 1440, left: 1440 } // 1 inch margins
 }
 },
 children: [/* content */]
}]
\`\`\`

```

**\*\*Common page sizes (DXA units, 1440 DXA = 1 inch):\*\***

Paper	Width	Height	Content Width (1" margins)
US Letter	12,240	15,840	9,360
A4 (default)	11,906	16,838	9,026

**\*\*Landscape orientation:\*\*** docx-js swaps width/height internally, so pass portrait dimensions and let it handle the swap:

```

\`\`\`javascript
size: {
 width: 12240, // Pass SHORT edge as width
 height: 15840, // Pass LONG edge as height
 orientation: PageOrientation.LANDSCAPE // docx-js swaps them in the XML
},
// Content width = 15840 - left margin - right margin (uses the long edge)
\`\`\`

```

**### Styles (Override Built-in Headings)**

Use Arial as the default font (universally supported). Keep titles black for readability.

```

\`\`\`javascript
const doc = new Document({
 styles: {
 default: { document: { run: { font: "Arial", size: 24 } } }, // 12pt default
 paragraphStyles: [
 // IMPORTANT: Use exact IDs to override built-in styles
 { id: "Heading1", name: "Heading 1", basedOn: "Normal", next: "Normal", quickFormat: true,
 run: { size: 32, bold: true, font: "Arial" },
 paragraph: { spacing: { before: 240, after: 240 }, outlineLevel: 0 } }, // outlineLevel required for TOC
 { id: "Heading2", name: "Heading 2", basedOn: "Normal", next: "Normal", quickFormat: true,
 run: { size: 28, bold: true, font: "Arial" },
 paragraph: { spacing: { before: 180, after: 180 }, outlineLevel: 1 } },
]
 },
 sections: [{
 children: [
 new Paragraph({ heading: HeadingLevel.HEADING_1, children: [new TextRun("Title")] }),
]
 }]
}]);
\`\`\`

```

**### Lists (NEVER use unicode bullets)**

```

```javascript
// ■ WRONG - never manually insert bullet characters
new Paragraph({ children: [new TextRun("• Item")] }) // BAD
new Paragraph({ children: [new TextRun("\u2022 Item")] }) // BAD

// ■ CORRECT - use numbering config with LevelFormat.BULLET
const doc = new Document({
  numbering: {
    config: [
      { reference: "bullets",
        levels: [{ level: 0, format: LevelFormat.BULLET, text: "•", alignment: AlignmentType.LEFT,
          style: { paragraph: { indent: { left: 720, hanging: 360 } } } } ] },
      { reference: "numbers",
        levels: [{ level: 0, format: LevelFormat.DECIMAL, text: "%1.", alignment: AlignmentType.LEFT,
          style: { paragraph: { indent: { left: 720, hanging: 360 } } } } ] },
    ]
  },
  sections: [{
    children: [
      new Paragraph({ numbering: { reference: "bullets", level: 0 },
        children: [new TextRun("Bullet item")] }),
      new Paragraph({ numbering: { reference: "numbers", level: 0 },
        children: [new TextRun("Numbered item")] }),
    ]
  }]
});

// ■■ Each reference creates INDEPENDENT numbering
// Same reference = continues (1,2,3 then 4,5,6)
// Different reference = restarts (1,2,3 then 1,2,3)
```

```

### ### Tables

**\*\*CRITICAL: Tables need dual widths\*\*** - set both `columnWidths` on the table AND `width` on each cell. Without both, tables render incorrectly on some platforms.

```

```javascript
// CRITICAL: Always set table width for consistent rendering
// CRITICAL: Use ShadingType.CLEAR (not SOLID) to prevent black backgrounds
const border = { style: BorderStyle.SINGLE, size: 1, color: "CCCCC" };
const borders = { top: border, bottom: border, left: border, right: border };

new Table({
  width: { size: 9360, type: WidthType.DXA }, // Always use DXA (percentages break in Google Docs)
  columnWidths: [4680, 4680], // Must sum to table width (DXA: 1440 = 1 inch)
  rows: [
    new TableRow({
      children: [
        new TableCell({
          borders,
          width: { size: 4680, type: WidthType.DXA }, // Also set on each cell
          shading: { fill: "D5E8F0", type: ShadingType.CLEAR }, // CLEAR not SOLID
          margins: { top: 80, bottom: 80, left: 120, right: 120 }, // Cell padding (internal, not added to width)
          children: [new Paragraph({ children: [new TextRun("Cell")] })]
        })
      ]
    })
  ]
})
```

```

```

]
 })
}
})
\`\`\`

```

**\*\*Table width calculation:\*\***

Always use `WidthType.DXA` – `WidthType.PERCENTAGE` breaks in Google Docs.

```

\`\`\`javascript
// Table width = sum of columnWidths = content width
// US Letter with 1" margins: 12240 - 2880 = 9360 DXA
width: { size: 9360, type: WidthType.DXA },
columnWidths: [7000, 2360] // Must sum to table width
\`\`\`

```

**\*\*Width rules:\*\***

- **\*\*Always use `WidthType.DXA`\*\*** – never `WidthType.PERCENTAGE` (incompatible with Google Docs)
- Table width must equal the sum of `columnWidths`
- Cell `width` must match corresponding `columnWidth`
- Cell `margins` are internal padding - they reduce content area, not add to cell width
- For full-width tables: use content width (page width minus left and right margins)

### Images

```

\`\`\`javascript
// CRITICAL: type parameter is REQUIRED
new Paragraph({
 children: [new ImageRun({
 type: "png", // Required: png, jpg, jpeg, gif, bmp, svg
 data: fs.readFileSync("image.png"),
 transformation: { width: 200, height: 150 },
 altText: { title: "Title", description: "Desc", name: "Name" } // All three required
 })]
})
\`\`\`

```

### Page Breaks

```

\`\`\`javascript
// CRITICAL: PageBreak must be inside a Paragraph
new Paragraph({ children: [new PageBreak()] })

// Or use pageBreakBefore
new Paragraph({ pageBreakBefore: true, children: [new TextRun("New page")] })
\`\`\`

```

### Hyperlinks

```

\`\`\`javascript
// External link
new Paragraph({
 children: [new ExternalHyperlink({
 children: [new TextRun({ text: "Click here", style: "Hyperlink" })],
 link: "https://example.com",

```

```

 }]]
 })

 // Internal link (bookmark + reference)
 // 1. Create bookmark at destination
 new Paragraph({ heading: HeadingLevel.HEADING_1, children: [
 new Bookmark({ id: "chapter1", children: [new TextRun("Chapter 1")] }),
]})
 // 2. Link to it
 new Paragraph({ children: [new InternalHyperlink({
 children: [new TextRun({ text: "See Chapter 1", style: "Hyperlink" })],
 anchor: "chapter1",
 })]}])
}]]})
\`

```

### ### Footnotes

```

\`javascript
const doc = new Document({
 footnotes: {
 1: { children: [new Paragraph("Source: Annual Report 2024")] },
 2: { children: [new Paragraph("See appendix for methodology")] },
 },
 sections: [{
 children: [new Paragraph({
 children: [
 new TextRun("Revenue grew 15%"),
 new FootnoteReferenceRun(1),
 new TextRun(" using adjusted metrics"),
 new FootnoteReferenceRun(2),
],
 })],
 }]]
});
\`

```

### ### Tab Stops

```

\`javascript
// Right-align text on same line (e.g., date opposite a title)
new Paragraph({
 children: [
 new TextRun("Company Name"),
 new TextRun("\tJanuary 2025"),
],
 tabStops: [{ type: TabStopType.RIGHT, position: TabStopPosition.MAX }],
})

```

```

// Dot leader (e.g., TOC-style)
new Paragraph({
 children: [
 new TextRun("Introduction"),
 new TextRun({ children: [
 new PositionalTab({
 alignment: PositionalTabAlignment.RIGHT,
 relativeTo: PositionalTabRelativeTo.MARGIN,

```

```

 leader: PositionalTabLeader.DOT,
 }},
 "3",
]}),
],
})
\..

```

### ### Multi-Column Layouts

```

```javascript
// Equal-width columns
sections: [{
  properties: {
    column: {
      count: 2,          // number of columns
      space: 720,        // gap between columns in DXA (720 = 0.5 inch)
      equalWidth: true,
      separate: true,    // vertical line between columns
    },
  },
  children: [/* content flows naturally across columns */]
}]

// Custom-width columns (equalWidth must be false)
sections: [{
  properties: {
    column: {
      equalWidth: false,
      children: [
        new Column({ width: 5400, space: 720 }),
        new Column({ width: 3240 }),
      ],
    },
  },
  children: [/* content */]
}]
\..

```

Force a column break with a new section using `type: SectionType.NEXT\_COLUMN`.

Table of Contents

```

```javascript
// CRITICAL: Headings must use HeadingLevel ONLY - no custom styles
new TableOfContents("Table of Contents", { hyperlink: true, headingStyleRange: "1-3" })
\..

```

### ### Headers/Footers

```

```javascript
sections: [{
  properties: {
    page: { margin: { top: 1440, right: 1440, bottom: 1440, left: 1440 } } // 1440 = 1 inch
  },
  headers: {

```

```

    default: new Header({ children: [new Paragraph({ children: [new TextRun("Header")] })] })
  },
  footers: {
    default: new Footer({ children: [new Paragraph({
      children: [new TextRun("Page "), new TextRun({ children: [PageNumber.CURRENT] })]
    })] })
  },
  children: [/* content */]
}]
\`

```

Critical Rules for docx-js

- ****Set page size explicitly**** - docx-js defaults to A4; use US Letter (12240 x 15840 DXA) for US documents
- ****Landscape: pass portrait dimensions**** - docx-js swaps width/height internally; pass short edge as `width`, long edge as `height`, and set `orientation: PageOrientation.LANDSCAPE`
- ****Never use `\n`**** - use separate Paragraph elements
- ****Never use unicode bullets**** - use `LevelFormat.BULLET` with numbering config
- ****PageBreak must be in Paragraph**** - standalone creates invalid XML
- ****ImageRun requires `type`**** - always specify png/jpg/etc
- ****Always set table `width` with DXA**** - never use `WidthType.PERCENTAGE` (breaks in Google Docs)
- ****Tables need dual widths**** - `columnWidths` array AND cell `width`, both must match
- ****Table width = sum of columnWidths**** - for DXA, ensure they add up exactly
- ****Always add cell margins**** - use `margins: { top: 80, bottom: 80, left: 120, right: 120 }` for readable padding
- ****Use `ShadingType.CLEAR`**** - never SOLID for table shading
- ****Never use tables as dividers/rules**** - cells have minimum height and render as empty boxes (including in headers/footers); use `border: { bottom: { style: BorderStyle.SINGLE, size: 6, color: "2E75B6", space: 1 } }` on a Paragraph instead. For two-column footers, use tab stops (see Tab Stops section), not tables
- ****TOC requires HeadingLevel only**** - no custom styles on heading paragraphs
- ****Override built-in styles**** - use exact IDs: "Heading1", "Heading2", etc.
- ****Include `outlineLevel`**** - required for TOC (0 for H1, 1 for H2, etc.)

Editing Existing Documents

****Follow all 3 steps in order.****

Step 1: Unpack

```

\`bash
python scripts/office/unpack.py document.docx unpacked/
\`

```

Extracts XML, pretty-prints, merges adjacent runs, and converts smart quotes to XML entities (`“` etc.) so they survive editing. Use `--merge-runs false` to skip run merging.

Step 2: Edit XML

Edit files in `unpacked/word/`. See XML Reference below for patterns.

****Use "Claude" as the author**** for tracked changes and comments, unless the user explicitly requests use of a different name.

****Use the Edit tool directly for string replacement. Do not write Python scripts.**** Scripts introduce unnecessary complexity. The Edit tool shows exactly what is being replaced.

****CRITICAL: Use smart quotes for new content.**** When adding text with apostrophes or quotes, use XML entities to produce smart quotes:

```

\`xml

```



```
<!-- Use these entities for professional typography -->
<w:t>&#x2019;s a quote: &#x201C;Hello&#x201D;</w:t>
<!--
```

```
| Entity | Character |
|-----|-----|
| `&#x2018;` | ` ' ` (left single) |
| `&#x2019;` | ` ' ` (right single / apostrophe) |
| `&#x201C;` | ` " ` (left double) |
| `&#x201D;` | ` " ` (right double) |
```

```
**Adding comments:** Use `comment.py` to handle boilerplate across multiple XML files (text must be pre-escaped XML):
```

```
```bash
python scripts/comment.py unpacked/ 0 "Comment text with ’ and “"
python scripts/comment.py unpacked/ 1 "Reply text" --parent 0 # reply to comment 0
python scripts/comment.py unpacked/ 0 "Text" --author "Custom Author" # custom author name
```
```

Then add markers to document.xml (see Comments in XML Reference).

```
### Step 3: Pack
```

```
```bash
python scripts/office/pack.py unpacked/ output.docx --original document.docx
```
```

Validates with auto-repair, condenses XML, and creates DOCX. Use `--validate false` to skip.

```
**Auto-repair will fix:**
```

```
- `durableId` &#x2013;= 0x7FFFFFFF (regenerates valid ID)
- Missing `xml:space="preserve"` on `<w:t>` with whitespace
```

```
**Auto-repair won't fix:**
```

```
- Malformed XML, invalid element nesting, missing relationships, schema violations
```

```
### Common Pitfalls
```

```
- **Replace entire `<w:r>` elements**: When adding tracked changes, replace the whole `<w:r>...</w:r>` block with  
`<w:del>...<w:ins>...` as siblings. Don't inject tracked change tags inside a run.  
- **Preserve `<w:rPr>` formatting**: Copy the original run's `<w:rPr>` block into your tracked change runs to maintain bold, font size,  
etc.
```

```
---
```

```
## XML Reference
```

```
### Schema Compliance
```

```
- **Element order in `<wPr>`**: `<pStyle>`, `<numPr>`, `<spacing>`, `<ind>`, `<jc>`, `<rPr>`  
last  
- **Whitespace**: Add `xml:space="preserve"` to `<w:t>` with leading/trailing spaces  
- **RSIDs**: Must be 8-digit hex (e.g., `00AB1234`)
```

```
### Tracked Changes
```

```
**Insertion:**
```

```
```xml
<w:ins w:id="1" w:author="Claude" w:date="2025-01-01T00:00:00Z">
 <w:r><w:t>inserted text</w:t></w:r>
</w:ins>
```

```

```

**Deletion:**
```xml
<del w:id="2" w:author="Claude" w:date="2025-01-01T00:00:00Z">
 <w:r><w:delText>deleted text</w:delText></w:r>

```

**Inside `<del>`:`*: Use `<delText>` instead of `<t>`, and `<delInstrText>` instead of `<instrText>`.

**Minimal edits** - only mark what changes:
```xml
<!-- Change "30 days" to "60 days" -->
<w:r><w:t>The term is </w:t></w:r>
<del w:id="1" w:author="Claude" w:date="...">
 <w:r><w:delText>30</w:delText></w:r>

<w:ins w:id="2" w:author="Claude" w:date="...">
 <w:r><w:t>60</w:t></w:r>
</w:ins>
<w:r><w:t> days.</w:t></w:r>
```

**Deleting entire paragraphs/list items** - when removing ALL content from a paragraph, also mark the paragraph mark as deleted so it merges with the next paragraph. Add `<del/>` inside `<p><rPr>`:
```xml
<w:p>
 <w:pPr>
 <w:numPr>...</w:numPr> <!-- list numbering if present -->
 <w:rPr>
 <del w:id="1" w:author="Claude" w:date="2025-01-01T00:00:00Z"/>
 </w:rPr>
 </w:pPr>
 <del w:id="2" w:author="Claude" w:date="2025-01-01T00:00:00Z">
 <w:r><w:delText>Entire paragraph content being deleted...</w:delText></w:r>

</w:p>
```

Without the `<del/>` in `<p><rPr>`, accepting changes leaves an empty paragraph/list item.

**Rejecting another author's insertion** - nest deletion inside their insertion:
```xml
<w:ins w:author="Jane" w:id="5">
 <del w:author="Claude" w:id="10">
 <w:r><w:delText>their inserted text</w:delText></w:r>

</w:ins>
```

**Restoring another author's deletion** - add insertion after (don't modify their deletion):
```xml
<del w:author="Jane" w:id="5">
 <w:r><w:delText>deleted text</w:delText></w:r>

<w:ins w:author="Claude" w:id="10">

```

```
<w:r><w:t>deleted text</w:t></w:r>
</w:ins>
```
```

Comments

After running `comment.py` (see Step 2), add markers to document.xml. For replies, use `--parent` flag and nest markers inside the parent's.

****CRITICAL:** ``<w:commentRangeStart>`` and ``<w:commentRangeEnd>`` are siblings of ``<w:r>``, never inside ``<w:r>``.**

```
```xml
<!-- Comment markers are direct children of w:p, never inside w:r -->
<w:commentRangeStart w:id="0"/>
<w:del w:id="1" w:author="Claude" w:date="2025-01-01T00:00:00Z">
 <w:r><w:delText>deleted</w:delText></w:r>
</w:del>
<w:r><w:t>more text</w:t></w:r>
<w:commentRangeEnd w:id="0"/>
<w:r><w:rPr><w:rStyle w:val="CommentReference"/></w:rPr><w:commentReference w:id="0"/></w:r>

<!-- Comment 0 with reply 1 nested inside -->
<w:commentRangeStart w:id="0"/>
 <w:commentRangeStart w:id="1"/>
 <w:r><w:t>text</w:t></w:r>
 <w:commentRangeEnd w:id="1"/>
 <w:commentRangeEnd w:id="0"/>
<w:r><w:rPr><w:rStyle w:val="CommentReference"/></w:rPr><w:commentReference w:id="0"/></w:r>
<w:r><w:rPr><w:rStyle w:val="CommentReference"/></w:rPr><w:commentReference w:id="1"/></w:r>
```
```

Images

1. Add image file to `word/media/`

2. Add relationship to `word/\_rels/document.xml.rels`:

```
```xml
<Relationship Id="rId5" Type=".../image" Target="media/image1.png"/>
```
```

3. Add content type to `[Content\_Types].xml`:

```
```xml
<Default Extension="png" ContentType="image/png"/>
```
```

4. Reference in document.xml:

```
```xml
<w:drawing>
 <wp:inline>
 <wp:extent cx="914400" cy="914400"/> <!-- EMUs: 914400 = 1 inch -->
 <a:graphic>
 <a:graphicData uri=".../picture">
 <pic:pic>
 <pic:blipFill><a:blip r:embed="rId5"/></pic:blipFill>
 </pic:pic>
 </a:graphicData>
 </a:graphic>
 </wp:inline>
 </w:drawing>
```
```

Dependencies

```
- **pandoc**: Text extraction
- **docx**: `npm install -g docx` (new documents)
- **LibreOffice**: PDF conversion (auto-configured for sandboxed environments via `scripts/office/soffice.py`)
- **Poppler**: `pdftoppm` for images
[assistant] (called TaskUpdate)
[assistant] (called TaskUpdate)
[assistant] (called ToolSearch)
[assistant] (called Write)
[assistant] (called TaskUpdate)
[assistant] (called TaskUpdate)
[assistant] (called mcp__workspace__bash)
[assistant] (called mcp__workspace__bash)
[assistant] (called mcp__workspace__bash)
[assistant] (called TaskUpdate)
[assistant] (called TaskUpdate)
[assistant] [View your roadmap](computer:///C%3A%5CUsers%5Ctiago%5COneDrive%5CDocuments%5CClaude%5CProjects%5CDevOps%202026/DevOps_Assignment2_Roadmap.docx)
```

7-day plan from today (Sat 25 Apr) through Fri 1 May, with each day mapped to the Assignment 2 spec items: VPC + master EC2 → custom AMI + ALB → ASG + scaling policies → custom metrics + load testing → additional functionality → report, Excel template, and ZIP submission. Includes a risk register (account wipe, NAT costs, alarm tuning) and a marking-scheme alignment table so the plan tracks back to the 60/20/20 weighting.

[user] why when I am pasting over a text from pdf, it has gotten this way with long spaces

[assistant] That layout is from Word interpreting the PDF's invisible structure rather than the words themselves. PDFs don't store flowing text – they store each line (often each word) as a positioned object with explicit x/y coordinates. When you paste, Word tries to preserve those positions and ends up creating either a multi-column layout or fully-justified text with huge gaps to keep words sitting where the PDF placed them.

The specific things you're seeing:

The text is split into columns (left to right reads: "You may choose any web application, ideally one that relies" → then jumps to the next visual column "on a third party or chosen instances and"...) – that's because the PDF had the paragraph laid out across the page width and the paste preserved column boundaries instead of reading line-by-line.

The huge spaces between words come from full justification being pasted in along with non-breaking spaces or tab characters that the PDF used to position glyphs.

How to fix it:

The cleanest fix is to paste as plain text instead: copy from the PDF, then in Word use Ctrl+Shift+V (or Home → Paste dropdown → Keep Text Only / the "A" icon). That strips the positioning and gives you a normal paragraph.

If you've already pasted and don't want to redo it, select the broken paragraph and:

1. Ctrl+L to remove justification (left-align it)
2. Then Ctrl+A on just that paragraph and use Find & Replace (Ctrl+H) with "Use wildcards" off – find ` ` (multiple spaces) and replace with a single space. Run it a few times until nothing changes.
3. Also try Ctrl+H, find `^t` (tab) → replace with single space.

If the columns are real Word columns (not just spacing), go to Layout → Columns → One to collapse them.

For Assignment 2 specifically – since that text is the assignment brief you'll likely want to quote in your report – I'd suggest either retyping the requirement headings in your own words (which is what the spec asks for anyway: "explain in your own words") or pasting plain text and reformatting once.

```
[user] tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ ssh -1 devops2026.pem ec2-user@52.91.66.167
SSH protocol v.1 is no longer supported
[assistant] Typo – `-1` (the digit one) tells SSH to use the obsolete protocol v1. You want `-i` (the letter i), which means "identity file."
```

```

```
ssh -i devops2026.pem ec2-user@52.91.66.167
```
```

If you get a "permissions are too open" error after fixing that, the key file needs to be locked down:

```

```
chmod 400 devops2026.pem
```
```

Then retry the SSH command.

```
[user] tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ chmod 400 devops2026.pem
```

```
tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ ls -l
```

```
total 172
```

```
-rw-rw-r-- 1 tiago tiago 14878 Mar  8 08:54 [devops1.py](http://devops1.py)
```

```
-r----- 1 tiago tiago 1674 Jan 31 14:37 devops2026.pem
```

```
-rw-rw-r-- 1 tiago tiago 351 Mar 19 18:40 index.html
```

```
-rw-rw-r-- 1 tiago tiago 140630 Mar 19 18:40 logo.jpg
```

```
-rw-rw-r-- 1 tiago tiago 1247 Mar  8 08:35 [monitoring.sh](http://monitoring.sh)
```

```
-rw-rw-r-- 1 tiago tiago 136 Mar 19 18:40 tvillaca-websites.txt
```

```
tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ ssh -1 devops2026.pem ec2-user@52.91.66.167
```

```
SSH protocol v.1 is no longer supported
```

```
[assistant] Same typo – you're still passing `-1` (the digit one) instead of `-i` (the lowercase letter i).
```

Run exactly this:

```

```
ssh -i devops2026.pem ec2-user@52.91.66.167
```
```

The character right after `ssh` and the dash should be the letter ****i****, not the number ****1****. On most keyboards they look almost identical in the terminal font, which is why it's an easy slip.

```
[user] database disconnected
```

```
database connection error: MongoServerError: connect ECONNREFUSED 172.31.40.212:27017
```

```
MongooseServerSelectionError: connect ECONNREFUSED 172.31.40.212:27017
```

```
  at _handleConnectionErrors (/home/ec2-user/PlaceMark-1.0/node_modules/mongoose/lib/connection.js:1169:11)
```

```
  at NativeConnection.openUri (/home/ec2-user/PlaceMark-1.0/node_modules/mongoose/lib/connection.js:1100:11) {
```

```
  errorLabelSet: Set(0) {},
```

```
  reason: TopologyDescription {
```

```
    type: 'Single',
```

```
    servers: Map(1) { '172.31.40.212:27017' => [ServerDescription] },
```

```
    stale: false,
```

```
    compatible: true,
```

```
    heartbeatFrequencyMS: 10000,
```

```
    localThresholdMS: 15,
```

```
    setName: null,
```

```
    maxElectionId: null,
```

```
    maxSetVersion: null,
```

```
    commonWireVersion: 0,
```

```
    logicalSessionTimeoutMinutes: null
```

```
  },
```

```
  code: undefined,
```

```

cause: TopologyDescription {
  type: 'Single',
  servers: Map(1) { '172.31.40.212:27017' => [ServerDescription] },
  stale: false,
  compatible: true,
  heartbeatFrequencyMS: 10000,
  localThresholdMS: 15,
  setName: null,
  maxElectionId: null,
  maxSetVersion: null,
  commonWireVersion: 0,
  logicalSessionTimeoutMinutes: null
}
}

```

[ec2-user@ip-172-31-34-107 PlaceMark-1.0]\$

[assistant] `ECONNREFUSED` means the network packet *reached* `172.31.40.212` and the OS actively refused it – so it's not a security-group block (that would hang/time out). It's almost always one of three things on the Mongo host:

1. ****mongod isn't running.****
2. ****mongod is running but bound only to `127.0.0.1`**** – this is the Mongo default and the most common cause when ECONNREFUSED comes from a private IP.
3. ****mongod is listening on a different port.****

****Diagnose from the Mongo host (172.31.40.212):****

SSH into the Mongo instance and check:

```

```
sudo systemctl status mongod
sudo ss -tlnp | grep 27017
```

```

If `ss` shows `127.0.0.1:27017` (and nothing on `0.0.0.0:27017` or the private IP), that's #2.

****Fix #2 – bind Mongo to all interfaces:****

Edit `/etc/mongod.conf`:

```

```
sudo nano /etc/mongod.conf
```

```

Find the `net:` block and change:

```

```yaml
net:
 port: 27017
 bindIp: 127.0.0.1
```

```

to:

```

```yaml
net:
 port: 27017
 bindIp: 0.0.0.0
```

```

...

Then restart:

...

```
sudo systemctl restart mongod
sudo ss -tlnp | grep 27017
...
```

You should now see `0.0.0.0:27017`.

****Fix #1 – start mongod if it's not running:****

...

```
sudo systemctl start mongod
sudo systemctl enable mongod
...
```

If status shows it crashed, `sudo journalctl -u mongod -n 100` will show why.

****Then verify from the app host (172.31.34.107):****

...

```
nc -vz 172.31.40.212 27017
...
```

If that connects, your PlaceMark app will too. If `nc` *hangs* instead of refusing, that's now a security-group issue – open inbound 27017 on the Mongo instance's SG, allowing only the app SG (not 0.0.0.0/0).

****Security note for your assignment:**** binding to `0.0.0.0` is fine *if* the Mongo instance is in a private subnet and its SG only accepts 27017 from the app SG. Don't expose 27017 publicly. This is also a good thing to call out in your report's metric/architecture justification.

```
[user] [ec2-user@ip-172-31-40-212 ~]$ sudo systemctl status mongod
```

```
sudo ss -tlnp | grep 27017
```

```
■ mongod.service - MongoDB Database Server
   Loaded: loaded (/usr/lib/systemd/system/mongod.service; enabled; preset: disabled)
   Active: inactive (dead)
     Docs: https://docs.mongodb.org/manual
```

```
[ec2-user@ip-172-31-40-212 ~]$
```

```
[assistant] That's it – mongod is installed and enabled but not running (`Active: inactive (dead)`). Start it:
```

...

```
sudo systemctl start mongod
sudo systemctl status mongod
sudo ss -tlnp | grep 27017
...
```

After `start`, status should show `Active: active (running)` and `ss` should show something listening on 27017.

****Two things to check next:****

First, what address is it bound to? If `ss` shows `127.0.0.1:27017` only, your app on the other instance still won't connect – you'll need to edit `/etc/mongod.conf` and change `bindIp: 127.0.0.1` to `bindIp: 0.0.0.0`, then `sudo systemctl restart mongod`.

Second, the security group on this Mongo instance must allow inbound TCP 27017 from the app instance (172.31.34.107) – ideally by referencing the app's security group rather than its IP.

If `start` fails, run `sudo journalctl -u mongod -n 50 --no-pager` and paste what it says – usually it's a permissions issue on `/var/lib/mongo` or `/

```
var/log/mongodb`.
[user] [ec2-user@ip-172-31-40-212 ~]$ sudo systemctl status mongod
* mongod.service - MongoDB Database Server
   Loaded: loaded (/usr/lib/systemd/system/mongod.service; enabled; preset: disabled)
   Active: failed (Result: exit-code) since Sun 2026-04-26 11:14:44 UTC; 1min 19s ago
   Duration: 287ms
   Docs: https://docs.mongodb.org/manual
   Process: 26746 ExecStart=/usr/bin/mongod $OPTIONS (code=exited, status=14)
   Main PID: 26746 (code=exited, status=14)
   CPU: 71ms
Apr 26 11:14:43 ip-172-31-40-212.ec2.internal systemd[1]: Started mongod.service - MongoDB Database Server.
Apr 26 11:14:43 ip-172-31-40-212.ec2.internal mongod[26746]: {"t":{"$date":"2026-04-26T11:14:43.743+00:00"},"s":"I", "c":"-", "id":8991200,
"ctx":"main","msg":"Shuffling initializers","attr":{"se&gt;
Apr 26 11:14:43 ip-172-31-40-212.ec2.internal mongod[26746]: {"t":{"$date":"2026-04-26T11:14:43.750+00:00"},"s":"I", "c":"CONTROL", "id":7484500,
"ctx":"main","msg":"Environment variable MONGODB_CONFIG&gt;
Apr 26 11:14:44 ip-172-31-40-212.ec2.internal systemd[1]: mongod.service: Main process exited, code=exited, status=14/n/a
Apr 26 11:14:44 ip-172-31-40-212.ec2.internal systemd[1]: mongod.service: Failed with result 'exit-code'.
[assistant] Exit code 14 from mongod almost always means it can't write to its data directory or log file – usually because something started mongod
as root once and left files owned by root, so the `mongod` user can no longer touch them.
```

Get the real reason from the journal (the status output truncated the important line):

```
...
sudo journalctl -u mongod -n 80 --no-pager
...
```

Look for a line containing `msg":"...` near the end – it'll say something like "Unable to lock file", "DBPathInUse", "Permission denied", or "NonExistentPath".

While you wait on that, check ownership of the data and log paths:

```
...
ls -ld /var/lib/mongo /var/log/mongodb
ls -l /var/log/mongodb/
...
```

Both directories should be owned by `mongod:mongod`. If anything is owned by `root`, fix it:

```
...
sudo chown -R mongod:mongod /var/lib/mongo /var/log/mongodb
sudo systemctl start mongod
sudo systemctl status mongod
...
```

Paste the journal output and the `ls -l` results if it still fails – the exact log line will tell us whether it's permissions, a stale lock file (`/var/lib/mongo/mongod.lock`), a corrupted WiredTiger file, or something else.

```
[user] [ec2-user@ip-172-31-40-212 ~]$ sudo nano /etc/yum.repos.d/mongodb-org-8.2.repo
[ec2-user@ip-172-31-40-212 ~]$ sudo journalctl -u mongod -n 80 --no-pager
Apr 26 11:14:43 ip-172-31-40-212.ec2.internal systemd[1]: Started mongod.service - MongoDB Database Server.
Apr 26 11:14:43 ip-172-31-40-212.ec2.internal mongod[26746]: {"t":{"$date":"2026-04-26T11:14:43.743+00:00"},"s":"I", "c":"-", "id":8991200,
"ctx":"main","msg":"Shuffling initializers","attr":{"seed":2696866593}}
Apr 26 11:14:43 ip-172-31-40-212.ec2.internal mongod[26746]: {"t":{"$date":"2026-04-26T11:14:43.750+00:00"},"s":"I", "c":"CONTROL", "id":7484500,
"ctx":"main","msg":"Environment variable MONGODB_CONFIG_OVERRIDE_NOFORK == 1, overriding \"processManagement.fork\" to false"}
Apr 26 11:14:44 ip-172-31-40-212.ec2.internal systemd[1]: mongod.service: Main process exited, code=exited, status=14/n/a
Apr 26 11:14:44 ip-172-31-40-212.ec2.internal systemd[1]: mongod.service: Failed with result 'exit-code'.
[assistant] The journal isn't showing the real reason because mongod writes its detailed errors to its own log file, not to systemd. Check the log
```


directly:

```
...
sudo tail -n 80 /var/log/mongodb/mongod.log
...
```

Also useful in parallel – check ownership and the lock file:

```
...
ls -ld /var/lib/mongo /var/log/mongodb
ls -l /var/lib/mongo/ | head
ls -l /var/log/mongodb/
cat /etc/mongod.conf | grep -E "dbPath|path|bindIp|port"
...
```

The `tail` output will show the exact failure (one of these is the usual culprit):

- `Unable to lock file: /var/lib/mongo/mongod.lock` – leftover lock from an unclean shutdown. Fix: `sudo rm /var/lib/mongo/mongod.lock` then `sudo -u mongod mongod --dbpath /var/lib/mongo --repair` then start.
- `Permission denied` on dbPath or logpath – root owns files that should be `mongod:mongod`. Fix: `sudo chown -R mongod:mongod /var/lib/mongo /var/log/mongodb`.
- `NonExistentPath` – directory doesn't exist. Fix: `sudo mkdir -p /var/lib/mongo && sudo chown mongod:mongod /var/lib/mongo`.
- WiredTiger / corruption errors – usually means data dir was touched while mongod was off; `--repair` fixes most of it.

Paste the `tail` output and I'll point at the exact fix.

```
[user] ● mongod.service - MongoDB Database Server
   Loaded: loaded (/usr/lib/systemd/system/mongod.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-04-26 11:27:50 UTC; 3min 41s ago
     Docs: https://docs.mongodb.org/manual
  Main PID: 27256 (mongod)
    Memory: 103.1M
       CPU: 1.251s
    CGroup: /system.slice/mongod.service
           └─27256 /usr/bin/mongod -f /etc/mongod.conf
Apr 26 11:27:50 ip-172-31-40-212.ec2.internal systemd[1]: Started mongod.service - MongoDB Database Server.
Apr 26 11:27:50 ip-172-31-40-212.ec2.internal mongod[27256]: {"t":{"$date":"2026-04-26T11:27:50.080+00:00"},"s":"I",  "c":"-",          "id":8991200,
"ctx":"main","msg":"Shuffling initializers","attr":{"se&gt;
Apr 26 11:27:50 ip-172-31-40-212.ec2.internal mongod[27256]: {"t":{"$date":"2026-04-26T11:27:50.087+00:00"},"s":"I",  "c":"CONTROL",  "id":7484500,
"ctx":"main","msg":"Environment variable MONGODB_CONFIG&gt;
lines 1-13/13 (END)
[assistant] mongod is up. Two checks left before your PlaceMark app connects:
```

****1. Is it listening on the private IP, not just localhost?****

```
...
sudo ss -tlnp | grep 27017
...
```

If it shows `127.0.0.1:27017`, the app on `172.31.34.107` still can't reach it. Edit `/etc/mongod.conf`:

```
``yaml
net:
  port: 27017
  bindIp: 0.0.0.0
...
```

Then `sudo systemctl restart mongod`. If it already shows `0.0.0.0:27017` or `\*:27017`, you're good.

****2. From the app instance (172.31.34.107), test the connection:****

```

```
nc -vz 172.31.40.212 27017
```

```

- Connects → restart your PlaceMark app, the Mongoose error should be gone.

- Refused → check `bindIp` again.

- Hangs/times out → security group on the Mongo instance needs an inbound rule for TCP 27017 sourced from the app instance's SG (or its private IP `172.31.34.107/32` for now).

```
[user] [ec2-user@ip-172-31-40-212 ~]$ sudo ss -tlnp | grep 27017
```

```
LISTEN 0      4096      127.0.0.1:27017      0.0.0.0:*      users:(("mongod",pid=27256,fd=9))
```

[assistant] Confirmed – Mongo is only listening on `127.0.0.1:27017`, so connections from the app instance are getting refused at the kernel level.

Edit the config to bind to the private IP (or all interfaces):

```

```
sudo nano /etc/mongod.conf
```

```

Find the `net:` block and change `bindIp` from `127.0.0.1` to `0.0.0.0`:

```
```yaml
```

```
net:
```

```
 port: 27017
```

```
 bindIp: 0.0.0.0
```

```
```
```

Save (Ctrl+O, Enter, Ctrl+X), then restart and verify:

```

```
sudo systemctl restart mongod
```

```
sudo ss -tlnp | grep 27017
```

```

You should now see `0.0.0.0:27017` instead of `127.0.0.1:27017`.

Then from the app instance (172.31.34.107):

```

```
nc -vz 172.31.40.212 27017
```

```

If that connects, restart your PlaceMark app and the Mongoose error will clear.

****Important security follow-up for the assignment:**** binding to `0.0.0.0` exposes Mongo on every interface this instance has. That's only safe if (a) the instance is in a private subnet with no public IP, and (b) its security group only permits inbound 27017 from the app instance's security group – not from `0.0.0.0/0`. Worth calling out in your report.

```
[user] [ec2-user@ip-172-31-34-107 PlaceMark-1.0]$ npm run start
```

```
&gt; placemark-1.0@0.0.1 start
```

```
&gt; node src/server.js
```

```
[dotenv@17.3.1] injecting env (13) from .env -- tip: ■ run anywhere with `dotenvx run -- yourcommand`
```

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■■ load multiple .env files with { path: ['.env.local', '.env'] }
```

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■■■ suppress all logs with { quiet: true }
```

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■ auth for agents: https://vestauth.com
```

```
Server running on http://0.0.0.0:3000
```

```

database disconnected
database connection error: MongoInvalidArgumentError: Database names cannot contain the character '.'
MongoInvalidArgumentError: Database names cannot contain the character '.'
  at new Db (/home/ec2-user/PlaceMark-1.0/node_modules/mongodb/lib/db.js:96:19)
  at MongoClient.db (/home/ec2-user/PlaceMark-1.0/node_modules/mongodb/lib/mongo_client.js:399:20)
  at _setClient (/home/ec2-user/PlaceMark-1.0/node_modules/mongoose/lib/drivers/node-mongodb-native/connection.js:433:58)
  at NativeConnection.createClient (/home/ec2-user/PlaceMark-1.0/node_modules/mongoose/lib/drivers/node-mongodb-native/connection.js:344:3)
  at process.processTicksAndRejections (node:internal/process/task_queues:95:5) {
  errorLabelSet: Set(0) {}
}
[assistant] Good news – the network connection works now. The new error is just URI formatting: somewhere in your connection string, a `.` is being
interpreted as part of the database name, which Mongo doesn't allow.

```

A valid Mongo URI looks like:

```

```
mongodb://172.31.40.212:27017/placemark
```

```

The part ****after the last `/\*`** is the database name, and it can't contain dots.

Common ways this gets broken:

The ``.env`` is missing ``mongodb://`` and the IP itself becomes the path, e.g. ``db_url=172.31.40.212:27017/placemark`` – Mongoose treats the whole thing as a path and the dots in the IP trip the parser.

The dbname literally has a dot, e.g. ``mongodb://172.31.40.212:27017/placemark.db``.

There's an extra slash, e.g. ``mongodb://172.31.40.212:27017//172.31.40.212`` – the second IP becomes the dbname.

Check your ``.env``:

```

```
cat .env | grep -i -E "mongo|db_url|database"
```

```

Paste that line (you can blank out any password) and I'll spot the issue. The fix is almost certainly to make the var look like:

```

```
db_url=mongodb://172.31.40.212:27017/placemark
```

```

Then restart with ``npm run start``.

```
[user] [ec2-user@ip-172-31-34-107 PlaceMark-1.0]$ cat .env | grep -i -E "mongo|db_url|database"
```

```
#db=mongodb://localhost/placemark
```

```
#db=mongodb+srv://tvillaca:[20114785@placemarkcluster0.i0fobsj.mongodb.net](mailto:20114785@placemarkcluster0.i0fobsj.mongodb.net)/
```

```
?appName=placemarkCluster0
```

```
db=mongodb://172.31.40.212/PlaceMark-1.0?directConnection=true
```

[assistant] Found it. The database name in your URI is ``PlaceMark-1.0`` – the ``.`` in ``1.0`` is what Mongo is rejecting. Database names can't contain ``.``, ``/``, ``\``, space, ``"`,` ``$`,` ``*`,` ``<`,`>`,`:`,`|`,`` or ``?``.

Rename the database to something without a dot. Edit ``.env``:

```

```
db=mongodb://172.31.40.212:27017/placemark?directConnection=true
```

```

(I also added the `:27017` explicitly – it's the default but it makes the URI unambiguous.)

Then:

```
...  
npm run start  
...
```

The DB will be created automatically the first time the app writes to it. If you had previously seeded a database called `PlaceMark-1.0` somewhere it'll be empty – but since this is the first time the app has actually connected to this Mongo instance, that's fine.

[user] do i create a VPC first with public and private subnets or the instance first ?

[assistant] VPC first – always. An EC2 instance has to live inside a subnet, and a subnet has to live inside a VPC, so you build the network shell before anything goes into it. (AWS does give you a default VPC, but the assignment explicitly requires you to create your own, so don't use the default.)

The right build order for Assignment 2:

1. **VPC** (e.g., 10.0.0.0/16)
2. **Subnets** – at least 2 public subnets in different AZs (for the ALB and the ASG instances). Add private subnets only if you'll put a backend/DB there.
3. **Internet Gateway**, attached to the VPC
4. **Route table** for the public subnets with a default route (0.0.0.0/0) → IGW. Associate it with the public subnets. (Private subnets stay on the main route table or get their own without an IGW route.)
5. **Security groups** – at minimum a Web-SG (HTTP from ALB-SG, SSH from your IP) and an ALB-SG (HTTP from 0.0.0.0/0). You can create them now or just before the instance – either works.
6. **Master EC2 instance** – launch it into one of the public subnets, attach Web-SG, give it a public IP so you can SSH in and install your app.
7. From there: bake the AMI, then ALB → launch template → ASG → scaling policies, exactly as the roadmap lays out.

Doing it in this order means each step uses things that already exist, with no going back to fix associations.

One small but real tip: when you create the subnets, turn on "auto-assign public IP" only on the public ones. Saves headaches when you launch the master instance later.

[user] should I create 1 or 2 availability zones?

[assistant] Two – it's not really a choice for this assignment. An Application Load Balancer **requires** at least two subnets in two different Availability Zones; AWS won't let you create one with only a single AZ. Since the spec mandates an ELB, two AZs is the minimum.

Practical layout:

- **AZ A** (e.g., us-east-1a): 1 public subnet (+ 1 private subnet if you're going to put a DB or backend in private)
- **AZ B** (e.g., us-east-1b): 1 public subnet (+ 1 private subnet)

The ALB spans both public subnets, the Auto Scaling Group launches instances across both – so when CPU spikes and ASG scales out, instances land in different AZs and the ALB distributes traffic across them. That's exactly the "high availability" story the marker is looking for, and it's a clean talking point in your report ("designed for AZ failure tolerance").

You don't need three AZs for this assignment – two is the sweet spot: meets the ALB requirement, demonstrates HA, keeps the Learner Lab credit burn manageable.

[user] how to get my ip , from my hotspot?

[assistant] Quickest from the terminal:

```
...  
curl ifconfig.me  
...
```

Or any of these – same result:

```
...
```

```
curl https://api.ipify.org
curl https://icanhazip.com
curl ipinfo.io/ip
```
```

Or in a browser, just search "what is my ip" on Google.

That gives you your **public IP** – the address the internet (and AWS) sees when packets come from your laptop. That's what you want for an SSH security group rule. Add `/32`` after it in the SG (e.g., `203.0.113.45/32``) so the rule is exactly your IP, not a range.

**Heads-up about mobile hotspots:** the IP from a phone hotspot often changes – sometimes every few hours, often when you reconnect, and almost always if you change networks. Two consequences:

If SSH stops working tomorrow, your IP probably rotated. Just re-run `curl ifconfig.me`` and update the SG rule.

Some carriers use carrier-grade NAT (CGNAT), so the IP you see is shared with other customers. The rule still works for you, but it's not as tight as a fixed home/office IP.

For the assignment demo, just check your IP again the morning of and update the SG if needed – takes 30 seconds.

```
[user] Try `wget --help' for more options.
[ec2-user@ip-10-0-2-146 ~]$ wget http://ipinfo.io/ip -q0 -
wget: invalid option -- '0'
Usage: wget [OPTION]... [URL]...
Try `wget --help' for more options.
[ec2-user@ip-10-0-2-146 ~]$
[assistant] Two things going on.
```

**The typo:** that's a capital letter **O** (for "output"), not a zero. The flag is `-q0`` meaning "quiet, output to stdout":

```
```
wget -q0- http://ipinfo.io/ip
```
```

Or just use `curl``, which is simpler:

```
```
curl ifconfig.me
```
```

**The bigger issue:** you're running this *on the EC2 instance*, so the IP it returns is the **EC2 instance's** public IP, not your laptop's. That's not what you want for an SSH security group rule.

Run the command **on your own laptop** (the one you're SSHing from). On your Ubuntu VM:

```
```
curl ifconfig.me
```
```

That returns your hotspot's public IP – that's the one to put in the SG rule as `&lt;your-ip>/32``.

[user] why do I have a bastion server?

[assistant] A bastion host (also called a "jump box") is a small, hardened EC2 instance that sits in a **public subnet** and acts as the **only** SSH entry point into your VPC. Anything in your **private subnets** – your DB, internal services – has no public IP and can't be reached from the internet directly. To get to them, you SSH into the bastion first, then from the bastion you SSH into the private instance.

The reason it exists is security. If your MongoDB instance is in a private subnet (which is the pattern the assignment hints at when it says "install/integrate a database on EC2 instance(s) in private subnet(s)"), you have two choices:

1. Give the DB instance a public IP and open SSH to your laptop. Now your DB is reachable from the internet, and one misconfigured SG rule means it's exposed to attackers.
2. Keep the DB in a private subnet with no public IP. The only way in is through the bastion. The DB never appears on the public internet at all. Attack surface drops to a single small server you can patch and lock down.

The bastion pattern looks like:

```

...
Your laptop ■■SSH■■■ Bastion (public subnet, public IP, SSH from your IP only)
 ■
 ■■■SSH■■■ DB / app servers (private subnets, no public IP)
...
```

In SG terms: the bastion's SG accepts SSH from your laptop's IP. The private instances' SGs accept SSH **only** from the bastion's SG – not from your IP, not from `0.0.0.0/0`.

**\*\*For Assignment 2 specifically:\*\*** the bastion is worth real marks. The spec lists "use one or more security services" and "install/integrate a database on EC2 instance(s) in private subnet(s)" as additional functionality options – putting Mongo in a private subnet behind a bastion lines up with both. In the report, you can call it out as a defence-in-depth choice: the database is unreachable from the internet by design, the bastion is the single hardened ingress point, and SG-to-SG references mean even if the bastion's IP changes, the rule still works.

Practical tip when SSHing through it: use `ssh -J ec2-user@<bastion-ip>; ec2-user@<private-ip>` (the `-J` flag does the jump for you in one command, no need to copy keys onto the bastion).

```
[user] [ec2-user@ip-10-0-2-146 ~]$ ssh ec2-user@10.0.133.194
The authenticity of host '10.0.133.194 (10.0.133.194)' can't be established.
ED25519 key fingerprint is SHA256:oaKkbPKQCzjFFEOW2MyaL6rUun5kWYfJH60IDdyWvI.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.133.194' (ED25519) to the list of known hosts.
ec2-user@10.0.133.194: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[ec2-user@ip-10-0-2-146 ~]$
[assistant] You SSH'd in without a key, and EC2 instances only accept key-based auth – that's why it bounced you with "Permission denied (publickey)."
```

The trap with bastions is that the private key (`devops2026.pem`) lives on your laptop, not on the bastion. So when you try to jump from the bastion onward, there's no key for SSH to present. You've got two clean ways to fix this – **don't** copy the .pem onto the bastion (that's the insecure shortcut).

**\*\*Option 1 – ProxyJump from your laptop (simplest, my recommendation):\*\***

Run this from your laptop, not from the bastion:

```

...
ssh -i devops2026.pem -J ec2-user@<bastion-public-ip>; ec2-user@10.0.133.194
...
```

The `-J` flag tells SSH to bounce through the bastion automatically. The key stays on your laptop, your laptop authenticates to **both** hops, and you land on the private instance in one command.

**\*\*Option 2 – SSH agent forwarding:\*\***

On your laptop:

```

...
ssh-add devops2026.pem
ssh -A ec2-user@<bastion-public-ip>;
...
```

...

...

```
[user] exit
```

Let me know when you're ready to keep going on the assignment.

[illegible]

```
[ec2-user@ip-10-0-2-146 ~]$ ssh ec2-user@10.0.133.194
```

```
[ec2-user@ip-10-0-2-146 ~]$
```

**\*\*Quick fix – `ProxyJump` (no agent setup needed):\*\***

...

● ● ●

```
Or - load the key into the agent so -A- actually has something to forward:
```

...

551  
552

Then on the bastion:

```
ssh-add -l # should show the same key, forwarded from your laptop
ssh ec2-user@10.0.133.194
```

If `ssh-add -l` on the bastion says "no identities" or "could not open a connection to your authentication agent," forwarding isn't working – usually because the key wasn't added on the laptop side, or because `eval \$(ssh-agent)` didn't run.

For the assignment, ProxyJump is the cleanest pattern to demonstrate and document – it's a single command, no key copying, no agent state to manage. Worth a one-liner in the report explaining the secure access path.

```
[user] [dotenv@17.3.1] injecting env (13) from .env -- tip: ■ run anywhere with `dotenvx run -- yourcommand`
[dotenv@17.3.1] injecting env (0) from .env -- tip: prevent committing .env to code: https://dotenvx.com/precommit
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■ auth for agents: https://vestauth.com
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■ run anywhere with `dotenvx run -- yourcommand`
Server running on http://0.0.0.0:3000
New user object: tiago13@gmail.com
database disconnected
database connection error: MongoServerError: Socket 'connect' timed out after 30006ms (connectTimeoutMS: 30000)
MongoServerError: Socket 'connect' timed out after 30006ms (connectTimeoutMS: 30000)
 at _handleConnectionErrors (/home/ec2-user/PlaceMark-1.0/node_modules/mongoose/lib/connection.js:1169:11)
 at NativeConnection.openUri (/home/ec2-user/PlaceMark-1.0/node_modules/mongoose/lib/connection.js:1100:11) {
 errorLabelSet: Set(0) {},
 reason: TopologyDescription {
 type: 'Single',
 servers: Map(1) { '10.0.133.194:27017' => [ServerDescription] },
 stale: false,
 compatible: true,
 heartbeatFrequencyMS: 10000,
 localThresholdMS: 15,
 setName: null,
 maxElectionId: null,
 maxSetVersion: null,
 commonWireVersion: 0,
 logicalSessionTimeoutMinutes: null
 },
 code: undefined,
 cause: TopologyDescription {
 type: 'Single',
 servers: Map(1) { '10.0.133.194:27017' => [ServerDescription] },
 stale: false,
 compatible: true,
 heartbeatFrequencyMS: 10000,
 localThresholdMS: 15,
 setName: null,
 maxElectionId: null,
 maxSetVersion: null,
 commonWireVersion: 0,
 logicalSessionTimeoutMinutes: null
 }
}
[ec2-user@ip-10-0-8-52 PlaceMark-1.0]$
```

[assistant] Different error this time, and that's the diagnostic clue. Last time you got `ECONNREFUSED` (Mongo wasn't listening). Now you get a

**\*\*timeout\*\*** – packets aren't getting through to the host at all. That's almost always a security group, not a Mongo, problem.

This also looks like a fresh VPC (10.0.0.0/16) with the app at 10.0.8.52 and Mongo at 10.0.133.194 in a private subnet. So we're starting the SG



rules from scratch in the new VPC.

**\*\*First, confirm it's the SG (10 seconds):\*\***

From the app instance:

```
```\nnc -vz 10.0.133.194 27017\n```\n\n- **Hangs / "Operation timed out"** → SG or network issue (most likely)\n- ***"Connection refused"*** → got through to the host, but Mongo isn't bound right (back to the previous problem)
```

****Fix 1 – Mongo instance's security group:****

In the AWS console, go to the Mongo instance → Security → click its security group. Add an inbound rule:

- Type: Custom TCP
- Port: 27017
- Source: the ****app instance's security group ID**** (start typing ``sg-`` and pick from the list). Don't use `0.0.0.0/0`.

That's the most likely missing piece. Save it and retry – no instance restart needed.

****Fix 2 – confirm Mongo is bound to the right interface (in case ``nc`` shows refused):****

SSH to Mongo (via your bastion) and check:

```
```\nsudo ss -tlnp | grep 27017\n```\n
```

Should show ``0.0.0.0:27017``. If it shows ``127.0.0.1:27017``, edit ``/etc/mongod.conf`` and set ``bindIp: 0.0.0.0``, then ``sudo systemctl restart mongod``. (You did this before – but if this is a freshly built Mongo instance in the new VPC, the default config is back.)

**\*\*Fix 3 – ``env`` sanity check:\*\***

```
```\ngrep ^db= .env\n```\n
```

It should be:

```
```\ndb=mongodb://10.0.133.194:27017/placemark?directConnection=true\n```\n
```

– matching the **\*\*new\*\*** Mongo's private IP, not the old ``172.31.x.x`` one.

Run the ``nc`` test first and tell me what it says – that'll narrow it to one of the three above.

[assistant] That's the app instance's security group (it has port 3000 – the PlaceMark app port – and SSH 22). The one that needs the rule is the **\*\*Mongo instance's\*\*** security group, not this one.

**\*\*To find Mongo's SG:\*\***

1. EC2 console → Instances
2. Click on your Mongo instance (the one at 10.0.133.194)

3. Bottom panel → **Security** tab
4. Click the security group ID under "Security groups"
5. **Edit inbound rules** → **Add rule**

Add:

- Type: **Custom TCP**
- Port range: **27017**
- Source: **Custom** → start typing ``sg-`` and pick the **app instance's security group** from the list (the one in your screenshot – ``sgr-...`` is the rule ID, you want the parent group ID, which will look like ``sg-0xxxxxxxxxxxx``).

Save the rule, then re-run from the app instance:

```
```\nnc -vz 10.0.133.194 27017\n```
```

It should now connect. Restart your PlaceMark app and the Mongoose timeout will be gone.

Two side notes worth fixing for the report later:

The SSH rule on this app SG is ``0.0.0.0/0`` – anyone on the internet can attempt SSH. For the demo, tighten that to your laptop's ``/32`` or to the bastion's SG if you're going through one.

Port 3000 open to ``0.0.0.0/0`` is fine while you're testing, but the final architecture should have the **ALB** in front of the app, with port 3000 only allowed from the ALB's SG. That's part of the load-balancer story for the marker.
[assistant] This still isn't the Mongo SG – port 3000 is the PlaceMark app port. This SG looks like an app-tier rule set (allowing port 3000 from a "WebServer" SG and SSH from BastionSG). The Mongo SG should be the one attached to the instance running mongod, and the rule we need is **port 27017**, not 3000.

To make sure you're editing the right SG, do this:

1. EC2 console → **Instances** (left menu)
2. In the instance list, find the row where **Private IPv4 address = 10.0.133.194** (your Mongo instance)
3. Click that instance to select it
4. Bottom panel → **Security** tab
5. Under "Security groups" you'll see something like ``sg-0abc... / SomeName`` – click that SG link
6. **Inbound rules** tab → **Edit inbound rules** → **Add rule**:
 - Type: **Custom TCP**
 - Port range: **27017**
 - Source: type ``sg-`` and pick the **app's SG** (the one with port 3000 open from earlier – likely named something like ``WebServerSG``)
7. Save

If when you click into the Mongo instance's Security tab the SG name comes back as something you've already shown me, tell me which one – we'll edit that. But port 27017 must end up on whatever SG is attached to the instance at 10.0.133.194.

Quick way to verify which SG is the Mongo one without clicking around: from the app instance, run:

```
```\nnc -vz 10.0.133.194 27017\n```
```

If it times out, the SG isn't right yet. After you add the 27017 rule on the correct SG, that command should connect immediately – no instance restart needed.

[user] i have created a new VPC, hence New Instances.

[assistant] Got it – clean slate. That changes things: nothing carries over from the default VPC. Here's a focused checklist for the new setup so we

can pinpoint where the timeout is coming from.

**\*\*On the new Mongo instance (10.0.133.194):\*\***

Is mongod even installed yet? Last time we set it up on a different instance. From the bastion, jump to it and check:

```
```
```

```
sudo systemctl status mongod
```

```
```
```

- "Unit not found" → Mongo isn't installed on this instance. Install it (Amazon Linux 2023 + MongoDB 8.x repo), then `sudo systemctl enable --now mongod`.
- "active (running)" → check binding: `sudo ss -tlnp | grep 27017` – needs to be `0.0.0.0:27017`, not `127.0.0.1:27017`. If it's localhost only, edit `/etc/mongod.conf` → `bindIp: 0.0.0.0` → `sudo systemctl restart mongod`.
- "inactive (dead)" or "failed" → `sudo tail -50 /var/log/mongodb/mongod.log` will show why.

**\*\*Security group on the Mongo instance:\*\***

Find the SG attached to `10.0.133.194` (EC2 → Instances → click instance → Security tab → click the SG link). Add inbound:

- Type: Custom TCP, Port: 27017, Source: the **\*\*app instance's SG\*\*** (the WebServer SG you showed earlier)

Also add SSH inbound from the BastionSG so you can keep reaching it via the bastion.

**\*\*On the app instance (10.0.8.52):\*\***

`.env` is now pointing at the new Mongo IP?

```
```
```

```
grep ^db= .env
```

```
```
```

Should be:

```
```
```

```
db=mongodb://10.0.133.194:27017/placemark?directConnection=true
```

```
```
```

**\*\*Diagnostic in one line:\*\***

From the app instance:

```
```
```

```
nc -vz 10.0.133.194 27017
```

```
```
```

The result tells us exactly where to look:

- **\*\*Times out\*\*** → SG rule on Mongo's SG missing (most likely right now)
- **\*\*Connection refused\*\*** → SG is fine, but mongod isn't running or is bound to 127.0.0.1
- **\*\*Connects\*\*** → restart `npm run start` and you're done

Run the `nc` test and paste the result – that'll tell us which thing to fix first.

[assistant] That's the right one – **\*\*DBServerMongod\*\*** (sg-0e879c8c6bbd7526c) in the new VPC `vpc-0dae25bea32533c16`. The other "Mongodb" and "node" SGs are leftovers from your old VPC; ignore them.

For PlaceMark to reach Mongo, this SG needs two inbound rules:

**\*\*Click into DBServerMongod → Inbound rules → Edit inbound rules → Add rule:\*\***

**\*\*Rule 1 – let the app talk to Mongo:\*\***

- Type: **\*\*Custom TCP\*\***
- Port range: **\*\*27017\*\***
- Source: **\*\*Custom\*\*** → start typing `sg-` and pick **\*\*WebServerNode\*\*** (`sg-08b8070aaf5ecf2f4`)
- Description (optional): `Mongo from app tier`

**\*\*Rule 2 – let you SSH via the bastion (if not already there):\*\***

- Type: **\*\*SSH\*\***
- Port range: **\*\*22\*\***
- Source: **\*\*Custom\*\*** → pick **\*\*BastionSG\*\*** (`sg-067f9411d1d6584d0`)
- Description: `SSH from bastion only`

Save rules.

Then from the app instance (10.0.8.52):

```
...
nc -vz 10.0.133.194 27017
...
```

It should connect immediately – no instance restart needed for SG changes.

Once that connects, restart your PlaceMark app:

```
...
npm run start
...
```

The Mongoose timeout will be gone. If `nc` still times out, double-check that **\*\*10.0.133.194 actually has DBServerMongod attached\*\*** (EC2 → Instances → click Mongo instance → Security tab – the listed SG should be DBServerMongod). If it's still on `default`, swap it via Actions → Security → Change security groups.

[user] like this now?

[assistant] Yes – that's exactly right. Both rules look correct:

- **\*\*27017\*\*** from `sg-08b8070aaf5ecf2f4` (WebServerNode – your app instance) ✓
- **\*\*22\*\*** from `sg-067f9411d1d6584d0` (BastionSG) ✓

Click **\*\*Save rules\*\***, then from the app instance test it:

```
...
nc -vz 10.0.133.194 27017
...
```

Should connect immediately. Then `npm run start` and the Mongoose timeout should be gone.

One small thing worth doing now since you'll need it for the report: hit "Edit" again later and add a description to each rule (e.g., "Mongo from app tier", "SSH from bastion only"). Future-you and the marker both benefit.

[user] what is wrong?

i have inserted this code into User DATA and now it is nor start by itself

```
#!/bin/bash
```

```
su - ec2-user -c 'cd PlaceMark-1.0; npm run start'
```

[assistant] That User Data has a few problems baked into it. The good news: this is exactly the kind of thing the assignment wants you to nail down before baking the AMI.

**\*\*What's wrong with the script:\*\***

User Data runs as **root** very early in boot, with a minimal `PATH` and no shell environment loaded. Several things go wrong simultaneously:

`npm` isn't on root's `PATH` if Node was installed via `nvm` – `nvm` only loads in interactive shells, so `su - ec2-user -c '...'` may not find `npm` even though it works fine when you SSH in.

`npm run start` runs in the **foreground** and never returns. User Data treats that as the script hanging; when User Data eventually times out or the parent exits, your app dies with it.

There's no `nohup` / no detachment, no log redirection, so even if it started you'd have no way to see output.

It runs only on the first boot by default – it won't restart your app if the instance reboots or the process crashes.

**The right fix for an auto-scaling AMI: use a systemd service**

This is the pattern your marker will be looking for, because the AMI has to boot into a working app on every new instance the ASG launches. SSH into the instance and create a service file:

```
...
sudo nano /etc/systemd/system/placemark.service
...
```

Paste:

```
...ini
[Unit]
Description=PlaceMark Node.js App
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=ec2-user
WorkingDirectory=/home/ec2-user/PlaceMark-1.0
ExecStart=/bin/bash -lc 'npm run start'
Restart=on-failure
RestartSec=5
StandardOutput=append:/var/log/placemark.log
StandardError=append:/var/log/placemark.log

[Install]
WantedBy=multi-user.target
...
```

The `bash -lc` part loads `ec2-user`'s login shell so `nvm/Node/npm` are on `PATH`.

Then enable and start it:

```
...
sudo touch /var/log/placemark.log
sudo chown ec2-user:ec2-user /var/log/placemark.log
sudo systemctl daemon-reload
sudo systemctl enable placemark
sudo systemctl start placemark
sudo systemctl status placemark
...
```

Verify it serves traffic:

```
```
curl -I http://localhost:3000
tail -f /var/log/placemark.log
```
```

Now reboot the instance to prove it really starts on boot:

```
```
sudo reboot
```
```

Wait, SSH back in, run `systemctl status placemark` – if it's `active (running)` after a clean reboot with no user intervention, your AMI will work in the ASG.

**\*\*Empty out the broken User Data\*\*** for now – the systemd service does the job and User Data is unreliable for this. Later, when you bake the AMI, the service is already on the AMI, so every instance the ASG launches will start the app automatically.

**\*\*To debug what your current User Data did\*\*** (good for the report):

```
```
sudo cat /var/log/cloud-init-output.log
```
```

That shows exactly what happened when User Data ran – usually you'll see "npm: command not found" or similar, which is the smoking gun for the PATH issue.

[user] \_\_[Scaling a Node App - Starting your App or Database automatically](https://tutors.dev/lab/setu-hdip-comp-sci-2025-devops/topic-10-Security/unit-2-node-deployment/book-1-Deploying-a-node-app/4-Scaling%20a%20Node%20app#scaling-a-node-app---starting-your-app-or-database-automatically)\_\_

When creating the Launch Template for scaling you can specify a startup script (expand Advanced details and enter your script under User data). The following would start the sample app.

```
```
#!/bin/bash
su - ec2-user -c 'cd playtime; npm run start'
```
```

Note that this can also be done when launching instances manually.

help me with this ?

[assistant] The course pattern absolutely works, and since this is what your launch template will use later, let's get that working rather than swap to systemd. The script needs three things to be true at runtime; let's check them in order.

**\*\*The script:\*\***

```
```bash
#!/bin/bash
su - ec2-user -c 'cd PlaceMark-1.0; npm run start'
```
```

Note: `su -` (with the dash) runs a **\*\*login shell\*\***, which sources `~/.bash_profile` / `~/.bashrc`. That's where nvm gets loaded – so ``npm`` **\*should\*** be on PATH inside that subshell. The course pattern relies on that.

**\*\*Step 1 – Did User Data even run?\*\***

SSH into the instance (via bastion) and check:

```
```
```

```
sudo cat /var/log/cloud-init-output.log
```
```

Scroll to the bottom. You'll see one of:

- "npm: command not found" → nvm isn't being sourced for ec2-user (Step 2 fix).
- "no such file or directory" → the PlaceMark folder isn't where the script expects (Step 3 fix).
- App startup logs (the Hapi/Express banner, or your "Server running on http://0.0.0.0:3000" line) → it's actually running, the User Data is just blocking on it (which is fine).

**\*\*Step 2 – Confirm npm works for ec2-user via login shell:\*\***

This is the exact thing User Data is doing. As your laptop user via the bastion, jump to the app instance, then run:

```
```
sudo su - ec2-user -c 'which npm && node --version'
```
```

If that prints a path and a version, the course script will work. If you get "command not found", nvm isn't loading via login shell. The fix on Amazon Linux is to make sure `~/.bashrc` or `~/.bash\_profile` includes the nvm load line:

```
```
[ -s "$HOME/.nvm/nvm.sh" ] && \. "$HOME/.nvm/nvm.sh"
```
```

Or install Node from the system package instead of nvm so it's on the global PATH (simpler on Learner Lab images):

```
```
sudo dnf install -y nodejs npm
which npm      # should be /usr/bin/npm – works for any user, no nvm needed
```
```

**\*\*Step 3 – Confirm the app folder is in `/home/ec2-user/PlaceMark-1.0`:\*\***

```
```
ls /home/ec2-user/
```
```

Should show `PlaceMark-1.0/`. If you cloned it elsewhere, the `cd PlaceMark-1.0` in the User Data won't find it.

**\*\*Step 4 – Test the User Data script manually:\*\***

Before relying on first-boot, prove the *exact* command works while you're SSH'd in:

```
```
sudo bash -c 'su - ec2-user -c "cd PlaceMark-1.0; npm run start"'
```
```

If you get "Server running on http://0.0.0.0:3000", that's perfect – it'll work in User Data too. Ctrl+C to stop, then either reboot to retest User Data, or just bake the AMI with this validated.

**\*\*Quick sanity check right now:\*\***

While the instance is still booting (it shows "Initializing"), check whether port 3000 is actually serving:

```
```
```

```
curl -I http://10.0.8.52:3000
```

from the bastion. If you get a response, the app is up and User Data did its job – the "Initializing" status check just means AWS hasn't finished its automated probes yet (takes ~2 minutes).

Run `sudo cat /var/log/cloud-init-output.log` and paste the last 30 lines – that'll tell us exactly which of the three things is wrong.

```
[user] tiago@tiago-Virtual-Machine:~$ nano ~/.aws/credentials
tiago@tiago-Virtual-Machine:~$ eval $(ssh-agent)
ssh-add devops2026.pem
ssh-add -l          # confirm the key is now loaded
Agent pid 5687
devops2026.pem: No such file or directory
The agent has no identities.
tiago@tiago-Virtual-Machine:~$ cd Documents/DevOps/DevOps1
tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ eval $(ssh-agent)
Agent pid 5691
tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ ssh-add devops2026.pem
Identity added: devops2026.pem (devops2026.pem)
tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ ssh-add -l          # confirm the key is now loaded
2048 SHA256:lmD4M03NdTtuNc2tqzGTFfqr0cZpnbtzy7uGzNLVL6E devops2026.pem (RSA)
tiago@tiago-Virtual-Machine:~/Documents/DevOps/DevOps1$ ssh ec2-user@44.202.31.53
The authenticity of host '44.202.31.53 (44.202.31.53)' can't be established.
ED25519 key fingerprint is SHA256:ElKDo+jyQEY7w0ViS/f4WAJWHGgqesoMiE8oXbHiaTM.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:110: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.202.31.53' (ED25519) to the list of known hosts.
```

```

#_
~\  #####_      Amazon Linux 2023
~~ \#####\
~~  \###|
~~   \#/  _      https://aws.amazon.com/linux/amazon-linux-2023
~~    V~'  '-&gt;
~~~~
~~.._./_/_/
~~/_/_'/_/
~~/_m/'_/_/
```

Last login: Sun Apr 26 15:37:01 2026 from 89.19.88.119

```
[ec2-user@ip-10-0-8-52 ~]$ ls
PlaceMark-1.0  node_modules  package-lock.json  package.json
[ec2-user@ip-10-0-8-52 ~]$ cd PlaceMark-1.0
[ec2-user@ip-10-0-8-52 PlaceMark-1.0]$ npm run start
> placemark-1.0@0.0.1 start
> node src/server.js
[dotenv@17.3.1] injecting env (13) from .env -- tip: prevent committing .env to code: https://dotenvx.com/precommit
[dotenv@17.3.1] injecting env (0) from .env -- tip: prevent committing .env to code: https://dotenvx.com/precommit
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■ agentic secret storage: https://dotenvx.com/as2
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■■ specify custom .env file path with { path: '/custom/path/.env' }
Server running on http://0.0.0.0:3000
database connected to Placemark on 10.0.133.194
Loading dashboard for user: [tiago20@gmail.com](mailto:tiago20@gmail.com)
New route object: {
  title: 'ergerg',
  createdAt: 2026-04-27T16:18:59.503Z,
  createdBy: new ObjectId('69ee381fc50d8edb121eeabf')
```



```
}
Loading dashboard for user: [tiago20@gmail.com](mailto:tiago20@gmail.com)
[assistant] Everything is working – the app started, connected to Mongo at 10.0.133.194, and you've already logged in and created a route. So your stack is fully functional. Now we just need to make sure it starts automatically on every fresh instance the ASG launches. Since it works when you run it manually, the User Data script will work on a new instance baked from this state.
```

****Let's test that exact thing before baking the AMI:****

Stop the running app (Ctrl+C), then run the User Data command exactly as it'll execute on first boot – but in the background so you can watch logs:

```
...
sudo bash -c 'su - ec2-user -c "cd PlaceMark-1.0; npm run start"' &&
...
```

Wait 5-10 seconds, then verify it's serving:

```
...
curl -I http://localhost:3000
...
```

If you get a `200 OK` (or any HTTP response), the User Data approach is proven to work. Kill it (`fg` then Ctrl+C, or `pkill node`) and move to baking the AMI.

****Now the AMI + Launch Template flow:****

1. ****Stop the app**** (so nothing is mid-write when the AMI is baked):

```
...
pkill node
...
```

2. ****Stop the instance**** in the AWS console (Instance state → Stop instance). Wait until it's `stopped`.

3. ****Create the AMI****: right-click instance → Image and templates → Create image. Give it a name like `PlaceMark-app-v1`. Wait until the AMI status is `available`.

4. ****Create the launch template**** referencing this AMI. In ****Advanced details → User data****, paste:

```
...`bash
#!/bin/bash
su - ec2-user -c 'cd PlaceMark-1.0; npm run start'
...
```

5. ****Test the launch template**** by launching ONE instance from it (the launch template details page → Actions → Launch instance from template). Don't wire it to the ASG yet – just one test instance in your public subnet with the WebServerNode SG.

6. SSH in via the bastion and check `cloud-init-output.log` and port 3000:

```
...
sudo cat /var/log/cloud-init-output.log | tail -30
curl -I http://localhost:3000
...
```

If port 3000 responds without you doing anything else, the AMI is good and you're ready to wire it into the Auto Scaling Group.

****Two small things worth handling now while everything is fresh:****

The `.env` file is currently baked into the AMI with the Mongo private IP `10.0.133.194` hardcoded. That's fine for this assignment, but worth a sentence in your report about how a production version would inject DB connection details via Secrets Manager / SSM Parameter Store rather than

baking secrets into the AMI.

Restart the actual instance once before baking, and confirm the app comes back via User Data on a real boot, not just the manual test. That catches edge cases like file permissions or environment differences.

You're past the hardest part – the app, network, and DB are all talking. From here it's launch template → ASG → CloudWatch alarms → load testing, exactly as days 3-5 of the roadmap.

```
[user] [ec2-user@ip-10-0-8-52 PlaceMark-1.0]$ sudo bash -c 'su - ec2-user -c "cd PlaceMark-1.0; npm run start"' &&
```

```
[1] 3848
```

```
[ec2-user@ip-10-0-8-52 PlaceMark-1.0]$
```

```
&gt; placemark-1.0@0.0.1 start
```

```
&gt; node src/server.js
```

```
[dotenv@17.3.1] injecting env (13) from .env -- tip: ■ agentic secret storage: https://dotenvx.com/as2
```

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: prevent building .env in docker: https://dotenvx.com/prebuild
```

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: encrypt with Dotenvx: https://dotenvx.com
```

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: ■■ suppress all logs with { quiet: true }
```

```
Server running on http://0.0.0.0:3000
```

```
database connected to Placemark on 10.0.133.194
```

```
[assistant] That's the proof – the exact User Data command works and the app is up and connected to Mongo. You're ready to bake the AMI.
```

****Verify port 3000 responds (in the same shell):****

```
...
```

```
curl -I http://localhost:3000
```

```
...
```

You should see `HTTP/1.1 200 OK` (or 302 if it redirects to login). If yes, the User Data pattern is fully validated.

****Then clean up the test process:****

```
...
```

```
sudo pkill -f "node src/server.js"
```

```
...
```

Or bring it to the foreground and Ctrl+C:

```
...
```

```
fg
```

```
# then Ctrl+C
```

```
...
```

****Next: bake the AMI****

1. From the AWS console, ****stop the instance**** (don't terminate). Wait until state = `stopped`.

2. Right-click the instance → ****Image and templates**** → ****Create image****. Name it something memorable: `PlaceMark-app-v1` (or include date).

3. Wait for AMI status = `available` (check EC2 → AMIs in the left menu – usually 2-5 minutes).

****Then: create the launch template****

EC2 → Launch Templates → Create launch template:

- AMI: pick `PlaceMark-app-v1`

- Instance type: `t2.nano` or `t3.nano`

- Key pair: `devops2026`

- Network: don't pin a subnet here (let the ASG choose)

- Security group: `WebServerNode`

```
- IAM instance profile: pick one with CloudWatch `PutMetricData` permission (you'll need this for Day 5's custom metrics)
- **Advanced details → User data** (paste exactly):
  ```bash
 #!/bin/bash
 su - ec2-user -c 'cd PlaceMark-1.0; npm run start'
  ```
```

****Smoke-test the launch template once before wiring to an ASG:****

From the launch template's detail page → ****Actions → Launch instance from template**** → 1 instance in a public subnet of the new VPC. SSH into it via the bastion (or check the public IP directly) and:

```
```
sudo tail -50 /var/log/cloud-init-output.log
curl -I http://localhost:3000
```
```

If port 3000 responds without any manual intervention, the AMI is bullet-proof. Terminate that test instance, then create the Auto Scaling Group from the launch template.

[user] I am not using local host

[assistant] If you're SSH'd into the app instance itself (your prompt was `[ec2-user@ip-10-0-8-52 PlaceMark-1.0]\$`), then `localhost` is correct – it just means "the machine I'm currently logged into," which is the app instance. The app binds to `0.0.0.0` so localhost on its own loopback works fine.

If you've since logged out and are testing from somewhere else, swap the address based on where you are:

****From the bastion (or any other instance in the VPC):****

```
```
curl -I http://10.0.8.52:3000
```
```

****From your laptop, going through the bastion:****

First open an SSH tunnel from your laptop:

```
```
ssh -L 3000:10.0.8.52:3000 ec2-user@<bastion-public-ip>;
```
```

Then in a browser/another terminal on your laptop:

```
```
curl -I http://localhost:3000
```
```

(Here `localhost` is *your laptop*, and the tunnel forwards to the app instance.)

****From your laptop directly (only works if the app instance has a public IP and your laptop's IP is in WebServerNode SG for port 3000):****

```
```
curl -I http://<app-public-ip>;3000
```
```

Where are you running the test from? If you tell me, I'll give you the exact command.

[user] gotcha ! just wait then.

[assistant] Sounds good – take your time. I'll be here when you're ready for the next step.

[user] forget the AMI template. do I need to start my app automacally ?

[assistant] Yes – this is essential for the assignment, not optional. Here's why:

The whole point of the Auto Scaling Group is that AWS spins up brand new instances for you when load gets high. There's no human SSHing in to type `npm run start` on those new instances. If the app doesn't start by itself when the instance boots, here's what happens:

1. ASG launches a new instance from your launch template.

2. The OS boots fine. Instance status check: ✓ healthy.
3. ALB sends a health check to port 3000. Nothing answers.
4. ALB marks the target as ****unhealthy****, never sends traffic to it.
5. ASG sees an unhealthy target, may even terminate the instance and try again – same result.

End state: the auto-scaling demonstration breaks. Spec items #4, #5, and #6 all depend on new instances actually serving traffic.

****Two ways to make it auto-start, both acceptable:****

The User Data + AMI approach we already validated – every new instance from the launch template runs the script on first boot. That's the path the course teaches and what the launch template was made for.

A systemd service baked into the AMI – slightly more robust (restarts on crash, logs to journal) but the User Data path is simpler and matches the course material.

Either way, the requirement is the same: ****a fresh instance, with no human intervention, must end up serving the app on port 3000****. That's the bar the marker (and the ALB health check) will hold you to.

So if you're stepping back from the AMI flow, just know that whatever you do instead has to deliver the same outcome: app comes up automatically when the instance does.

[user] Ok , I did wrong then, I put the user data into my Placemark Web server. I need to create a launch template and put the user there. is it an AMI, or another instance ? what is the relation of it to Load Balancing ?

[assistant] Good question – these pieces have confusing names. Let me lay out what each one actually is and how they connect.

****The four things and what they really are:****

****Instance**** = a running virtual server. You can SSH into it. It has CPU, RAM, an IP. Your `10.0.8.52` is an instance.

****AMI (Amazon Machine Image)**** = a **frozen snapshot** of an instance's disk. It's not running, you can't SSH into it. It's the "save file" you use to clone identical instances later. Think of it like a system image: every new instance launched from this AMI starts with the exact same files, packages, and configuration as the moment you froze it.

****Launch template**** = a **recipe** for launching new instances. It says: "When AWS needs to spin up an instance, use AMI X, this instance type, this security group, this key pair, this user-data script." It's not an instance and not an image – it's just the configuration. The launch template doesn't run anything by itself.

****Auto Scaling Group (ASG)**** = a **manager** that uses the launch template to keep N instances alive, scaling up and down based on rules. ASG → reads launch template → launches instances.

****Load balancer (ALB)**** = receives traffic from users and forwards it to whichever instances the ASG currently has running. The ASG registers its instances into the ALB's "target group" automatically as it scales.

****How they connect – the chain:****

...

You build & test a "master" instance



You bake an AMI (frozen image of it)



You create a Launch Template that points at that AMI
(and includes the User Data script to start the app)



You create an Auto Scaling Group that uses that Launch Template

■
▼
You attach the ASG to an ALB Target Group

■
▼
Users hit the ALB → ALB distributes traffic across
whichever instances the ASG currently has running
...

****Where User Data fits:****

User Data on the *master instance* runs once, on that one instance's first boot. After that, it's done. So putting it on the master accomplishes nothing for scaling – by the time the ASG launches new instances, those new instances are running their *own* first boot and need *their own* User Data.

The User Data has to live on the **launch template**. Then every new instance the ASG creates runs the script as part of its first boot, starts the app, and becomes ready to serve traffic.

****The relationship to load balancing:****

The ALB doesn't care about your master instance, your AMI, or your launch template directly. It only knows about *target groups* – lists of instances to send traffic to. The chain is:

- ALB sends traffic → Target group → instances registered in it
- ASG controls *which* instances are in the target group (it adds new ones when scaling out, removes them when scaling in)
- ASG creates those instances using the launch template
- The launch template tells each instance "use this AMI, run this User Data"
- The AMI was baked from your master instance

So everything ultimately traces back to your master, but the ALB only ever sees the instances the ASG produces.

****Practical answer to your question:****

Yes, you need an AMI – that's the image baked from your current working master. The launch template references that AMI. Putting User Data on a regular running instance does nothing useful for scaling; it has to be on the launch template so it runs on every fresh instance the ASG creates.

The good news: the User Data you tested already works. So the steps are exactly:

1. Stop your master instance.
2. Create AMI from it.
3. Create launch template (AMI + User Data + SG + key pair).
4. Test by launching one instance from the template – confirm app comes up on port 3000 with no manual intervention.
5. Create ASG using that launch template, attached to an ALB target group.

[user] so to make it start automatically, I need the AMI first. is it ?

[assistant] Not quite – the AMI by itself doesn't start anything. The AMI is just a frozen copy of your disk; it has no behavior. What actually triggers the app to start automatically is one of two things, and they sit at different layers:

****Option 1 – Auto-start lives inside the AMI (systemd service):****

You configure a systemd service on your master instance now (`/etc/systemd/system/placemark.service`, enabled). When you bake the AMI, that service config is captured in the image. Every new instance launched from that AMI inherits the enabled service, so on boot, systemd notices the service and starts the app – no User Data needed.

****Option 2 – Auto-start lives in the launch template (User Data):****

The AMI just contains the code, dependencies, and `.env`. The launch template carries the User Data script. Every new instance launched from the

template runs that script on first boot, which starts the app.

Both options achieve the same thing: a fresh instance comes up serving on port 3000 with no human typing `npm run start`. Pick one, not both. The course teaches Option 2 (User Data on the launch template), so it's the simpler path for the assignment.

****So the full picture:****

The AMI is the **vehicle** that carries your code, dependencies, and config to every new instance the ASG creates. It's necessary because new instances need **something** to boot from, and that something has to already have your app installed. But the AMI on its own doesn't start your app – User Data (or a baked-in systemd service) does.

****The order matters:****

1. Make sure your master instance has everything the new instances need: code at `/home/ec2-user/PlaceMark-1.0`, npm/node working, `.env` file with the right Mongo IP, all dependencies installed (`npm install` already done).
2. Bake the AMI from this master.
3. Create the launch template with User Data: `#!/bin/bash` then `su - ec2-user -c 'cd PlaceMark-1.0; npm run start'`.
4. Test by launching one instance from the template – port 3000 should respond automatically.

So yes, ****you need the AMI first**** – but the auto-start magic happens at step 3 when you put User Data on the launch template, not in the AMI itself. [user] I mean, I did every thing right till now. I just need to adjust the sequence to get it working automatically. because I thought I can get it starting automatically now, but no, It is part of process creating an AMI, and then create the launch template. [assistant] Exactly right – you've got the mental model now. Everything you've built so far is correct and in the right place; nothing needs to be undone.

To clarify one small thing: technically you **can** make your current master instance auto-start its app right now (with a systemd service, for example). But that's only useful for **that one instance**. The assignment needs auto-start on ****every**** instance the ASG launches in the future, and those instances don't exist yet – they'll be born from the launch template. So the auto-start mechanism has to ride along **with** the launch template, which is why the order is:

1. ■ Master instance built and tested (you're here)
2. ■■ Bake the AMI (freezes everything: code, dependencies, config)
3. ■■ Create the launch template, with User Data set to start the app
4. ■■ Smoke-test by launching one instance from the template → confirm port 3000 comes up by itself
5. ■■ Create the ASG attached to the ALB target group

So your master instance doesn't need to auto-start anything – its job is just to be a working snapshot you bake into the AMI. The auto-start happens **later**, on the new instances the launch template produces.

You're in great shape. Stop the master, bake the AMI, and the rest is downstream.

[user] 7. Use of your own script to collect custom metrics on your auto scaled instances and push these to CloudWatch. For example this could be web server or other server logs, or OS activity (CPU, memory, disk, number of processes, etc).

should I do it before creating an AMI ?

[assistant] Yes – and it's a smart instinct to ask. Putting the metrics script on the master **before** baking the AMI is the right order, for two reasons:

****Why before the AMI:****

The whole point of custom metrics is collecting data from **every** auto-scaled instance, not just the master. If the script is baked into the AMI, every new instance the ASG launches inherits it and starts pushing its own metrics on boot. If you bake the AMI without the script, you'd either need to re-bake later or install the script manually on every new instance (which defeats auto-scaling).

The marker is looking for metrics that include something instance-specific (like the instance ID or hostname as a CloudWatch dimension), so they can see two or three different instances reporting concurrently. That story only works if the script ships with the AMI.

****The right sequence:****

1. Write the script on the master (bash + AWS CLI, or Python + boto3).
2. Schedule it – cron is simplest:

```
crontab -e
* * * * * /home/ec2-user/push_metrics.sh &gt;&gt; /home/ec2-user/metrics.log 2&gt;&1
...
```

(every minute)

3. Run it manually once to verify CloudWatch receives the metric. Check the AWS console → CloudWatch → Metrics → All metrics → your custom namespace.
4. ****Once that's confirmed working, bake the AMI.****
5. Launch a test instance from the launch template, wait 2 minutes, check CloudWatch – there should now be metrics from the **new** instance ID as well.

****One thing the AMI alone doesn't carry – IAM permissions:****

To call `cloudwatch:PutMetricData`, the instance needs an IAM instance profile attached. That gets configured on the ****launch template****, not in the AMI. In the Learner Lab, use the existing `LabInstanceProfile` (or the role that's already permitted) – pick it under "Advanced details → IAM instance profile" in the launch template.

Test that part now too: when you run your script on the master, it'll only succeed if the master itself has an IAM role attached. If you get an `AccessDeniedException`, attach the lab profile to the master via Actions → Security → Modify IAM role.

****A starter script (good template for the report):****

```
``bash
#!/bin/bash
NAMESPACE="PlaceMark/Custom"
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
REGION=$(curl -s http://169.254.169.254/latest/meta-data/placement/region)

MEM_USED=$(free | awk '/Mem:/ {printf("%.2f", $3/$2 * 100)}')
DISK_USED=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
PROCS=$(ps -e --no-headers | wc -l)

aws cloudwatch put-metric-data --region "$REGION" --namespace "$NAMESPACE" \
  --metric-name MemoryUsedPercent --value "$MEM_USED" --unit Percent \
  --dimensions InstanceId="$INSTANCE_ID"

aws cloudwatch put-metric-data --region "$REGION" --namespace "$NAMESPACE" \
  --metric-name DiskUsedPercent --value "$DISK_USED" --unit Percent \
  --dimensions InstanceId="$INSTANCE_ID"

aws cloudwatch put-metric-data --region "$REGION" --namespace "$NAMESPACE" \
  --metric-name ProcessCount --value "$PROCS" --unit Count \
  --dimensions InstanceId="$INSTANCE_ID"
...
```

Save as `/home/ec2-user/push_metrics.sh`, `chmod +x`, run it once, check CloudWatch, then schedule via cron, **then** bake.

The `InstanceId` dimension is the part that makes this work for the marker – different instances show up as separate data series in CloudWatch automatically.

[user] this is a script from my college

```
#!/bin/bash TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-meta data-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254 /latest/meta-data/instance-id)
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f\t", $3*100/$2 }')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_80=$(netstat -an | grep 80 | wc -l)
IO_WAIT=$(iostat | awk 'NR==4 {print $5}')
aws cloudwatch put-metric-data --metric-name memory-usage --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $USEDMEMORY
aws cloudwatch put-metric-
```

```
data --metric-name Tcp_connections --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN aws cloudwatch put-metric-data --metric-
name TCP_connection_on_port_80 --dimension s Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN_PORT_80 aws cloudwatch put-metric-data --
metric-name IO_WAIT --dimensions Instance=$INSTAN CE_ID --namespace "Custom" --value $IO_WAIT
```

use it as a base , and give me suggestions on what to add to it

[assistant] Good starting point. Let me flag the issues with the original and then give you an upgraded version.

****Issues with the script as-is:****

`netstat` is deprecated on Amazon Linux 2023 – install it with `sudo dnf install -y net-tools` or replace with `ss`. Worth replacing because the marker can criticise outdated tooling.

`iostat` needs the `sysstat` package – not installed by default on AL2023. Without it the script silently records empty values. `sudo dnf install -y sysstat`.

Port ****80**** is the wrong port for your app – PlaceMark serves on ****3000****. Unless you put nginx in front on 80, that metric will always be zero. Either change to 3000 or leave 80 and document that it's for future nginx fronting.

No `--region` flag on the `aws` calls. Works if the instance has a default region in `~/.aws/config`, but for a script that runs from cron on a fresh instance there's no guarantee. Pull the region from instance metadata.

No error handling – if the metadata token call fails, every metric posts as empty.

The original includes `free -m` printing a tab character – purely cosmetic, but `awk '{printf "%.2f", ...}'` without `\t` is cleaner.

****What I'd add for this assignment specifically:****

The spec literally lists examples: "web server or other server logs, or OS activity (CPU, memory, disk, number of processes, etc)." Hitting more of those gives the report richer material. My recommended additions:

- ****Disk used %**** – explicitly named in the spec, trivial to compute
- ****Process count**** – explicitly named in the spec
- ****Node process count**** – proves **your app** is running, not just any process
- ****Load average (1-minute)**** – classic OS health signal, often more telling than CPU%
- ****App request count in last minute**** – parsed from the PlaceMark log (this is the "web server logs" angle the spec mentions)

That puts you across both buckets the spec calls out (OS activity + server logs), which is a clean talking point in the report ("I covered both example categories").

****Improved script:****

```
```bash
#!/bin/bash
/home/ec2-user/push_metrics.sh
Pushes custom metrics to CloudWatch every minute.
Categories covered: OS activity (memory, disk, processes, load) + app log (request count).

set -u

--- IMDSv2: get a session token, then query metadata ---
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id)
REGION=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/placement/region)
```



```

NAMESPACE="PlaceMark/Custom"
DIM="Instance=$INSTANCE_ID"

--- OS metrics ---
USEDMEMORY=$(free | awk '/Mem:/ {printf "%.2f", $3*100/$2}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", "", $5); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
NODE_COUNT=$(pgrep -c node || echo 0)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)

--- Network metrics (ss replaces deprecated netstat) ---
TCP_CONN=$(ss -tan | tail -n +2 | wc -l)
TCP_CONN_APP=$(ss -tan state established '(sport = :3000 or dport = :3000)' | tail -n +2 | wc -l)

--- App log metric: requests in the last minute (adjust path as needed) ---
APP_LOG="/var/log/placemark.log"
if [[-f "$APP_LOG"]]; then
 ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
 REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
else
 REQ_LAST_MIN=0
fi

--- Push to CloudWatch ---
push() {
 aws cloudwatch put-metric-data \
 --region "$REGION" --namespace "$NAMESPACE" \
 --dimensions "$DIM" \
 --metric-name "$1" --value "$2" --unit "$3"
}

push MemoryUsedPercent "$USEDMEMORY" Percent
push DiskUsedPercent "$DISK_USED" Percent
push ProcessCount "$PROC_COUNT" Count
push NodeProcessCount "$NODE_COUNT" Count
push LoadAverage1Min "$LOAD_1M" None
push TcpConnections "$TCP_CONN" Count
push TcpConnectionsApp "$TCP_CONN_APP" Count
push RequestsLastMinute "$REQ_LAST_MIN" Count

```

**\*\*To install on the master before baking:\*\***

```

```bash
sudo dnf install -y sysstat net-tools      # only if you keep iostat/netstat
chmod +x /home/ec2-user/push_metrics.sh
/home/ec2-user/push_metrics.sh             # test run; check CloudWatch console
crontab -e
# add this line:
* * * * * /home/ec2-user/push_metrics.sh && /home/ec2-user/metrics.log 2>&&1
```

```

Wait 2 minutes, then check **\*\*CloudWatch → Metrics → All metrics → PlaceMark/Custom\*\***. You should see eight metric series tagged with the master's instance ID.

**\*\*Once that works on the master, bake the AMI.\*\*** Every ASG instance will then push its own metrics under its own ``Instance=...`` dimension – and you

can graph them side-by-side in CloudWatch as evidence in the report (that's a great screenshot for spec item #7).

**\*\*One justification line for the report:\*\***

The script hits both example categories the spec calls out (OS activity *and* server logs), and the `InstanceId`` dimension lets each auto-scaled instance report independently – so when the ASG scales out, you see new data series appear in CloudWatch automatically.

[user] give me a sequence to implement it now

[assistant] Here's a tight sequence. Each step has a verification – don't move on until that step's check passes.

**\*\*Step 1 – SSH into the master via bastion:\*\***

```
```bash
ssh -A ec2-user@<bastion-public-ip>
ssh ec2-user@10.0.8.52
```
```

**\*\*Step 2 – Confirm the master has an IAM role that allows PutMetricData:\*\***

```
```bash
aws sts get-caller-identity
```
```

- Returns an account/ARN → IAM role attached, good.

- "Unable to locate credentials" → in AWS console: EC2 → select the master → Actions → Security → Modify IAM role → pick `LabInstanceProfile`` (or whatever the Learner Lab role is). Then re-run.

**\*\*Step 3 – Install missing tools:\*\***

```
```bash
sudo dnf install -y sysstat net-tools
which aws ss free df ps awk
```
```

All should return paths.

**\*\*Step 4 – Make sure the app log path matches reality:\*\***

The script reads `~/var/log/placemark.log``, but currently your app logs to the terminal. Pick one of:

- **\*\*Quickest:\*\*** edit the script's `APP_LOG=`` line to point wherever your app actually logs (or skip the `RequestsLastMinute`` metric).

- **\*\*Better:\*\*** redirect the app's stdout to that file when it's started. If you're keeping the User Data approach, change it to:

```
```bash
#!/bin/bash
su - ec2-user -c 'cd PlaceMark-1.0; npm run start &&& /var/log/placemark.log 2&&&1'
sudo touch /var/log/placemark.log
sudo chown ec2-user:ec2-user /var/log/placemark.log
```
```

**\*\*Step 5 – Create the metrics script:\*\***

```
```bash
nano /home/ec2-user/push_metrics.sh
```
```

Paste the script from the previous reply. Save (Ctrl+O, Enter, Ctrl+X).

```
```bash
```

```
chmod +x /home/ec2-user/push_metrics.sh
```
```

**\*\*Step 6 – Run it once manually and check exit code:\*\***

```
```bash
/home/ec2-user/push_metrics.sh
echo "exit=$?"
```
```

`exit=0` means all `aws cloudwatch put-metric-data` calls succeeded. Anything else → run with debug:

```
```bash
bash -x /home/ec2-user/push_metrics.sh &&1 | tail -40
```
```

The trace shows which command failed (usually missing IAM permission or a tool that doesn't exist).

**\*\*Step 7 – Verify metrics actually arrived in CloudWatch:\*\***

In AWS console: **\*\*CloudWatch → Metrics → All metrics\*\***. You should see a custom namespace called **\*\*PlaceMark/Custom\*\***. Click it → you'll see your 8 metric names with the master's `InstanceId` dimension. Pick one (e.g., `MemoryUsedPercent`) and graph it – there should be a single data point at the time you ran the script.

If "PlaceMark/Custom" doesn't appear after 1–2 minutes, the script ran but the AWS call failed silently. Re-run with `bash -x`.

**\*\*Step 8 – Schedule via cron (every 1 minute):\*\***

```
```bash
crontab -e
```
```

Add this single line:

```
```
* * * * * /home/ec2-user/push_metrics.sh &&1; /home/ec2-user/metrics.log &&1
```
```

Save and exit. Confirm it's installed:

```
```bash
crontab -l
```
```

**\*\*Step 9 – Wait 3 minutes, verify cron is running it:\*\***

```
```bash
tail /home/ec2-user/metrics.log
sudo journalctl -u crond --since "5 minutes ago" | grep push_metrics
```
```

The journal line shows cron firing the script; the `metrics.log` should be empty if the script succeeded silently (or contain stderr if anything broke).

In CloudWatch, refresh the graph from Step 7 – you should now see 3 data points, one per minute. **\*\*That's your "spec item #7 done" evidence – screenshot the graph for the report.\*\***

**\*\*Step 10 – Only now: stop the instance and bake the AMI.\*\***

Once the metrics are flowing reliably for ~3 minutes, you're ready. Stop the master in the console, then create the AMI. Every new instance launched from this AMI will inherit the cron job and start pushing its own metrics under its own `InstanceId` automatically – that's the screenshot that earns the marks for #7 ("metrics from multiple auto-scaled instances visible side-by-side").

**\*\*While you wait between steps\*\***, write the metric-justification paragraph for the report. Just two sentences: which metrics you chose, why they matter for an auto-scaled web app, and that the `InstanceId` dimension means each ASG instance reports independently.

[user] I would use the bastion to access the webserver and the database. is it ? my webserver and the bastion are in the public subnet so I dont see the need to use the bastion there. could you explain ?

[assistant] Sharp observation – and you're right that *\*technically\** nothing prevents you from SSHing into the web server directly if it has a public IP. But the bastion pattern isn't really about whether the target is in a public or private subnet. It's about **\*\*where SSH ingress is allowed to come from\*\***. Two different things.

**\*\*The two questions are independent:\*\***

"Public subnet" only means the subnet has a route to the Internet Gateway – i.e., its instances can reach the internet outbound, and *\*can\** have a public IP if you give them one. It doesn't say anything about who can reach them inbound. Inbound is controlled entirely by the security group.

So an instance can absolutely sit in a public subnet with a public IP, but with an SG that says "no SSH from the internet – only from BastionSG." That's the recommended pattern.

**\*\*Why route SSH through the bastion even for public-subnet instances:\*\***

If the web server's SG allows SSH from `0.0.0.0/0` (or even just from your laptop's IP), then port 22 is exposed to the internet on every web server instance. Multiply that by the ASG creating 2-4 instances at any time and you have multiple SSH endpoints to keep locked down, patched, and monitored.

If the web server's SG only allows SSH from the **\*\*bastion's SG\*\***, then:

- The internet sees only ports 80/443 (or 3000) on web servers – no SSH endpoint to attack.
- The bastion is your *\*single\** SSH entry into the whole VPC. One small, hardened, easy-to-audit instance.
- When the ASG scales out, every new instance automatically inherits "SSH only from bastion" – no manual SG fiddling per instance.
- If you ever want to disable SSH access entirely for the demo, you stop the bastion and SSH disappears across the board.

That's the security-marks angle for your report: **\*\*defence in depth – minimise public attack surface\*\***, even on instances that have public IPs by necessity.

**\*\*So for your two backends:\*\***

**\*\*Mongo (private subnet, no public IP):\*\*** you *\*must\** use the bastion. There is no other route – the instance has no public-facing presence at all. Its SG should allow SSH only from BastionSG and Mongo (27017) only from WebServerNode SG.

**\*\*Web server (public subnet, public IP, behind ALB later):\*\*** you *\*can\** SSH directly, but you *\*should\** route through the bastion. Tighten WebServerNode SG so SSH (22) is sourced from **\*\*BastionSG only\*\***, not from your laptop's IP. The web/app port (3000 today, or whatever the ALB health-checks) stays open as needed – but only that, nothing else.

**\*\*What this looks like in your report:\*\***

You can frame it as a single principle: "All administrative SSH access flows through a single hardened bastion in the public subnet. Application servers expose only their service ports to the ALB; database servers expose only their service port to the application servers. No instance other than the bastion accepts SSH from the internet." One paragraph, one diagram arrow, real marks under "use of security services" and document quality.

**\*\*Practical change to make right now:\*\***

1. Edit `WebServerNode` SG → change SSH (22) source from your laptop IP (or `0.0.0.0/0`, if it's that) to `BastionSG`.
2. Test: from your laptop, `ssh ec2-user@10.0.8.52` should now fail. From the bastion, it should still work.

That single change is worth a sentence in the report about deliberately removing direct internet SSH access from app servers.  
[assistant] That error is an AWS UI quirk, not a permissions or logic issue. AWS treats two kinds of "source" – CIDR ranges (like `0.0.0.0/0`) and security group references (like `sg-...`) – as fundamentally different rule types, and it won't let you \*swap\* one for the other on an existing rule. You have to delete the old rule and add a new one.

**\*\*Do this:\*\***

1. **\*\*Delete\*\*** the current SSH rule (the row with `sgr-071777b34c155645d` – click the **\*\*Delete\*\*** button on that row).
2. Click **\*\*Add rule\*\***.
3. New row:
  - Type: **\*\*SSH\*\***
  - Port: **\*\*22\*\*** (auto-fills)
  - Source: **\*\*Custom\*\***
  - In the search box, type `sg-` and pick **\*\*BastionSG\*\*** from the dropdown (don't paste the ID into the box that's complaining)
4. Save rules.

That will create the SSH-from-BastionSG rule cleanly.

**\*\*While you're here, also tighten the 3000 rule:\*\***

Right now port 3000 is open to `0.0.0.0/0`. For the final architecture (with the ALB in front), you'll want this changed to **\*\*source = the ALB's security group\*\***. Don't worry about that yet – leave 3000 open so you can curl the app directly while testing. Just remember to lock it down before the demo, when the ALB is in place.

After saving, verify:

- From your laptop: `ssh ec2-user@10.0.8.52` should now **\*\*fail / time out\*\*** (good – internet SSH is closed).
- From the bastion: `ssh ec2-user@10.0.8.52` should still work (with agent forwarding, no `-i` needed).

[user] it is not accepting the I pick the bastion  
[assistant] That's the same rule from before (`sgr-071777b34c155645d`). Even though you've cleared the source field, AWS still remembers that this rule was originally created with a CIDR source, so it won't let you switch its source type to a security group reference. The error you saw earlier was the symptom of this – clearing the field didn't reset the rule's underlying type.

You have to fully delete and recreate it:

**\*\*1. Delete the SSH rule:\*\***

On the SSH row (`sgr-071777b34c155645d`), click the **\*\*Delete\*\*** button on the right side of that row. The row should disappear or get crossed out. Then click **\*\*Save rules\*\*** at the bottom – this commits the deletion.

**\*\*2. Now add a new SSH rule:\*\***

Click **\*\*Edit inbound rules\*\*** again → **\*\*Add rule\*\***:

- Type: **\*\*SSH\*\***
- Port: **\*\*22\*\*** (auto-fills)
- Source: **\*\*Custom\*\***
- In the source search, type `sg`, then click **\*\*BastionSG\*\*** from the dropdown
- Save rules

Because this is a brand-new rule, AWS will accept the SG reference cleanly – no error.

**\*\*Why the two-step is required:\*\*** AWS doesn't expose rule mutation as "change source type"; it only allows delete + recreate. The UI doesn't make that obvious. Same trap catches everyone once.

[user] [ec2-user@ip-10-0-2-146 ~]\$ ssh ec2-user@54.145.135.97

^X^C

[ec2-user@ip-10-0-2-146 ~]\$ ^C

```
[ec2-user@ip-10-0-2-146 ~]$
It is not working
[assistant] You're trying to SSH to the web server's **public IP** (`54.145.135.97`) from the bastion. That's the trap.
```

Here's what's happening: when you SSH from inside the VPC to another instance's *public* IP, AWS routes that traffic out through the Internet Gateway and back in. The web server's security group sees the source as the **\*\*bastion's public IP\*\***, not the bastion's SG membership. Your new rule "SSH from BastionSG" doesn't match a public-IP source, so the packet is dropped → connection hangs.

**\*\*Fix: SSH to the web server's private IP.\*\***

```
...
ssh ec2-user@10.0.8.52
...
```

When you use the private IP, traffic stays *inside* the VPC, the source is the bastion's private interface, AWS recognizes it as coming from BastionSG, and the rule matches.

**\*\*Why this works the way it does:\*\***

Security group rules that reference another SG (e.g., "allow from BastionSG") only match when AWS can identify the source as belonging to that SG – which only happens for **\*\*VPC-internal\*\*** traffic. Public-IP traffic is treated as coming from the internet, even if both endpoints are in the same VPC. So the rule of thumb is:

- **\*\*Same VPC → use private IP\*\*** (SG references work, traffic is free, no IGW hop)
- **\*\*Across VPCs / from outside → use public IP\*\*** (need CIDR-based rules, traffic goes via IGW)

For your laptop → bastion you must use the bastion's public IP (you're outside the VPC). For bastion → web server / Mongo you should always use the private IP.

```
[user] [ec2-user@ip-10-0-2-146 ~]$ ssh ec2-user@54.145.135.97
^X^C
[ec2-user@ip-10-0-2-146 ~]$ ^C
[ec2-user@ip-10-0-2-146 ~]$ ssh ec2-user@10.0.8.52
The authenticity of host '10.0.8.52 (10.0.8.52)' can't be established.
ED25519 key fingerprint is SHA256:ElKDo+jyQEY7w0ViS/f4WAJWHGgqesoMiE8oXbHiaTM.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.8.52' (ED25519) to the list of known hosts.
ec2-user@10.0.8.52: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[ec2-user@ip-10-0-2-146 ~]$
[assistant] Good news – the SG is working now. "Permission denied (publickey)" means SSH reached the host this time (no timeout), but no key was offered. So agent forwarding isn't carrying your key through to the bastion.
```

**\*\*Quick check on the bastion:\*\***

```
...
ssh-add -l
...
```

- "no identities" or "could not open a connection to your authentication agent" → the key isn't being forwarded from your laptop. You probably SSH'd to the bastion this time without ``-A``.
- Lists ``devops2026.pem`` → agent forwarding is fine, but the web server doesn't have your key in its ``authorized_keys``. (Less likely since the same AMI/keypair was used.)

**\*\*Fix: re-SSH to the bastion with agent forwarding enabled.\*\***

Exit back to your laptop:

```
```  
exit  
```
```

On your laptop, make sure the agent has your key:

```
```  
ssh-add -l  
```
```

If empty, re-add it:

```
```  
ssh-add ~/Documents/DevOps/DevOps1/devops2026.pem  
```
```

Then SSH to the bastion with ``-A``:

```
```  
ssh -A ec2-user@<bastion-public-ip>  
```
```

Confirm the key forwarded:

```
```  
ssh-add -l  
```
```

Should now show your key. Then:

```
```  
ssh ec2-user@10.0.8.52  
```
```

Should land you on the web server.

**\*\*Or – skip the two-hop manual chain and use ProxyJump from your laptop in one command:\*\***

```
```  
ssh -J ec2-user@<bastion-public-ip> ec2-user@10.0.8.52  
```
```

That bounces through the bastion automatically and is what I'd use day-to-day. It also means you only need the key on your laptop, never on the bastion.

```
[user] <uploaded_files>
<file><file_path>C:\Users\tiago\AppData\Roaming\Claude\local-agent-mode-sessions\e9d07b8f-8b36-4151-9675-df4d5410645a\cec1aa73-8902-4292-97c8-7e05531718df\local_0c2f2061-0a01-4e2f-8a93-c04db8facce1\uploads\AWS CloudWatch Custom Metrics lab.pdf</file_path><file_uuid>745c5858-a2c0-4c8f-8db2-2d37d3ef8529</file_uuid></file>
</uploaded_files>
```

use this file as base,

[assistant] (called Read)

[assistant] The lab gives you the exact recipe – let's follow its structure and just fold in the small fixes for Amazon Linux 2023 and your app's actual port (3000, not 80). Here's the implementation sequence.

**\*\*Step 1 – Confirm IAM role is attached (lab requirement):\*\***

In AWS console: EC2 → select your master instance → Actions → Security → Modify IAM role → pick **LabInstanceProfile** (uses `LabRole`). Save.

On the master:

```
```bash
aws sts get-caller-identity
```
```

Should return an account/ARN. If "Unable to locate credentials," the role didn't attach – refresh the console and re-check.

**Step 2 – Set the region (lab requirement):**

```
```bash
aws configure
```
```

Press Enter through Access Key and Secret Key (the IAM role provides them). For region, type `us-east-1`. Output format can be left blank or `json`.

**Step 3 – Install packages the lab script needs:**

The lab's script uses `netstat` and `iostat`, neither of which ship with AL2023:

```
```bash
sudo dnf install -y net-tools sysstat cronie cronie-anacron
```
```

(`cronie` is also in the lab's instructions.)

**Step 4 – Save the script as `mem.sh`:**

```
```bash
nano /home/ec2-user/mem.sh
```
```

Paste this – it's the lab's script, with port 80 swapped for **3000** (your real app port) and three extra OS metrics (disk, processes, load) for spec coverage:

```
```bash
#!/bin/bash
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
```

```
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
IO_WAIT=$(iostat | awk 'NR==4 {print $5}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", ""); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)
```

```
aws cloudwatch put-metric-data --metric-name memory-usage --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $USEDMEMORY
aws cloudwatch put-metric-data --metric-name Tcp_connections --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN
aws cloudwatch put-metric-data --metric-name TCP_connection_on_port_3000 --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN_PORT_3000
aws cloudwatch put-metric-data --metric-name IO_WAIT --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $IO_WAIT
aws cloudwatch put-metric-data --metric-name Disk_used_percent --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $DISK_USED
```



```
aws cloudwatch put-metric-data --metric-name Process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $PROC_COUNT
aws cloudwatch put-metric-data --metric-name Load_avg_1min --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $LOAD_1M
```
```

Save (Ctrl+O, Enter, Ctrl+X).

**\*\*Step 5 – Make it executable and test once (lab steps 2-3):\*\***

```
```bash
chmod +x /home/ec2-user/mem.sh
/home/ec2-user/mem.sh
echo "exit=$?"
```
```

`exit=0` means every metric posted. If you see `AccessDeniedException`, the IAM role isn't attached. If `command not found: netstat` or `iostat`, repeat Step 3.

**\*\*Step 6 – Verify in CloudWatch (lab "Monitor your EC2 instance" steps):\*\***

AWS console → CloudWatch → Metrics → All metrics → **\*\*Custom\*\*** namespace → **\*\*Instance\*\*** dimension → pick your master's `InstanceId`. You should see all 7 metric names (memory-usage, Tcp\_connections, TCP\_connection\_on\_port\_3000, IO\_WAIT, Disk\_used\_percent, Process\_count, Load\_avg\_1min). One data point each.

**\*\*Step 7 – Schedule via cron (lab's exact instructions):\*\***

```
```bash
sudo systemctl enable --now crond
env EDITOR=nano crontab -e
```
```

Add this single line:

```
```
*/1 * * * * /home/ec2-user/mem.sh
```
```

Save and exit. You'll see `crontab: installing new crontab`.

**\*\*Step 8 – Wait 3 minutes and confirm it's running automatically:\*\***

```
```bash
sudo journalctl -u crond --since "5 minutes ago" | grep mem.sh
```
```

You should see the cron firing the script. Refresh the CloudWatch graph from Step 6 – there should now be 3 data points, one per minute, drawing a real time series.

**\*\*Screenshot now for the report:\*\*** the CloudWatch graph showing your custom metrics flowing from the master's InstanceId. That's spec item #7's first piece of evidence.

**\*\*Step 9 – Now (and only now) bake the AMI:\*\***

The mem.sh file, the cron entry, and the installed packages are all captured in the AMI. Every instance the ASG launches inherits them and starts pushing its own metrics under its own InstanceId – which means after scale-out, you'll see multiple data series in CloudWatch (one per instance). Screenshot that *after* scale-out for the second piece of evidence.

**\*\*One bonus from the lab worth using in the report:\*\***

The "Other Uses" section shows a compound metric (logical AND of two thresholds). Mention in the report that you considered it, and either implement a small "danger" metric like the lab shows, or explain why you didn't (simpler primary metrics gave clearer scaling signals). Either way it earns a sentence under "metric justification."

```
[user] [ec2-user@ip-10-0-8-52 ~]$ nano /home/ec2-user/mem.sh
```

```
[ec2-user@ip-10-0-8-52 ~]$ chmod +x /home/ec2-user/mem.sh
```

```
/home/ec2-user/mem.sh
```

```
echo "exit=$?"
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
Partial credentials found in shared-credentials-file, missing: aws_secret_access_key
```

```
exit=255
```

```
[ec2-user@ip-10-0-8-52 ~]$
```

```
[assistant] That's the AWS CLI's classic gotcha. You ran `aws configure` and entered something for the access key (or pressed a partial key), but skipped or partially filled the secret key. Now the file `~/.aws/credentials` has a half-finished entry that the CLI is using **instead of** your IAM role. The CLI sees credentials in the file → tries to use them → file is incomplete → fails.
```

```
Fix: delete the partial credentials file so the CLI falls back to the IAM role.
```

```
```bash
```

```
rm ~/.aws/credentials
```

```
```
```

```
(Optionally also remove the config file if it has stray entries:)
```

```
```bash
```

```
cat ~/.aws/config          # check what's in it
```

```
```
```

```
Keep just the region. The cleanest version is:
```

```
```bash
```

```
mkdir -p ~/.aws
```

```
cat &gt; ~/.aws/config &&&EOF
```

```
[default]
```

```
region = us-east-1
```

```
EOF
```

```
```
```

```
No credentials file needed – the IAM role (`LabInstanceProfile`) provides credentials automatically via the instance metadata service.
```

```
Verify it's now using the role:
```

```
```bash
```

```
aws sts get-caller-identity
```

```
```
```

```
The `Arn` in the output should contain `LabRole` or `assumed-role`. If it does, you're good.
```

```
Re-run the script:
```

```
```bash
```

```
/home/ec2-user/mem.sh
```

```
echo "exit=$?"
```

```
```
```

`exit=0` and CloudWatch should accept the metrics this time. Then continue with Step 7 (cron).

```
[user] sudo journalctl -u crond --since "5 minutes ago" | grep mem.sh
```

what is it for?

[assistant] It's just a verification command to prove that cron is actually firing your script every minute, not silently doing nothing.

Breaking it apart:

`journalctl` reads the system log. `-u crond` filters to only entries from the cron daemon. `--since "5 minutes ago"` limits to the last 5 minutes.

`| grep mem.sh` narrows further to lines mentioning your script.

If cron is working, you'll see one line per minute, something like:

```
```
```

```
Apr 27 12:01:01 ip-10-0-8-52 CROND[12345]: (ec2-user) CMD (/home/ec2-user/mem.sh)
```

```
```
```

If you see nothing, the cron daemon either isn't running, didn't pick up your crontab entry, or the syntax is wrong – and you can debug from there:

```
```
```

```
sudo systemctl status crond      # is the daemon even running?
```

```
crontab -l                       # is the entry actually saved?
```

```
```
```

It's basically a "did the schedule fire?" sanity check. The script's `*output*` doesn't show up here – for that, the CloudWatch graph is the source of truth (or redirect script output to a log: append `&gt;&gt; /home/ec2-user/mem.log 2&gt;&1`` to your crontab line).

```
[user] [ec2-user@ip-10-0-8-52 ~]$ sudo journalctl -u crond --since "5 minutes ago" | grep mem.sh
```

```
[ec2-user@ip-10-0-8-52 ~]$
```

[assistant] Empty output is a clue – cron may not be running, or your crontab entry isn't actually saved. Run these to diagnose:

**\*\*1. Is the cron daemon active?\*\***

```
```bash
```

```
sudo systemctl status crond
```

```
```
```

- "active (running)" → daemon is fine.

- "inactive (dead)" or "not found" → start and enable it:

```
```bash
```

```
sudo systemctl enable --now crond
```

```
```
```

**\*\*2. Is your cron entry actually saved?\*\***

```
```bash
```

```
crontab -l
```

```
```
```

You should see exactly:

```
```
```

```
*/1 * * * * /home/ec2-user/mem.sh
```

```
```
```

If ``crontab: no crontab for ec2-user``, your edit didn't save. Re-add it:

```
```bash
env EDITOR=nano crontab -e
```
```

Paste the line, save (Ctrl+O, Enter, Ctrl+X).

**\*\*3. How long has it been since you saved the crontab?\*\***

Cron only fires on minute boundaries (e.g., 12:01:00, 12:02:00). If you saved 30 seconds ago, no fire has happened yet. Wait until at least one full minute boundary has passed, then re-check:

```
```bash
sudo journalctl -u crond --since "10 minutes ago" | tail -20
```
```

(Without `grep` this time – see *\*all\** recent cron activity.)

**\*\*4. AL2023 sometimes logs cron differently – also try:\*\***

```
```bash
sudo tail -50 /var/log/cron 2>/dev/null
sudo grep CRON /var/log/messages 2>/dev/null | tail -20
```
```

**\*\*5. Most reliable proof – let the script write its own log:\*\***

Edit the crontab again:

```
```bash
env EDITOR=nano crontab -e
```
```

Change the line to:

```
```
*/1 * * * * /home/ec2-user/mem.sh && /home/ec2-user/mem.log 2>&1
```
```

Wait 2 minutes, then:

```
```bash
ls -la /home/ec2-user/mem.log
tail /home/ec2-user/mem.log
```
```

If the file exists with a recent modification time, cron is firing. If `mem.log` has error output, you'll see exactly what's failing.

Run the first two commands (`systemctl status crond` and `crontab -l`) and paste the output – those will tell us where to look next.

```
[user] [ec2-user@ip-10-0-8-52 ~]$ sudo systemctl status crond
```

```
● crond.service - Command Scheduler
 Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; preset: enabled)
 Active: active (running) since Tue 2026-04-28 17:51:19 UTC; 3min 8s ago
 Main PID: 4346 (crond)
 Tasks: 1 (limit: 516)
 Memory: 1.0M
 CPU: 128ms
```

```

CGroup: /system.slice/crond.service
███4346 /usr/sbin/crond -n
Apr 28 17:51:19 ip-10-0-8-52.ec2.internal systemd[1]: Started crond.service - Command Scheduler.
Apr 28 17:51:19 ip-10-0-8-52.ec2.internal crond[4346]: (CRON) STARTUP (1.5.7)
Apr 28 17:51:19 ip-10-0-8-52.ec2.internal crond[4346]: (CRON) INFO (Syslog will be used instead of sendmail.)
Apr 28 17:51:19 ip-10-0-8-52.ec2.internal crond[4346]: (CRON) INFO (RANDOM_DELAY will be scaled with factor 54% if used.)
Apr 28 17:51:19 ip-10-0-8-52.ec2.internal crond[4346]: (CRON) INFO (running with inotify support)
[ec2-user@ip-10-0-8-52 ~]$ crontab -l
*/1 * * * * /home/ec2-user/mem.sh
[ec2-user@ip-10-0-8-52 ~]$
[assistant] Cron is running and your entry is saved correctly – so the script *should* be firing. The empty journalctl result probably just means
AL2023 isn't logging cron's per-job activity to systemd-journald (it varies). Let's verify another way: have the script write its own evidence.

```

**\*\*Add output redirection to your cron line:\*\***

```

```bash
env EDITOR=nano crontab -e
```

```

Change the line to:

```

```
*/1 * * * * /home/ec2-user/mem.sh && /home/ec2-user/mem.log 2>&1
```

```

Save and exit.

**\*\*Wait 2 minutes, then:\*\***

```

```bash
ls -la /home/ec2-user/mem.log
cat /home/ec2-user/mem.log
```

```

If the file exists with a fresh timestamp, cron is firing. If it's empty (the script outputs nothing on success), `ls -la` showing recent modification is enough proof. If it has error text, that's your debugging clue.

**\*\*At the same time, check CloudWatch directly\*\*** – that's the source of truth anyway. AWS console → **\*\*CloudWatch** → Metrics → All metrics → Custom → Instance\*\* dimension → pick your instance ID. You should see your 7 metric names with multiple data points building up over time, one per minute.

If CloudWatch is showing fresh data points → cron is working, the script is working, and you're done with item #7. The journalctl thing was just a logging quirk.

Also worth running once more for cleaner journal output:

```

```bash
sudo journalctl --since "5 minutes ago" | grep -i CRON
```

```

(Without `-u crond` – broader filter catches whichever facility actually logged it.)

[user] how can I see what in it?

[assistant] All 7 metrics are flowing – that's exactly what you wanted to see. To view the actual data:

**\*\*Tick the checkbox\*\*** next to any metric (e.g., `memory-usage`). A graph will appear above the list showing data points over time. You can tick multiple at once to overlay them on the same chart.

**\*\*A few useful controls once a graph is up:\*\***

Top-right of the graph there are time-range buttons (1h, 3h, 12h, 1d, 3d, 1w, custom). For testing, set it to **\*\*1h\*\*** so you actually see your recent data points.

Set the **\*\*Period\*\*** dropdown (in the same area) to **\*\*1 Minute\*\*** – your script writes one point per minute, so any longer period averages them and the line looks flat.

Hover over any point to see its exact value and timestamp.

**\*\*Quick sanity views to try right now:\*\***

Tick ``memory-usage`` and ``Process_count`` together – you'll see both as separate lines on the same graph. Useful screenshot for the report ("multiple OS metrics flowing from the same instance").

Tick ``TCP_connection_on_port_3000`` – should be near zero now, will spike when you load-test on Day 5. That's a good "before/after" pair for the report.

**\*\*For the report later:\*\***

Once you've launched the ASG and have multiple instances, tick the *\*same\** metric across *\*different\** InstanceIds (you'll see new entries appear with new instance IDs after scale-out). That'll draw multiple lines on the same chart – one per ASG instance – which is the prize-winning screenshot for spec item #7.

You can also click **\*\*Graphed metrics (1)\*\*** tab at the top to rename, change colour, or change the statistic (Average, Sum, Max, Min) – handy if you want to graph the *\*peak\** memory usage rather than the most recent value.

[user] I wanted to add the extra stuff from your script to mine

```
#!/bin/bash
/home/ec2-user/push_metrics.sh
Pushes custom metrics to CloudWatch every minute.
Categories covered: OS activity (memory, disk, processes, load) + app log (request count).
set -u
--- IMDSv2: get a session token, then query metadata ---
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id)
REGION=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/placement/region)
NAMESPACE="PlaceMark/Custom"
DIM="Instance=$INSTANCE_ID"
--- OS metrics ---
USEDMEMORY=$(free | awk '/Mem:/ {printf "%.2f", $3*100/$2}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", ""); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
NODE_COUNT=$(pgrep -c node || echo 0)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)
--- Network metrics (ss replaces deprecated netstat) ---
TCP_CONN=$(ss -tan | tail -n +2 | wc -l)
TCP_CONN_APP=$(ss -tan state established '(sport = :3000 or dport = :3000)' | tail -n +2 | wc -l)
--- App log metric: requests in the last minute (adjust path as needed) ---
APP_LOG="/var/log/placemark.log"
if [[-f "$APP_LOG"]]; then
 ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
 REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
else
 REQ_LAST_MIN=0
fi
```

```

fi
--- Push to CloudWatch ---
push() {
 aws cloudwatch put-metric-data \
 --region "$REGION" --namespace "$NAMESPACE" \
 --dimensions "$DIM" \
 --metric-name "$1" --value "$2" --unit "$3"
}
push MemoryUsedPercent "$USEDMEMORY" Percent
push DiskUsedPercent "$DISK_USED" Percent
push ProcessCount "$PROC_COUNT" Count
push NodeProcessCount "$NODE_COUNT" Count
push LoadAverage1Min "$LOAD_1M" None
push TcpConnections "$TCP_CONN" Count
push TcpConnectionsApp "$TCP_CONN_APP" Count
push RequestsLastMinute "$REQ_LAST_MIN" Count
[assistant] Here's your existing `mem.sh` with the two genuinely new bits merged in – keeping your naming style, the `Custom` namespace, and the same
lab structure. The new metrics are **Node_process_count** (proves the app is running) and **Requests_last_minute** (parses the app log, covers the
"server logs" angle the spec asks for).

```

Replace the contents of `/home/ec2-user/mem.sh` with this:

```

```bash
#!/bin/bash
TOKEN=`curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"`
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)

# --- OS metrics ---
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
IO_WAIT=$(iostat | awk 'NR==4 {print $5}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", "", $5); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)

# --- New: app-specific metrics ---
NODE_COUNT=$(pgrep -c node || echo 0)

APP_LOG="/var/log/placemark.log"
if [[ -f "$APP_LOG" ]]; then
    ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
    REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
else
    REQ_LAST_MIN=0
fi

# --- Push to CloudWatch ---
aws cloudwatch put-metric-data --metric-name memory-usage --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $USEDMEMORY
aws cloudwatch put-metric-data --metric-name Tcp_connections --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN
aws cloudwatch put-metric-data --metric-name TCP_connection_on_port_3000 --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN_PORT_3000
aws cloudwatch put-metric-data --metric-name IO_WAIT --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $IO_WAIT
aws cloudwatch put-metric-data --metric-name Disk_used_percent --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $DISK_USED
aws cloudwatch put-metric-data --metric-name Process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $PROC_COUNT
aws cloudwatch put-metric-data --metric-name Load_avg_1min --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $LOAD_1M

```

```
aws cloudwatch put-metric-data --metric-name Node_process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $NODE_COUNT
aws cloudwatch put-metric-data --metric-name Requests_last_minute --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $REQ_LAST_MIN
```
```

Save, then test:

```
```bash
/home/ec2-user/mem.sh
echo "exit=$?"
```
```

`exit=0` and CloudWatch should pick up the two new metric names within 1-2 minutes – refresh the metrics list and you'll see 9 instead of 7.

**\*\*Important note about `Requests\_last\_minute`:\*\***

Right now your app logs to your terminal when you run `npm run start`, not to `/var/log/placemark.log`. So this metric will report 0 until you redirect the app's output. Two ways to fix:

**\*\*Easiest\*\*** – change the User Data (or the way you start the app) to append output to a log file:

```
```bash
#!/bin/bash
sudo touch /var/log/placemark.log
sudo chown ec2-user:ec2-user /var/log/placemark.log
su - ec2-user -c 'cd PlaceMark-1.0; npm run start &&& /var/log/placemark.log 2&&&&1'
```
```

Then PlaceMark's startup banner and request lines land in the log file, and the `grep` in your metric script can count them.

**\*\*Alternative\*\*** – if PlaceMark uses a logger like Pino or Winston, configure it to write to a file directly. Either approach is fine for the report.

Once that's in place, generate some traffic with `curl -I http://10.0.8.52:3000` a few times and `Requests\_last\_minute` will start showing real numbers. That's a strong screenshot for the load-test section: traffic generated → request count rises → CPU rises → ASG scales out.

**\*\*Why these two are worth the addition for the marker:\*\***

`Node\_process\_count` proves your *application* is running, not just that the OS is alive. If it ever drops to 0, you have evidence to discuss in the report (the instance was healthy at OS level but the app died – exactly the kind of thing custom metrics catch that ALB health checks might miss).

`Requests\_last\_minute` is parsed from server logs, which is one of the two example categories the spec explicitly names ("web server or other server logs, or OS activity"). With both categories covered, you get a clean line in the report: *"the metrics span both bucket types the specification calls out."*

[user] the user data will be for the launch template, is it ?

[assistant] Yes – exactly right. The User Data goes on the **\*\*launch template\*\***, so every new instance the ASG creates runs it on first boot. The master itself doesn't need it (you've been starting the app manually for testing).

So the chain looks like:

```
...
Launch template
 ■ (carries User Data + AMI + SG + IAM profile)
 ▼
ASG launches new instance
 ■
 ▼
Instance boots → User Data runs → app starts → metrics start flowing
```



```

****Practical version of the User Data for your launch template:****

```
```bash
#!/bin/bash
Ensure log file exists with right ownership (so PlaceMark can write to it)
touch /var/log/placemark.log
chown ec2-user:ec2-user /var/log/placemark.log

Start the app as ec2-user, with output redirected to the log
su - ec2-user -c 'cd PlaceMark-1.0; npm run start &&& /var/log/placemark.log 2&&&1'
```
```

The ``touch``/``chown`` lines run as root (User Data always does), which is exactly what you need for creating files in ``/var/log/``. Then ``su -`` drops into ec2-user to start the app.

****Things that must be on the AMI itself (not in User Data):****

- The PlaceMark code at ``/home/ec2-user/PlaceMark-1.0``
- ``node_modules`` already installed
- ``.env`` with the Mongo IP
- ``/home/ec2-user/mem.sh`` (the metrics script)
- The crontab entry (``crontab -l`` should still show it on a new instance)
- Installed packages: ``net-tools``, ``sysstat``, ``cronie``, ``cronie-anacron``
- The crond service enabled (so it auto-starts on boot)

Anything in the user's home dir, package installs, and enabled services are captured in the AMI. The User Data only needs to start the app – everything else is already there from the snapshot.

****Quick pre-bake checklist on the master:****

```
```bash
1. crond is enabled so it autostarts on every new instance
sudo systemctl is-enabled crond # should print "enabled"

2. Crontab is set
crontab -l # should show */1 * * * * /home/ec2-user/mem.sh

3. Script exists and is executable
ls -la /home/ec2-user/mem.sh # -rwxr-xr-x ...

4. Test script runs cleanly
/home/ec2-user/mem.sh &&& echo "OK"

5. Stop any running app process so AMI is baked clean
pkill node 2&&&/dev/null
```
```

If all five pass, you're ready to bake. Stop the instance from the console, then "Create image."

[user] am I ready to bake the AMI ?

[assistant] Almost – let me give you a 5-command pre-flight check. Run these on the master and if all pass, you're cleared to bake.

```
```bash
1. crond will auto-start on every new instance from this AMI
sudo systemctl is-enabled crond
```

```
2. Crontab is saved
crontab -l
```

```
3. Metrics script exists, is executable, and runs cleanly
ls -la /home/ec2-user/mem.sh
/home/ec2-user/mem.sh && echo "SCRIPT OK"
```

```
4. App code, .env, and node_modules are in place
ls /home/ec2-user/PlaceMark-1.0/.env
ls /home/ec2-user/PlaceMark-1.0/node_modules | head -3
```

```
5. Required packages installed
rpm -q net-tools sysstat crontab
```

**\*\*Expected outcomes:\*\***

1. ``enabled``
2. Shows ``*/1 * * * * /home/ec2-user/mem.sh`` (with or without the log redirect)
3. Lists the file with ``-rwxr-xr-x`` and prints ``SCRIPT OK``
4. ``.env`` listed (no error), `node_modules` contains directories
5. All three packages ``installed``

If any of those fail, fix it before baking – it's much cheaper than discovering after launching from a broken AMI.

**\*\*Two things worth double-checking that aren't in those commands:\*\***

The ``.env`` file points at the Mongo private IP ``10.0.133.194``, not an old IP from a previous VPC. New ASG instances will inherit whatever's in there.

```
```bash
grep ^db= /home/ec2-user/PlaceMark-1.0/.env
```
```

Should be ``db=mongodb://10.0.133.194:27017/placemark?directConnection=true`` or similar.

Stop any running node process so the AMI is baked clean (no in-flight DB writes):

```
```bash
pkill node 2>/dev/null
ps aux | grep node | grep -v grep    # should return nothing
```
```

**\*\*If all checks pass – bake it:\*\***

1. AWS console → EC2 → Instances → select your master.
2. Instance state → **\*\*Stop instance\*\***. Wait until state = ``stopped``.
3. Actions → Image and templates → **\*\*Create image\*\***.
4. Name: ``PlaceMark-master-v1`` (or include today's date – useful when you re-bake later).
5. Description: optional, but good for the report ("PlaceMark app + custom metrics cron + Node deps, baked 28 Apr 2026").
6. Wait until **\*\*AMIs\*\*** view shows status = ``available`` (usually 2-5 minutes).

Once the AMI is ``available``, you're ready to create the launch template.

Run the 5 checks and tell me which (if any) fail – or if all pass, ship it.

```
[user] # 1. crond will auto-start on every new instance from this AMI
sudo systemctl is-enabled crond
```

```
2. Crontab is saved
crontab -l
3. Metrics script exists, is executable, and runs cleanly
ls -la /home/ec2-user/mem.sh
/home/ec2-user/mem.sh && echo "SCRIPT OK"
4. App code, .env, and node_modules are in place
ls /home/ec2-user/PlaceMark-1.0/.env
ls /home/ec2-user/PlaceMark-1.0/node_modules | head -3
5. Required packages installed
rpm -q net-tools sysstat cronic
[assistant] Looks like the commands came through but the output didn't – paste the results so I can confirm each check passed.
```

Just run them on the master instance (over your bastion → 10.0.8.52 SSH session) and paste everything that prints. Each command should give a short, clear answer:

```
- `is-enabled crond` → one line (`enabled` / `disabled` / `static`)
- `crontab -l` → your cron line
- `ls -la mem.sh` → file permissions line
- `/home/ec2-user/mem.sh && echo "SCRIPT OK"` → either `SCRIPT OK` or an error
- `ls .env` → the path or "No such file"
- `ls node_modules | head -3` → three folder names
- `rpm -q net-tools sysstat cronic` → three lines, each with a package version
```

Paste the whole block and I'll tell you if you're cleared to bake or what needs fixing first.

```
[user] [ec2-user@ip-10-0-8-52 ~]$ sudo systemctl is-enabled crond
enabled
[ec2-user@ip-10-0-8-52 ~]$ crontab -l
*/1 * * * * /home/ec2-user/mem.sh
[ec2-user@ip-10-0-8-52 ~]$ ls -la /home/ec2-user/mem.sh
/home/ec2-user/mem.sh && echo "SCRIPT OK"
-rwxr-xr-x. 1 ec2-user ec2-user 2260 Apr 28 18:00 /home/ec2-user/mem.sh
SCRIPT OK
[ec2-user@ip-10-0-8-52 ~]$ ls /home/ec2-user/PlaceMark-1.0/.env
ls /home/ec2-user/PlaceMark-1.0/node_modules | head -3
/home/ec2-user/PlaceMark-1.0/.env
@apidevtools
@eslint
@eslint-community
[ec2-user@ip-10-0-8-52 ~]$ rpm -q net-tools sysstat cronic
net-tools-2.0-0.59.20160912git.amzn2023.0.3.x86_64
sysstat-12.5.6-1.amzn2023.0.3.x86_64
cronic-1.5.7-1.amzn2023.0.2.x86_64
[ec2-user@ip-10-0-8-52 ~]$
[assistant] All five checks pass. You're cleared to bake.
```

**\*\*Two final pre-bake actions on the master:\*\***

```
```bash
# Confirm .env points at the right Mongo IP
grep ^db= /home/ec2-user/PlaceMark-1.0/.env

# Stop any running app process so the AMI is baked clean
pkill node 2>/dev/null
ps aux | grep node | grep -v grep    # should return nothing
```
Then bake it:
```

1. AWS console → EC2 → Instances → select the master (`i-06ce3cb...`).
2. **Instance state** → Stop instance. Wait until state = `stopped` (~30 seconds).
3. **Actions** → Image and templates → Create image.
4. Name: `PlaceMark-master-v1` (or `PlaceMark-master-2026-04-28`).
5. Description (optional but useful for the report): `PlaceMark app + custom metrics cron + Node deps, master baked 28 Apr 2026`.
6. Leave "No reboot" unchecked (default – AWS will ensure a clean snapshot).
7. Click **Create image**.
8. Go to EC2 → **AMIs** in the left menu and watch the status. When it flips from `pending` to `available` (2–5 minutes), the AMI is ready.

**Once it's `available`, next up is the launch template:**

EC2 → Launch Templates → Create launch template:

```
- AMI: pick `PlaceMark-master-v1`
- Instance type: `t2.nano` or `t3.nano`
- Key pair: `devops2026`
- Network settings: leave subnet unset (the ASG will pick one)
- Security group: `WebServerNode`
- IAM instance profile (Advanced details): `LabInstanceProfile` – required for the metrics cron to work
- User data (Advanced details):
  ```bash
  #!/bin/bash
  touch /var/log/placemark.log
  chown ec2-user:ec2-user /var/log/placemark.log
  su - ec2-user -c 'cd PlaceMark-1.0; npm run start && /var/log/placemark.log 2&&&1'
```

Smoke-test it by launching one instance from the template (Actions → Launch instance from template, 1 instance, public subnet). SSH in via the bastion and:

```
```bash
sudo tail -30 /var/log/cloud-init-output.log
curl -I http://localhost:3000
```
```

If port 3000 responds and CloudWatch starts showing metrics from the *new* InstanceId within 2 minutes – your AMI is bullet-proof and you're ready for the ASG.

Go ahead and bake – I'll be here for the launch template + ASG step.

```
[user] [ec2-user@ip-10-0-8-52 ~]$ grep ^db= /home/ec2-user/PlaceMark-1.0/.env
db=mongodb://10.0.133.194/Placemark?directConnection=true
[ec2-user@ip-10-0-8-52 ~]$ pkill node 2>/dev/null
ps aux | grep node | grep -v grep # should return nothing
root      1031  0.0  0.0    0    0 ?          I&lt;  16:51   0:00 [xfs-inodegc/xvd]
[ec2-user@ip-10-0-8-52 ~]$
[assistant] Both checks are clean:
```

`.env` points at `10.0.133.194/Placemark` – correct.`

The `ps` line that came back ([xfs-inodegc/xvd])` is a kernel thread, not your app. The square brackets are the giveaway – and the "node" in the grep is matching the "node" inside "i**node**gc" (the XFS filesystem garbage collector). No Node.js processes are running. You're clean.`

Go bake the AMI. EC2 → Instances → select master → Stop → wait for `stopped` → Actions → Image and templates → Create image → name it `PlaceMark-master-v1` → Create.

Then EC2 → AMIs in the left menu, wait for status `available`. Drop me a line when it's ready and we'll do the launch template.

```
[user] #!/bin/bash
touch /var/log/placemark.log
chown ec2-user:ec2-user /var/log/placemark.log
su - ec2-user -c 'cd PlaceMark-1.0; npm run start && /var/log/placemark.log 2&&&1'
[assistant] No response requested.
[user] explain that to me?
[assistant] Sure, line by line:
```

```
```bash
#!/bin/bash
```
```

The "shebang" – tells the OS to interpret this file with bash. Required as the first line of any User Data script. AWS reads it and runs the rest as a bash script during the instance's first boot.

```
```bash
touch /var/log/placemark.log
```
```

Creates an empty log file. User Data runs as **root** automatically, which is why we can write to `/var/log/` (a system directory `ec2-user` can't normally touch). If the file already exists, `touch` just updates its timestamp – no harm done.

```
```bash
chown ec2-user:ec2-user /var/log/placemark.log
```
```

Changes the file's ownership from root to `ec2-user`. This matters because the next line drops to `ec2-user` to run the app, and `ec2-user` needs write permission on the log to append to it. Format is `chown user:group filename`.

```
```bash
su - ec2-user -c 'cd PlaceMark-1.0; npm run start && /var/log/placemark.log 2&&&1'
```
```

This is the working line. Breaking it down further:

- `su - ec2-user` – "switch user" to `ec2-user`. The dash makes it a **login shell**, which is the important bit: it sources `ec2-user's .bash_profile` and `.bashrc`, so anything Node-related (especially if it was installed via `nvm`) ends up on the `PATH`.
- `-c '...'` – instead of starting an interactive shell, run the quoted command and exit.
- Inside the quotes there are two commands separated by `;`:
 - `cd PlaceMark-1.0` – change into the app folder (login shells start in `/home/ec2-user`, so this finds the folder relative to home).
 - `npm run start` – runs the script defined as `"start"` in `package.json`, which kicks off `node src/server.js`.
- `&& /var/log/placemark.log` – append the app's standard output to the log file.
- `2&&&1` – also send standard error to the same place. The `2` means `stderr`, the `&&1` means "wherever stdout is currently going."

So combined, that last line means: "as `ec2-user`, in the `PlaceMark-1.0` folder, run the app and write all of its output (both normal logs and error messages) to `/var/log/placemark.log`."

Why this matters for the assignment:

When the ASG scales out and creates a new instance, this script runs automatically – no human typing `npm run start` on each new box. That's the whole point of putting it in the launch template. The log redirect is doing double duty: it gives you a place to look for problems if a fresh instance fails to start (`ssh in → tail /var/log/`

placemark.log`), AND it feeds your `Requests_last_minute` metric, which `grep`s that file every minute.

Why root only at the top, then drops to ec2-user? Two reasons. First, the log file lives in `/var/log/` – only root can create things there. Second, the app itself should NOT run as root (security best practice) – running web apps as root means a vulnerability gives attackers root, not just "ec2-user." So we use root briefly to set up the log file, then permanently demote to ec2-user before running the actual code. That's a worth-mentioning detail in your report.

[user] do I really need this ? touch /var/log/placemark.log chown ec2-user:ec2-user /var/log/placemark.log

[assistant] Honest answer: it depends on **where** you log to.

****If you keep the path `/var/log/placemark.log`, yes – you need both lines.** Here's why:**

The redirect `&& /var/log/placemark.log` happens ****inside ec2-user's shell**** (because of the `su - ec2-user`). Ec2-user doesn't have write permission to `/var/log/` – only root does. So without the touch/chown setup, bash tries to create the file as ec2-user, gets "Permission denied," and the whole `npm run start` line fails. The app never starts on a new ASG instance, the ALB marks it unhealthy, and your auto-scaling demo breaks.

The two lines exist because ****User Data starts as root**** (so it *can* write to `/var/log/`), and we use that brief root power to create the file and hand it to ec2-user before dropping privileges.

****Simpler alternative – drop both lines, log inside ec2-user's home:****

Change the User Data to:

```
```bash
#!/bin/bash
su - ec2-user -c 'cd PlaceMark-1.0; npm run start && /home/ec2-user/placemark.log 2&&&1'
```
```

Now ec2-user is creating the log file in its own home directory – full write permission, no setup needed. Cleaner User Data.

If you go this route, also update the metrics script to point at the new path:

```
```bash
APP_LOG="/home/ec2-user/placemark.log"
```
```

...and rebuild the AMI (or edit the file before baking).

****Which to choose:****

`/var/log/` is the conventional place for service logs on Linux – it's where logrotate, sysadmins, and most monitoring tools look first. Worth a sentence in the report ("I followed Linux convention for log placement").

`/home/ec2-user/` is simpler – fewer User Data lines, no permission gymnastics, and works without root privileges.

For an assignment, either is fine. I'd lean toward ****dropping the lines**** and using `/home/ec2-user/placemark.log` because it removes a class of failure mode (permission errors on first boot) and the User Data becomes self-contained – easier to copy-paste into the launch template, easier to explain in the report, and one less thing that can break silently.

[user] how do I see that link on the browser ?

[assistant] A bit context-dependent – which link do you want to view? Going to assume you mean ****the PlaceMark app**** in your browser, since that's what we're working on.

****To open PlaceMark in your browser:****

```
```
http://<web-server-public-ip>;3000
```
```

Get the public IP from the AWS console: EC2 → Instances → click your master → "Public IPv4 address" in the details. Currently that was `54.145.135.97`, so:

```
...
http://54.145.135.97:3000
...
```

...in your laptop's browser address bar.

****Two things have to be true for that to work:****

1. ****The app is actually running.**** You did `kill node` to clean up before baking the AMI, so nothing is serving on port 3000 right now. Either:
 - Start it again manually on the master:

```
ssh -J ec2-user@<bastion-ip> ec2-user@10.0.8.52
cd PlaceMark-1.0 && npm run start
...
```

- Or wait for the AMI to be ready, then launch an instance from the launch template (the User Data starts the app for you). Then use *that* instance's public IP.

2. ****Your laptop's IP can reach port 3000 on the web server.**** The WebServerNode SG had port 3000 open to `0.0.0.0/0`, which means anyone can reach it. If that hasn't changed, you're fine. (After you put the ALB in front, you'd lock 3000 down to "only from ALB SG" and stop browsing it directly – you'd browse the ALB's DNS instead.)

****Quick test before opening the browser:****

```
...
curl -I http://54.145.135.97:3000
...
```

If you get any HTTP response (200, 302, even 404), the network path is good and the browser will load it. If it hangs, the app isn't running. If it says "connection refused," the app crashed. If it times out, the SG is blocking your IP.

If you meant a different link (CloudWatch URL, ALB endpoint, AMI status page, etc.), let me know which and I'll point you at the right thing.

[user] is it related to this one ?

Generate test traffic to the load balancer (e.g. using curl/wget or a web testing tool) and show that the load is distributed across more than one web server. Showing the distribution of the load should be done in two ways; for example you could use metadata to show different instance attributes and also display web server or load balancer logs.

[assistant] Related, but not the same thing. Browsing `http://<master-public-ip>;3000` only tests ****that one instance**** – there's no load balancer in the picture yet, so it tells you nothing about distribution.

Spec item #6 needs traffic hitting the ****ALB's DNS name**** (something like `placemark-alb-1234567890.us-east-1.elb.amazonaws.com`), which then distributes the requests across whichever instances the ASG has running. That's where you can show "request 1 went to instance A, request 2 went to instance B."

****Two things the spec wants you to demonstrate:****

****Evidence #1 – instance metadata in the response****

The marker wants to see *different* instances answering the same URL. The cleanest way is to add a tiny endpoint to PlaceMark (or a static file in the web root) that returns the current instance's metadata. For example, an `/whoami` route that responds:

```
...
instance-id: i-06ce3cb...
availability-zone: us-east-1a
```

```
hostname: ip-10-0-8-52
```
```

Then run:

```
```
for i in {1..20}; do curl -s http://&lt;alb-dns>/whoami; done
```
```

You'll get a mix of different instance-ids back – that's the screenshot.

**\*\*Evidence #2 – server or ALB logs\*\***

Either:

- Enable **\*\*ALB access logs to S3\*\*** (in the ALB settings → Edit attributes → Access logs → on, point at an S3 bucket). Each request gets logged with the target instance.
- Or show your **\*\*web server / app logs\*\*** from each instance – both files showing requests with different request IDs / timestamps from the same load test.

Together those two screenshots prove the ALB distributed the load – that's spec item #6 done.

**\*\*What you can do right now to prep for #6:\*\***

Add a `/whoami` route to PlaceMark before baking. Quick example you can drop into `src/server.js` or wherever your routes live:

```
```js
import http from 'http';

server.route({
  method: 'GET',
  path: '/whoami',
  handler: async (request, h) => {
    const token = await fetch('http://169.254.169.254/latest/api/token', {
      method: 'PUT',
      headers: { 'X-aws-ec2-metadata-token-ttl-seconds': '60' }
    }).then(r => r.text());
    const meta = (path) => {
      fetch(`http://169.254.169.254/latest/meta-data/${path}`, {
        headers: { 'X-aws-ec2-metadata-token': token }
      }).then(r => r.text());
    };
    return {
      instanceId: await meta('instance-id'),
      az: await meta('placement/availability-zone'),
      hostname: await meta('hostname'),
    };
  }
});
```
```

(Adjust to whatever framework PlaceMark uses – Hapi, Express, Fastify – the pattern is the same.)

Test it locally:

```
```
curl http://localhost:3000/whoami
```
```



If you get JSON back with the master's instance ID, you're done. Bake the AMI with that route in place, and after the ASG has 2 instances, hitting `/whoami` repeatedly through the ALB shows different instance IDs – perfect evidence for the report.

**\*\*To answer your direct question:\*\***

Browsing the app right now on the master's IP is fine for *\*checking the app works\**. It's not part of spec #6 – that comes later, after the ALB and ASG are up, and you hit the ALB's DNS instead of any single instance.

[user] how can I read the log ?

[assistant] Several tools depending on what you want to see:

**\*\*See the whole file:\*\***

```
```bash
cat /home/ec2-user/placemark.log
```
```

Fine for short logs – dumps everything. Becomes unwieldy once it grows past a screen.

**\*\*See the last 30 lines (most useful for "did the app start ok?"):\*\***

```
```bash
tail /home/ec2-user/placemark.log      # last 10 lines
tail -50 /home/ec2-user/placemark.log  # last 50 lines
```
```

**\*\*Follow the log live (this is the one you'll use most):\*\***

```
```bash
tail -f /home/ec2-user/placemark.log
```
```

The terminal stays open and prints new lines as they're written. Hit `curl http://localhost:3000` from another shell and you'll see the request appear in real time. Press **\*\*Ctrl+C\*\*** to stop following.

**\*\*Page through a long log:\*\***

```
```bash
less /home/ec2-user/placemark.log
```
```

Use arrow keys / Page Up / Page Down to navigate. `/text` to search forward, `n` for next match, `q` to quit.

**\*\*Search for specific lines:\*\***

```
```bash
grep "error" /home/ec2-user/placemark.log
grep -i "mongo" /home/ec2-user/placemark.log # case-insensitive
tail -100 /home/ec2-user/placemark.log | grep "GET"
```
```

**\*\*Useful combos for the assignment:\*\***

While load-testing, watch requests arrive in real time:

```
```bash
tail -f /home/ec2-user/placemark.log
```
```

```

After a load test, count how many requests hit a given instance:

```
```bash
grep -c "GET|POST" /home/ec2-user/placemark.log
```
```

If the log path is `/var/log/placemark.log` instead, swap the path. If you ever can't read it as `ec2-user` (`Permission denied`), prefix with `sudo`:

```
```bash
sudo tail -f /var/log/placemark.log
```
```

```
[user] ```
{
  method: 'GET',
  path: '/testlb',
  handler: function (request, h) {
    return('Server: ' + os.hostname());
  },
  config: {auth: false}
},
},
```

how to get the instance ID?

[assistant] The instance ID lives in the EC2 instance metadata service (IMDSv2). Inside Node, you fetch it via HTTP from `169.254.169.254` – same endpoint the bash metrics script uses. Here's the route written in your style:

```
```js
import os from 'os';
import http from 'http';

// Helper: query IMDSv2 with a session token
async function getInstanceMetadata(path) {
 // Step 1: get a session token (PUT request)
 const token = await new Promise((resolve, reject) => {
 const req = http.request({
 host: '169.254.169.254',
 path: '/latest/api/token',
 method: 'PUT',
 headers: { 'X-aws-ec2-metadata-token-ttl-seconds': '60' },
 timeout: 1000,
 }, res => {
 let data = '';
 res.on('data', c => data += c);
 res.on('end', () => resolve(data));
 });
 req.on('error', reject);
 req.on('timeout', () => req.destroy(new Error('IMDS timeout')));
 req.end();
 });

 // Step 2: GET the metadata path with that token
 return new Promise((resolve, reject) => {
 const req = http.request({
 host: '169.254.169.254',

```

```

 path: `/latest/meta-data/${path}`,
 method: 'GET',
 headers: { 'X-aws-ec2-metadata-token': token },
 timeout: 1000,
 }, res => {
 let data = '';
 res.on('data', c => data += c);
 res.on('end', () => resolve(data));
 });
req.on('error', reject);
req.on('timeout', () => req.destroy(new Error('IMDS timeout')));
req.end();
});
}

```

Then your route:

```

```js
{
  method: 'GET',
  path: '/testlb',
  handler: async function (request, h) {
    try {
      const instanceId = await getInstanceMetadata('instance-id');
      const az = await getInstanceMetadata('placement/availability-zone');
      return `Server: ${os.hostname()} | Instance: ${instanceId} | AZ: ${az}`;
    } catch (e) {
      return `Server: ${os.hostname()} | (metadata unavailable: ${e.message})`;
    }
  },
  config: { auth: false }
}
```

```

Hitting `/testlb` will return something like:

```

```
Server: ip-10-0-8-52.ec2.internal | Instance: i-06ce3cb7970b64bff | AZ: us-east-1a
```

```

When the ALB is up and the ASG has 2 instances, repeated calls to `/testlb` through the ALB will return *different* instance IDs and AZs – exactly the metadata-based evidence the spec asks for under #6.

**\*\*Three things worth noting:\*\***

1. The endpoint `169.254.169.254` is a "link-local" address that only works *from inside an EC2 instance*. If you run this code on your laptop, the request will time out – that's why the try/catch falls back to just the hostname.
2. The token has a TTL – I set it to 60 seconds. For a route that's hit per-request, that's fine. If you want to optimise, cache the token for ~5 minutes and refresh on demand.
3. You could also reach for the AWS SDK (`@aws-sdk/client-imsd` or `aws-sdk`), but it's overkill for two metadata calls and adds another dependency. Plain `http` is lighter and matches the bash style you've already used.

**\*\*Quick test on the master before baking:\*\***

```

```
npm run start          # in one shell
curl http://localhost:3000/testlb
```

```

If you see the instance ID, you're set. Stop the app, rebake the AMI to capture the new route, then update the launch template's AMI reference (or just create a new launch template version pointing at the new AMI).

[user] I just need the instance ID , forget the rest

[assistant] Minimal version – just adds the instance ID to your existing route:

```

```js
import os from 'os';
import http from 'http';

function getInstanceId() {
  return new Promise((resolve) => {
    // Get token
    const tokenReq = http.request({
      host: '169.254.169.254',
      path: '/latest/api/token',
      method: 'PUT',
      headers: { 'X-aws-ec2-metadata-token-ttl-seconds': '60' },
      timeout: 1000,
    }, (res) => {
      let token = '';
      res.on('data', (c) => token += c);
      res.on('end', () => {
        // Get instance-id with token
        http.get({
          host: '169.254.169.254',
          path: '/latest/meta-data/instance-id',
          headers: { 'X-aws-ec2-metadata-token': token },
          timeout: 1000,
        }, (r) => {
          let id = '';
          r.on('data', (c) => id += c);
          r.on('end', () => resolve(id));
        }).on('error', () => resolve('unknown'));
      });
    });
    tokenReq.on('error', () => resolve('unknown'));
    tokenReq.end();
  });
}
```

```

Your route:

```

```js
{
  method: 'GET',
  path: '/testlb',
  handler: async function (request, h) {
    const id = await getInstanceId();
    return `Server: ${os.hostname()} | Instance: ${id}`;
  },
}

```

```
    config: { auth: false }
}
...
```

Test:

```
...
curl http://localhost:3000/testlb
...
```

Returns: `Server: ip-10-0-8-52.ec2.internal | Instance: i-06ce3cb7970b64bff`
[user] how to check is my database is running >
[assistant] Depends on where you're running the check from. Three options:

****From the Mongo instance itself (most direct):****

SSH in via the bastion (`ssh -J ec2-user@<bastion-ip> ec2-user@10.0.133.194`), then:

```
```bash
sudo systemctl status mongod
```
```

- `Active: active (running)` → it's up.
- `inactive (dead)` → start it: `sudo systemctl start mongod`
- `failed` → check the log: `sudo tail -50 /var/log/mongodb/mongod.log`

Also confirm it's actually listening:

```
```bash
sudo ss -tlnp | grep 27017
```
```

Should show `0.0.0.0:27017`.

****From the app instance (network-level check, most useful for the assignment):****

```
```bash
nc -vz 10.0.133.194 27017
```
```

- `succeeded` / `Connected to ...` → Mongo is reachable, network and SG are fine.
- Hangs / `Operation timed out` → SG blocking.
- `Connection refused` → got to the host, but Mongo isn't running.

This is the test that matters because it mimics what your app does.

****From inside Mongo (proves it's actually serving queries):****

On the Mongo instance:

```
```bash
mongosh --eval "db.runCommand({ ping: 1 })"
```
```

Should return `{ ok: 1 }`. If you get a connection error, the daemon is up but something's misconfigured (binding, auth).
For day-to-day "is it running?" checks, the `nc -vz` from the app instance is the quickest and tells you the most: covers daemon up, binding correct,

SG allowing the app, network path open – all in one ping.

```
[user] ```
2026/05/01 17:12:48Z: SSM Agent unable to acquire credentials: &lt;error&gt;no valid credentials could be retrieved for ec2 identity. Default Host
Management Err: error calling RequestManagedInstanceRoleToken: AccessDeniedException: Systems Manager's instance management role is not configured
for account: 628057338489
```
```

[assistant] Harmless noise – ignore it. That's the AWS SSM (Systems Manager) Agent, which ships pre-installed on Amazon Linux 2023 AMIs. It tries to register the instance with Systems Manager so AWS features like Session Manager, Run Command, and Patch Manager work. In the AWS Academy Learner Lab, the IAM permissions for "Default Host Management" aren't configured – so SSM Agent fails to register and logs this every few minutes.

**\*\*Why it doesn't matter for your assignment:\*\***

It does *not* affect your PlaceMark app, the custom CloudWatch metrics, CloudWatch alarms, the ALB, the ASG, or anything you're being graded on. SSM is a separate AWS service for remote administration, completely orthogonal to load balancing and auto-scaling. The Learner Lab simply doesn't grant the role SSM wants, so the agent grumbles but the rest of the system is unaffected.

**\*\*If the noise bothers you (e.g., it's spamming the journal), you can disable the agent:\*\***

```
```bash
sudo systemctl disable --now amazon-ssm-agent
```
```

That's optional and not needed for the assignment. I'd actually leave it enabled – disabling it is one more thing to bake into the AMI, and the marker will recognize the message as a known Learner Lab artefact.

If you want a one-line throwaway mention in your report (under "limitations of the Learner Lab"), it's something like: **"SSM Agent registration is disabled in the AWS Academy environment, producing harmless credential-error log entries – these do not affect application or scaling behaviour."**

Otherwise: keep moving. The error is cosmetic.

[user] how to solve it, I think I need to update the lab authentication

[assistant] That "Access denied" on Account name and color is **\*\*normal for AWS Academy Learner Lab\*\*** – not something you need to fix. The Learner Lab restricts your IAM permissions to managing resources (EC2, VPC, CloudWatch, etc.), but doesn't grant you ``Organizations:DescribeAccount``, which is what populates those fields. So the dropdown can't fetch them and shows "Access denied." It's cosmetic.

**\*\*You can ignore it.\*\*** It does not affect your ability to launch instances, configure security groups, push metrics, or any of the assignment work. The account ID (628057338489) is visible, the federated user (``Tiago_Villaca``) is correct – those are the only fields that actually matter.

**\*\*When Learner Lab auth genuinely *does* need refreshing:\*\***

The Learner Lab session itself only lasts ~4 hours, and credentials expire when you stop the lab or close the AWS Academy tab. Symptoms of a real auth problem:

- Console operations fail with "Token expired" or "Your session has expired" banners.
- AWS CLI calls return "ExpiredToken" or "InvalidClientTokenId."
- Instances you launched earlier still run, but you can't see them or do anything to them.

If that happens, the fix is: go back to the AWS Academy Moodle, find your Learner Lab module, click **\*\*Start Lab\*\*** to refresh the session, then click **\*\*AWS\*\*** to open a new console tab. If you've also configured CLI credentials on a local machine, copy the new short-lived keys from "AWS Details → AWS CLI" into `~/aws/credentials`` (you don't need this on the EC2 instances because they use the `LabInstanceProfile` role).

Right now though, what you're showing is just a normal lab limitation. Carry on with the AMI / launch template work.

[user] But my cloud watch metrics are not being updated

[assistant] That's a real issue, separate from the cosmetic "Access denied" thing. Let's narrow down which of three causes you're hitting.

**\*\*Likely cause #1 – the master instance is stopped because you started baking the AMI.\*\***

When you stop the master to bake the AMI, the ``mem.sh`` cron stops running too – no instance, no metrics. The CloudWatch graph keeps the historical

data but stops adding new points. Check:

AWS console → EC2 → Instances → look at the master's state. If it's `stopped`, that's why. The fix isn't to start it – once the AMI is `available`, you launch a new instance from the launch template, \*that\* instance starts pushing metrics under its own InstanceId, and you'll see a new line appear in CloudWatch.

**\*\*Likely cause #2 – Learner Lab session expired (~4 hours).\*\***

When the lab session expires, the IAM role token on your running instances goes stale and `aws cloudwatch put-metric-data` starts failing silently from cron. The instance is still running, it just can't authenticate to AWS anymore.

To check: SSH into the master (if it's running) and run:

```
```bash
aws sts get-caller-identity
```
```

- Returns ARN → role is fine, this isn't it.
- "ExpiredToken" / "InvalidClientTokenId" / "Unable to locate credentials" → the lab session expired.

Fix: in your AWS Academy Moodle, click **\*\*End Lab\*\*** then **\*\*Start Lab\*\*** again to refresh credentials. The instance picks up the new role token automatically within a minute or two – no restart needed.

**\*\*Likely cause #3 – the cron job is failing for a different reason.\*\***

If the instance is running and creds are fine, check the script's output:

```
```bash
ls -la /home/ec2-user/mem.log      # if you added the log redirect to crontab
tail -30 /home/ec2-user/mem.log
```

```
# Or run it once manually to see fresh output
/home/ec2-user/mem.sh
echo "exit=$?"
```
```

A non-zero exit code or visible error tells you what broke.

**\*\*Run this fast triage and tell me what you see:\*\***

1. EC2 console – what's the master's state? (`running` / `stopped`?)
2. If running, paste the result of `aws sts get-caller-identity` from the master.
3. If running and creds are fine, paste the result of `/home/ec2-user/mem.sh && echo "exit=\$?"`.

That'll tell us which lane the problem is in. My bet is #1 (you stopped the master to bake) – in which case the answer is "wait for the AMI, then launch from the launch template, and the metrics will resume from the new instance."

[user] how to make the crono run again when it lauches?

[assistant] Good news – you've already done it. Cron auto-runs on every new instance from this AMI because three things were captured in the snapshot:

1. **\*\*The `crond` service is enabled\*\*** (`systemctl is-enabled crond` returned `enabled`). When a new instance from this AMI boots, systemd reads that "enabled" state and starts crond automatically – no User Data needed.
2. **\*\*The crontab entry is saved\*\*** in `/var/spool/cron/ec2-user` (that's where `crontab -e` writes to, behind the scenes). That folder is part of the disk, so it's part of the AMI.
3. **\*\*The script + packages are present\*\*** – `/home/ec2-user/mem.sh` and the installed `net-tools`/`sysstat`/`cronie` packages are all on the AMI.

So when a new instance launches:

- Linux boots → systemd starts → systemd starts `cron` (because enabled) → cron reads the saved crontab → fires `mem.sh` every minute → metrics flow into CloudWatch under the \*new\* instance's InstanceId.

**\*\*The one thing the AMI doesn't carry: IAM credentials.\*\***

The role that lets `aws cloudwatch put-metric-data` succeed comes from the **\*\*launch template\*\*** (the IAM instance profile field). When you set up the launch template, make sure you select **\*\*`LabInstanceProfile`\*\*** under "Advanced details → IAM instance profile." If you forget, the new instance will have cron running and the script firing, but every `aws cloudwatch` call will fail with `Unable to locate credentials` – silently breaking metrics.

**\*\*To prove all this works after the AMI is ready:\*\***

1. Launch one test instance from the launch template (Actions → Launch instance from template, 1 instance, public subnet).
2. Wait ~3 minutes for boot + cron's first fire.
3. Refresh CloudWatch → Metrics → Custom → Instance dimension.
4. You should now see a **\*\*new InstanceId\*\*** appear in the list – the new instance's metrics – alongside (or replacing) the master's.

That's the proof of cron auto-run from the AMI, and it's the same screenshot pattern you'll use later for the report when the ASG scales out.

**\*\*If metrics from the new instance don't appear:\*\***

SSH into it via the bastion and check, in order:

```
```bash
sudo systemctl status crond          # should be active (running)
crontab -l                           # should show your line
aws sts get-caller-identity           # should return an ARN (proves IAM role attached)
/home/ec2-user/mem.sh && echo OK      # manual run; should print OK
```
```

The first command that fails tells you what's missing. But if all four passed on the master before baking, all four will pass on the new instance – that's exactly what AMIs preserve.

[user] ● crond.service - Command Scheduler

Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; preset: enabled)

Active: active (running) since Fri 2026-05-01 18:47:58 UTC; 55min ago

Main PID: 2142 (crond)

Tasks: 2 (limit: 516)

Memory: 1.7M

CPU: 2.098s

CGroup: /system.slice/crond.service

■2142 /usr/sbin/crond -n

■3752 /usr/sbin/anacron -s

May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Will run job `cron.daily` in 11 min.

May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Will run job `cron.weekly` in 31 min.

May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Will run job `cron.monthly` in 51 min.

May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Jobs will be executed sequentially

May 01 19:01:01 ip-10-0-20-203.ec2.internal run-parts[3754]: (/etc/cron.hourly) finished 0anacron

May 01 19:01:01 ip-10-0-20-203.ec2.internal CROND[3737]: (root) CMDEND (run-parts /etc/cron.hourly)

May 01 19:12:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.daily` started

May 01 19:12:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.daily` terminated

May 01 19:32:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.weekly` started

May 01 19:32:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.weekly` terminated

\* /1 \* \* \* \* /home/ec2-user/[mem.sh](http://mem.sh)

An error occurred (ExpiredToken) when calling the GetCallerIdentity operation: The security token included in the request is expired

An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired

An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired



An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired  
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired  
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired  
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired  
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired  
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired  
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]

To see help text, you can run:

```
aws help
aws <command> help
aws <command> <subcommand> help
Unknown options: 0
```

[assistant] There's the smoking gun: `ExpiredToken`. Your Learner Lab session expired, the IAM role's session credentials are dead, and every `aws cloudwatch put-metric-data` call is bouncing back. Cron is firing perfectly – the script just can't talk to AWS anymore.

**\*\*Fix – refresh the Learner Lab session:\*\***

1. Go to your AWS Academy Moodle.
2. Open the Learner Lab module.
3. If it shows a green/red dot indicating the lab is on, click **\*\*End Lab\*\***, wait for it to stop, then **\*\*Start Lab\*\*** again.
4. Wait until the dot turns green and "AWS" link is clickable.
5. Click **\*\*AWS\*\*** to open a fresh console tab.

Within ~60 seconds the running EC2 instance picks up new credentials automatically from the instance metadata service – no instance restart needed.

**\*\*Verify the fix:\*\***

SSH back into the instance and run:

```
``bash
aws sts get-caller-identity
``
```

Should return a fresh ARN, no expired-token error. Then:

```
``bash
/home/ec2-user/mem.sh &&& echo OK
``
```

Should print `OK`. Refresh the CloudWatch graph in another minute – new data points should resume.

**\*\*Why this happens (one-line for the report):\*\***

AWS Academy Learner Lab sessions expire after a few hours. The IAM role attached to the instance is still attached, but the temporary session credentials it issues become invalid when the lab session ends – restarting the lab issues fresh ones, which the instance picks up automatically. This is a Learner Lab limitation, not something a production setup would hit.

**\*\*Side issue I spotted in your output: "Unknown options: 0"\*\***

That last line suggests one of the metric values in `mem.sh` is being passed in a way that confuses the AWS CLI – probably `IO\_WAIT` from `iostat | awk 'NR==4 {print \$5}'`. The `iostat` output format on AL2023 may not put a value in row 4 column 5 (sometimes that line is blank or the `0.00` is in a different column), so `IO\_WAIT` ends up empty, the CLI sees `--value` followed by `--metric-name`, and complains.

Quick safety net – wrap the value in quotes and default to 0 if empty. In `mem.sh`, change:

```
``bash
```

```
IO_WAIT=$(iostat | awk 'NR==4 {print $5}')
```

to:

```
```bash
IO_WAIT=$(iostat | awk 'NR==4 {print $5}')
```

And in the put-metric-data call, quote the variable:

```
```bash
aws cloudwatch put-metric-data --metric-name IO_WAIT --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$IO_WAIT"
```

Once you fix the lab session and re-run, you should see all metrics resume cleanly.

```
[user] [ec2-user@ip-10-0-20-203 ~]$ aws sts get-caller-identity
```

An error occurred (ExpiredToken) when calling the GetCallerIdentity operation: The security token included in the request is expired

```
[ec2-user@ip-10-0-20-203 ~]$
```

[assistant] Lab session is still expired. A few possibilities – let's narrow down.

**\*\*Step 1 – make sure the Learner Lab is actually started:\*\***

Go to AWS Academy Moodle, open the Learner Lab module, and look at the **\*\*dot indicator\*\*** next to the lab.

- ■ Red → lab is stopped. Click **\*\*Start Lab\*\***.
- ■ Yellow → lab is starting. Wait until it goes green.
- ■ Green → lab is running. If you just restarted it, give it ~60 seconds for the instance metadata service to refresh.

If you didn't actually click End Lab → Start Lab in the Moodle, that's why creds are still expired. The AWS console doesn't restart the lab session – it has to be done from the AWS Academy portal.

**\*\*Step 2 – make sure no stale credentials file is overriding the role:\*\***

```
```bash
ls -la ~/.aws/
cat ~/.aws/credentials &>/dev/null
cat ~/.aws/config &>/dev/null
```

If `~/.aws/credentials` exists with old keys, the AWS CLI uses **those** instead of the IAM role and will keep failing forever. Delete it:

```
```bash
rm ~/.aws/credentials
```

Keep `~/.aws/config` only if it just has the region:

```
```bash
cat &> ~/.aws/config &&EOF
[default]
region = us-east-1
EOF
```

****Step 3 – check what the instance metadata service is actually serving:****

```
```bash
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/iam/security-credentials/
```
```

This returns the role name (e.g., `LabRole` or similar). Then:

```
```bash
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/iam/security-credentials/LabRole
```
```

(Substitute whatever role name came back.) Look at the `Expiration` field in the JSON – that's a UTC timestamp. If it's in the past, the lab session hasn't refreshed yet. If it's in the future (a few hours from now), creds are fresh and the problem is somewhere else.

****Run all three steps and paste:****

1. The Moodle lab indicator color (or just confirm "Yes I clicked Start Lab")
2. The output of `ls -la ~/.aws/`
3. The `Expiration` line from the metadata response

That'll point us at the exact problem. Most likely it's #1 (lab not restarted yet) or #2 (stale credentials file).

```
[user] [ec2-user@ip-10-0-20-203 ~]$ nano ~/.aws/credentials
[ec2-user@ip-10-0-20-203 ~]$ sudo systemctl status crond          # should be active (running)
crontab -l              # should show your line
aws sts get-caller-identity # should return an ARN (proves IAM role attached)
/home/ec2-user/[mem.sh](http://mem.sh) && echo OK                # manual run; should print OK
● crond.service - Command Scheduler
   Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-05-01 18:47:58 UTC; 58min ago
     Main PID: 2142 (crond)
        Tasks: 2 (limit: 516)
       Memory: 1.7M
          CPU: 2.213s
      CGroup: /system.slice/crond.service
              ┌─2142 /usr/sbin/crond -n
              └─3752 /usr/sbin/anacron -s
May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Will run job `cron.daily' in 11 min.
May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Will run job `cron.weekly' in 31 min.
May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Will run job `cron.monthly' in 51 min.
May 01 19:01:01 ip-10-0-20-203.ec2.internal anacron[3752]: Jobs will be executed sequentially
May 01 19:01:01 ip-10-0-20-203.ec2.internal run-parts[3754]: (/etc/cron.hourly) finished 0anacron
May 01 19:01:01 ip-10-0-20-203.ec2.internal CROND[3737]: (root) CMDEND (run-parts /etc/cron.hourly)
May 01 19:12:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.daily' started
May 01 19:12:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.daily' terminated
May 01 19:32:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.weekly' started
May 01 19:32:01 ip-10-0-20-203.ec2.internal anacron[3752]: Job `cron.weekly' terminated
*/1 * * * * /home/ec2-user/[mem.sh](http://mem.sh)
{
  "UserId": "AROAZE0ZCOJ4Q0PNF3GZU:user4776643=Tiago_Villaca",
  "Account": "628057338489",
  "Arn": "arn:aws:sts::628057338489:assumed-role/voclabs/user4776643=Tiago_Villaca"
}
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
```

To see help text, you can run:

```
aws help
aws &lt;command> help
aws &lt;command> &lt;subcommand> help
```

Unknown options: 0

```
[ec2-user@ip-10-0-20-203 ~]$
```

[assistant] Good news on the auth – `aws sts get-caller-identity` is returning a valid ARN now. The lab session is refreshed.

****But the "Unknown options: 0" error from `mem.sh` is still there**, which means the script itself has a bug separate from the auth issue. One of your metric variables is empty when the script runs, so the AWS CLI sees `--value` followed by nothing and gets confused.**

The most likely culprit is `IO\_WAIT`. On AL2023's version of `iostat`, the layout is:

```
Linux 6.1.x ...
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.50    0.00    0.30    0.00    0.00   99.20

Device:            tps    kB_read/s    kB_wrtn/s ...
xvda                2.10    45.30         60.20    ...
```

The line `awk 'NR==4 {print \$5}'` is supposed to grab the `%iowait` value, but on AL2023 that lands on a different row depending on whether iostat is being run for the first time.

****Quick fix – make `mem.sh` resilient to empty values:****

```
```bash
nano /home/ec2-user/mem.sh
```
```

After each variable assignment, add a fallback to 0 if empty. Replace the metrics section with:

```
```bash
--- OS metrics ---
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
IO_WAIT=$(iostat | awk '/^avg-cpu/ {getline; print $4}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", ""), $5; print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)

--- Default to 0 if any value came back empty ---
USEDMEMORY=${USEDMEMORY:-0}
TCP_CONN=${TCP_CONN:-0}
TCP_CONN_PORT_3000=${TCP_CONN_PORT_3000:-0}
IO_WAIT=${IO_WAIT:-0}
DISK_USED=${DISK_USED:-0}
PROC_COUNT=${PROC_COUNT:-0}
LOAD_1M=${LOAD_1M:-0}
```
```

Two changes worth noting:

The `IO\_WAIT` line now finds the `avg-cpu` header dynamically and reads the next line's column 4 (`%iowait`), regardless of where that block lands in

the iostat output. More robust than `NR==4`.

The `\${VAR:-0}` syntax means "use the variable, but if it's empty, substitute `0`". So even if a command fails silently, you'll push a 0 instead of nothing.

****Re-run and verify:****

```
```bash
/home/ec2-user/mem.sh && echo "SCRIPT OK"
```
```

Should print `SCRIPT OK` with no `usage:` errors.

****Important note about your credentials file:****

You ran `nano ~/.aws/credentials` and pasted in your personal AWS Academy CLI keys – that's why `aws sts` works now. ****Don't bake the AMI with that file present.**** Two reasons:

1. Those keys expire when your lab session ends. New instances launched from the AMI hours later would inherit dead credentials and fail silently.
2. Anyone with the AMI gets your keys. Even in a Learner Lab that's bad practice.

Before you bake, do this:

```
```bash
rm ~/.aws/credentials
```
```

The instance has the `LabInstanceProfile` IAM role attached, which provides credentials automatically via the metadata service. With `~/.aws/credentials` removed, the CLI falls back to the role – and *that* mechanism keeps working on every new ASG instance because the launch template attaches the role to each one.

Once you remove the file, re-test:

```
```bash
aws sts get-caller-identity
```
```

Should still return an ARN (this time showing `LabRole` or `assumed-role/voclabs/...` from the role, not the user). Then bake the AMI.

```
[user] #!/bin/bash
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
# --- OS metrics ---
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
IO_WAIT=$(iostat | awk 'NR==4 {print $5}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", ""); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)
# --- New: app-specific metrics ---
NODE_COUNT=$(pgrep -c node || echo 0)
APP_LOG="/var/log/placemark.log"
if [[ -f "$APP_LOG" ]]; then
    ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
    REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
else
```

```

    REQ_LAST_MIN=0
fi
# --- Push to CloudWatch ---
aws cloudwatch put-metric-data --metric-name memory-usage --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $USEDMEMORY
aws cloudwatch put-metric-data --metric-name Tcp_connections --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN
aws cloudwatch put-metric-data --metric-name TCP_connection_on_port_3000 --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $TCP_CONN_
PORT_3000
aws cloudwatch put-metric-data --metric-name IO_WAIT --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $IO_WAIT
aws cloudwatch put-metric-data --metric-name Disk_used_percent --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $DISK_USED
aws cloudwatch put-metric-data --metric-name Process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $PROC_COUNT
aws cloudwatch put-metric-data --metric-name Load_avg_1min --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $LOAD_1M
aws cloudwatch put-metric-data --metric-name Node_process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $NODE_COUNT
aws cloudwatch put-metric-data --metric-name Requests_last_minute --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value $REQ_LAST_MIN
[assistant] Here's the fixed full script – I've changed the `IO_WAIT` extraction to be robust to AL2023's iostat layout, added empty-value defaults
to every metric, and quoted the variables on the AWS CLI calls so any stray whitespace can't corrupt the arguments. Save this over your existing
`mem.sh`:

```bash
#!/bin/bash
TOKEN=`curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"`
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)

--- OS metrics ---
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
IO_WAIT=$(iostat | awk '/^avg-cpu/ {getline; print $4}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", "", $5); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)

--- App-specific metrics ---
NODE_COUNT=$(pgrep -c node || echo 0)
APP_LOG="/var/log/placemark.log"
if [[-f "$APP_LOG"]]; then
 ONE_MIN_AGO=$(date -d "1 min ago" +%Y-%m-%dT%H:%M)
 REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
else
 REQ_LAST_MIN=0
fi

--- Default any empty values to 0 so AWS CLI doesn't choke ---
USEDMEMORY=${USEDMEMORY:-0}
TCP_CONN=${TCP_CONN:-0}
TCP_CONN_PORT_3000=${TCP_CONN_PORT_3000:-0}
IO_WAIT=${IO_WAIT:-0}
DISK_USED=${DISK_USED:-0}
PROC_COUNT=${PROC_COUNT:-0}
LOAD_1M=${LOAD_1M:-0}
NODE_COUNT=${NODE_COUNT:-0}
REQ_LAST_MIN=${REQ_LAST_MIN:-0}

--- Push to CloudWatch ---
aws cloudwatch put-metric-data --metric-name memory-usage --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$USEDMEMORY"
aws cloudwatch put-metric-data --metric-name Tcp_connections --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$TCP_CONN"
aws cloudwatch put-metric-data --metric-name TCP_connection_on_port_3000 --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$TCP_CONN_

```

```

PORT_3000"
aws cloudwatch put-metric-data --metric-name IO_WAIT --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$IO_WAIT"
aws cloudwatch put-metric-data --metric-name Disk_used_percent --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$DISK_USED"
aws cloudwatch put-metric-data --metric-name Process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$PROC_COUNT"
aws cloudwatch put-metric-data --metric-name Load_avg_1min --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$LOAD_1M"
aws cloudwatch put-metric-data --metric-name Node_process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$NODE_COUNT"
aws cloudwatch put-metric-data --metric-name Requests_last_minute --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$REQ_LAST_
MIN"
```

```

****Save and re-test:****

```

```bash
nano /home/ec2-user/mem.sh
paste the above, save, exit
chmod +x /home/ec2-user/mem.sh
/home/ec2-user/mem.sh && echo "SCRIPT OK"
```

```

You should see `SCRIPT OK` with no AWS CLI usage errors. Wait 1-2 minutes, refresh Cloudwatch, all 9 metrics should show fresh data points.

****One quick sanity check before you bake:****

```

```bash
echo "MEMORY=$USEDMEMORY DISK=$DISK_USED IO=$IO_WAIT LOAD=$LOAD_1M PROC=$PROC_COUNT"
```

```

Run that **inside** the script (or paste a debug line `echo "DEBUG: ..." &&2`` near the top). Every value should be a number, none empty. The defaults take care of any silent failures, but it's good to confirm the underlying commands actually return data on this AMI.

****Reminder for AMI baking**** – before you stop the instance:

```

```bash
rm ~/.aws/credentials # so the AMI uses the role, not your personal keys
kill node # clean shutdown
```

```

Then stop → create image → launch template references the new AMI → IAM instance profile = `LabInstanceProfile` on the launch template.

```

[user] [ec2-user@ip-10-0-24-62 ~]$ chmod +x /home/ec2-user/[mem.sh](http://mem.sh)
/home/ec2-user/[mem.sh](http://mem.sh) && echo "SCRIPT OK"
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
An error occurred (ExpiredToken) when calling the PutMetricData operation: The security token included in the request is expired
[&lt;class 'decimal.ConversionSyntax'&gt;]
[ec2-user@ip-10-0-24-62 ~]$
[assistant] Two separate problems in that output:

```

****Problem 1 – ExpiredToken (8 of 9 errors):****

The credentials are stale again. You're on a different IP now (`10.0.24.62` vs the previous `10.0.20.203`), so this is probably a fresh instance launched from the AMI – and the AMI is carrying a stale `~/.aws/credentials` file from when you edited it before baking. That file overrides the IAM

role.

Fix on this instance:

```
```bash
ls -la ~/.aws/
rm -f ~/.aws/credentials
aws sts get-caller-identity
```
```

The ARN should now show `assumed-role/...LabRole...` (not your personal user) – proof it's using the IAM role from the launch template.

If `aws sts` still says `ExpiredToken` after deleting the file, the Learner Lab session itself is dead – go to AWS Academy Moodle, ****End Lab → Start Lab****, wait 60 seconds, then re-test.

****Problem 2 – `decimal.ConversionSyntax` (the 9th error):****

That's the AWS CLI choking locally on a malformed value before it even tries the API call. It's caused by this idiom in your script:

```
```bash
NODE_COUNT=$(pgrep -c node || echo 0)
REQ_LAST_MIN=$(... || echo 0)
```
```

Subtle bug: `pgrep -c node` ****always outputs the count**** (even `0`), but returns exit code 1 when there are no matches. So when no node processes are running, the substitution captures `"0\n0"` – ***two* zeros**, the first from `pgrep`'s stdout, the second from the `|| echo 0` fallback. AWS CLI then sees `--value "0\n0"` and fails decimal parsing.

Same trap on the `grep` line.

****Fix – capture stdout, then default if empty:****

Replace those two lines with:

```
```bash
NODE_COUNT=$(pgrep -c node 2>/dev/null)
NODE_COUNT=${NODE_COUNT:-0}

APP_LOG="/var/log/placemark.log"
if [[-f "$APP_LOG"]]; then
 ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
 REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null)
 REQ_LAST_MIN=${REQ_LAST_MIN:-0}
else
 REQ_LAST_MIN=0
fi
```
```

Both `pgrep -c` and `grep -c` print `0` to stdout when nothing matches, and `${VAR:-0}` only fires if the variable is genuinely empty – no double-echo problem.

****Apply both fixes:****

```
```bash
rm -f ~/.aws/credentials
nano /home/ec2-user/mem.sh
```



```
replace the NODE_COUNT and REQ_LAST_MIN lines as above, save
/home/ec2-user/mem.sh && echo "SCRIPT OK"
```
```

You should now get a clean `SCRIPT OK` with no errors.

****Heads-up about the AMI:****

If this new instance was launched from the AMI you just baked, it inherited the bug. You should:

1. Fix `mem.sh` on the master (or on this test instance) to confirm it works.
2. Delete the stale credentials file from the master too.
3. Re-bake the AMI (you can update the existing one or create a new version) so the bug isn't propagated to ASG instances later.

Or, simpler if you're under time pressure: leave the AMI as-is, and accept that `Node\_process\_count` and `Requests\_last\_minute` will silently fail until you re-bake. The other 7 metrics will work fine for the report's main story.

[user] could you explain each of this variable ?

```
#!/bin/bash
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
# --- OS metrics ---
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
TCP_CONN=$(netstat -an | wc -l)
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
IO_WAIT=$(iostat | awk '/^avg-cpu/ {getline; print $4}')
DISK_USED=$(df / | awk 'NR==2 {gsub("%", "", $5); print $5}')
PROC_COUNT=$(ps -e --no-headers | wc -l)
LOAD_1M=$(awk '{print $1}' /proc/loadavg)
# --- App-specific metrics ---
NODE_COUNT=$(pgrep -c node || echo 0)
APP_LOG="/var/log/placemark.log"
if [[ -f "$APP_LOG" ]]; then
    ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
    REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
else
    REQ_LAST_MIN=0
fi
# --- Default any empty values to 0 so AWS CLI doesn't choke ---
USEDMEMORY=${USEDMEMORY:-0}
TCP_CONN=${TCP_CONN:-0}
TCP_CONN_PORT_3000=${TCP_CONN_PORT_3000:-0}
IO_WAIT=${IO_WAIT:-0}
DISK_USED=${DISK_USED:-0}
PROC_COUNT=${PROC_COUNT:-0}
LOAD_1M=${LOAD_1M:-0}
NODE_COUNT=${NODE_COUNT:-0}
REQ_LAST_MIN=${REQ_LAST_MIN:-0}
# --- Push to CloudWatch ---
aws cloudwatch put-metric-data --metric-name memory-usage --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$USEDMEMORY"
aws cloudwatch put-metric-data --metric-name Tcp_connections --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$TCP_CONN"
aws cloudwatch put-metric-data --metric-name TCP_connection_on_port_3000 --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$TCP_CONN_PORT_3000"
aws cloudwatch put-metric-data --metric-name IO_WAIT --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$IO_WAIT"
aws cloudwatch put-metric-data --metric-name Disk_used_percent --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$DISK_USED"
aws cloudwatch put-metric-data --metric-name Process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$PROC_COUNT"
aws cloudwatch put-metric-data --metric-name Load_avg_1min --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$LOAD_1M"
```

```
aws cloudwatch put-metric-data --metric-name Node_process_count --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$NODE_COUNT"
aws cloudwatch put-metric-data --metric-name Requests_last_minute --dimensions Instance=$INSTANCE_ID --namespace "Custom" --value "$REQ_LAST_MIN"
```

[assistant] Walking through each variable, what it captures, and why it's useful as a metric.

****Authentication / instance-identity setup****

```
```bash
TOKEN=`curl ...`
```
```

Asks the EC2 instance metadata service (IMDSv2) for a session token, valid for 6 hours (21600 seconds). All subsequent metadata queries must include this token in a header – it's a security feature so other processes on the box can't accidentally read instance metadata.

```
```bash
INSTANCE_ID=$(curl ... /latest/meta-data/instance-id)
```
```

Uses that token to fetch this EC2 instance's unique ID (e.g., `i-06ce3cb7970b64bff`). This is the ****most important variable**** in the script – it becomes the CloudWatch dimension that lets each ASG instance report its metrics separately. Without it, all instances would dump into one combined series.

****OS metrics****

```
```bash
USEDMEMORY=$(free -m | awk 'NR==2{printf "%.2f", $3*100/$2}')
```
```

`free -m` prints memory stats in MB. `awk 'NR==2'` picks line 2 (the "Mem:" line). `\$3` is used memory, `\$2` is total. So this calculates ****used memory as a percentage****, e.g., `42.50`. Useful because high memory pressure is a classic scaling trigger.

```
```bash
TCP_CONN=$(netstat -an | wc -l)
```
```

`netstat -an` lists all network connections (`-a` = all states, `-n` = numeric, no DNS lookups). `wc -l` counts the lines. Gives the ****total number of TCP/UDP connections**** on the box. Spikes during traffic surges.

```
```bash
TCP_CONN_PORT_3000=$(netstat -an | grep :3000 | wc -l)
```
```

Same as above, but filtered to lines containing `:3000` (your app's port). Shows ****how many connections are talking to PlaceMark right now****. Direct signal of how busy the app is.

```
```bash
IO_WAIT=$(iostat | awk '/^avg-cpu/ {getline; print $4}')
```
```

`iostat` prints CPU and I/O statistics. The awk pattern says: "find the line starting with `avg-cpu`, then jump to the next line and print the 4th column" – that column is `%iowait`, the percentage of CPU time spent waiting for disk. ****High iowait means storage is the bottleneck.****

```
```bash
DISK_USED=$(df / | awk 'NR==2 {gsub("%","", $5); print $5}')
```
```

`df /` shows disk usage of the root filesystem. `awk 'NR==2'` picks line 2 (the data row). `\$5` is the "Use%" column (e.g., `34%`), and `gsub("%","", \$5)` strips the percent sign. Result: a plain number like `34`. ****Catches disks filling up before the app crashes.****

```
```bash
PROC_COUNT=$(ps -e --no-headers | wc -l)
```
```

`ps -e` lists every process; `--no-headers` skips the header row; `wc -l` counts. Gives the ****total number of running processes****. Sudden jumps can

indicate a fork bomb, runaway service, or just legitimate scaling.

```
```bash
LOAD_1M=$(awk '{print $1}' /proc/loadavg)
```
```

`/proc/loadavg` is a kernel file containing three numbers: the 1-, 5-, and 15-minute load averages (and a couple of process counters). Awk grabs `$1` – the ****1-minute load average****. Load `>` number-of-CPU's means the system is overloaded; on a `t2.nano` with 1 vCPU, anything above `~1.0` sustained is a scale-out signal.

****App-specific metrics****

```
```bash
NODE_COUNT=$(pgrep -c node || echo 0)
```
```

`pgrep -c node` counts how many running processes are named "node". If your PlaceMark app is alive, this is ≥ 1 . If it crashed or never started, it's 0. ****Direct proof the app process exists**** – useful because the OS could be healthy while the app is dead.

```
```bash
APP_LOG="/var/log/placemark.log"
```
```

Path to the file PlaceMark writes its stdout/stderr to. Set up by your User Data (the `&&& /var/log/placemark.log` redirect`).

```
```bash
if [[-f "$APP_LOG"]]; then
 ONE_MIN_AGO=$(date -d "1 min ago" "+%Y-%m-%dT%H:%M")
 REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" ...)
```
```

Only runs if the log file exists. `date -d "1 min ago"` produces a timestamp like ``2026-05-01T19:42``. `grep -c` then counts how many log lines contain that timestamp. So REQ_LAST_MIN` is **roughly the number of log entries in the previous minute** – a proxy for "how many requests did the app handle?" (Assumes PlaceMark logs each request with an ISO timestamp.)`

The `else REQ_LAST_MIN=0` branch handles the case where the log file isn't there yet (e.g., first boot before the app has flushed any output).`

****Empty-value defaults****

```
```bash
USEDMEMORY=${USEDMEMORY:-0}
TCP_CONN=${TCP_CONN:-0}
... etc.
```
```

The ``${VAR:-0}` syntax means: "use `${VAR}`, but if it's empty/unset, substitute `0`." A safety net – if any of the upstream commands silently produces no output (e.g., iostat layout shifts on a different kernel, the log file moves, a tool isn't installed), the script pushes a `0` instead of breaking the AWS CLI call with a malformed argument.`

****The push commands****

```
```bash
aws cloudwatch put-metric-data \
 --metric-name memory-usage \
 --dimensions Instance=$INSTANCE_ID \
 --namespace "Custom" \
 --value "$USEDMEMORY"
```
```

For each of the 9 variables, send one data point to CloudWatch:

- ``${--metric-name}``: name of the metric in CloudWatch (what shows in the console).

- `--dimensions Instance=<id>``: the per-instance tag. CloudWatch groups data points with the same name+dimension into one time series, so each instance gets its own line on the graph.
- `--namespace "Custom"```: the bucket where it lives in the CloudWatch UI (``All metrics → Custom``).
- `--value "$VAR"```: the actual number to record.

So every minute, cron runs the script, the script gathers 9 measurements, and pushes each one as a fresh data point under ``Custom / Instance=<this-instance-id> / <metric-name>``. Multiple instances build up multiple lines, all under the same metric names – that's what makes the side-by-side comparison in the CloudWatch graph possible.

****Why this set of 9 covers the spec well:****

The assignment specifically lists "OS activity (CPU, memory, disk, number of processes, etc.)" *and* "web server or other server logs" as the two categories of custom metrics worth gathering. Your script hits memory, disk, processes, load, and I/O wait from the OS bucket; TCP connections (general and on port 3000) from the network bucket; node-process count and request count from the app/log bucket. Both spec categories are covered, with one InstanceId dimension threading all of them together.

[user] how to I check the crono is enable and what it is the time between the measurements

[assistant] Two commands:

****Is cron enabled (auto-starts on boot)?****

```
```bash
sudo systemctl is-enabled crond
```
```

- ``enabled`` → cron will start automatically on every boot, including on new ASG instances launched from the AMI. ✓
- ``disabled`` → fix it: ``sudo systemctl enable crond``

To also confirm cron is running *right now* (in addition to being enabled):

```
```bash
sudo systemctl is-active crond
```
```

Should print ``active``.

****What's the interval between measurements?****

```
```bash
crontab -l
```
```

That dumps every cron entry for the current user. You should see something like:

```
```
*/1 * * * * /home/ec2-user/mem.sh
```
```

Reading the five fields left-to-right: **minute / hour / day-of-month / month / day-of-week**. The values mean:

- ``*/1`` in the minute field = **every 1 minute**
- ``*`` in the rest = "any value"

So your script runs **once per minute, every hour, every day**. (Note: ``*/1`` is identical to plain ``*`` – both mean "every minute". ``*/5`` would be every 5 minutes, ``*/15`` every 15, and so on.)

****To prove it's actually firing on schedule:****

Watch the journal in real time:

```
```bash
sudo journalctl -u crond -f
```
```

(`-f` = follow.) Wait one minute and you'll see a new line appear with `(ec2-user) CMD (/home/ec2-user/mem.sh)`. Press Ctrl+C to stop watching.

Or check CloudWatch directly – if a new data point appears every minute on each metric's graph, cron is firing on schedule. That's the source of truth.

```
[user] [ec2-user@ip-10-0-24-62 ~]$ sudo systemctl is-active crond
active
```

```
[ec2-user@ip-10-0-24-62 ~]$ crontab -l
```

```
*/1 * * * * /home/ec2-user/[mem.sh](http://mem.sh)
```

```
[ec2-user@ip-10-0-24-62 ~]$ sudo journalctl -u crond -f
```

```
May 03 12:19:35 ip-10-0-24-62.ec2.internal crond[2151]: (CRON) INFO (running with inotify support)
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal CROND[7050]: (root) CMD (run-parts /etc/cron.hourly)
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal run-parts[7053]: (/etc/cron.hourly) starting 0anacron
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal anacron[7062]: Anacron started on 2026-05-03
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal anacron[7062]: Will run job `cron.daily' in 38 min.
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal anacron[7062]: Will run job `cron.weekly' in 58 min.
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal anacron[7062]: Will run job `cron.monthly' in 78 min.
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal anacron[7062]: Jobs will be executed sequentially
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal run-parts[7064]: (/etc/cron.hourly) finished 0anacron
```

```
May 03 13:01:01 ip-10-0-24-62.ec2.internal CROND[7047]: (root) CMDEND (run-parts /etc/cron.hourly)
```

```
^C
```

```
[ec2-user@ip-10-0-24-62 ~]$
```

[assistant] Cron is active and your entry is set, but I notice the journal output only shows `*system*` cron jobs firing (the hourly ``run-parts /etc/cron.hourly``) – there's no line for your ``mem.sh`` job. That's a clue.

Two possibilities:

****Possibility 1 – cron is firing your job but not logging it to the journal.****

On AL2023, user-level cron jobs sometimes don't appear in ``journalctl -u crond`` (only the system-level ones do). If your script `*is*` firing every minute, you'd still see fresh data points in CloudWatch – that's the real test.

****Possibility 2 – your crontab didn't survive the AMI bake or never got loaded.****

You're on a fresh instance (``10.0.24.62``), launched from the AMI. The AMI captured ``/var/spool/cron/ec2-user`` if it was set on the master. Let's verify it actually got picked up here.

****Run these to find out which possibility you're in:****

```
```bash
```

```
1. Confirm your script exists on THIS instance and runs
```

```
ls -la /home/ec2-user/mem.sh
```

```
/home/ec2-user/mem.sh && echo "MANUAL RUN OK"
```

```
2. Confirm cron's user spool actually has your entry
```

```
sudo cat /var/spool/cron/ec2-user
```

```
3. Add a log redirect to the crontab so we get hard proof of firing
```

```
crontab -l # shows current entry
```

```
env EDITOR=nano crontab -e
```

```
change the line to:
```

```
*/1 * * * * /home/ec2-user/mem.sh >> /home/ec2-user/cron.log 2>&1
save & exit
```

```
4. Wait 2 minutes, then:
ls -la /home/ec2-user/cron.log # file should exist with recent timestamp
tail /home/ec2-user/cron.log
```
```

****What the outcomes mean:****

If `cat /var/spool/cron/ec2-user` shows your line and `cron.log` gets a fresh modification time within 2 minutes → cron is firing fine, the journal just isn't recording it (Possibility 1). Don't worry – CloudWatch is the source of truth.

If `cron.log` never appears or has no fresh writes → cron isn't firing your job (Possibility 2). The fix is to re-add the crontab entry on this instance:

```
```bash
env EDITOR=nano crontab -e
add: */1 * * * * /home/ec2-user/mem.sh
```
```

****The definitive check, regardless of journal logs:****

Open AWS console → CloudWatch → Metrics → Custom → Instance dimension → look for **this instance's ID** (`i-...` of `10.0.24.62`). If you see metrics under that ID with timestamps from the last 2–3 minutes, everything is working. If not, paste the output of the four commands above.

```
[user] [ec2-user@ip-10-0-24-62 ~]$ ls -la /home/ec2-user/[mem.sh](http://mem.sh)
/home/ec2-user/[mem.sh](http://mem.sh) & & echo "MANUAL RUN OK"
-rwxr-xr-x. 1 ec2-user ec2-user 2606 May  3 12:46 /home/ec2-user/[mem.sh](http://mem.sh)
[&lt;class 'decimal.ConversionSyntax'&gt;]
[ec2-user@ip-10-0-24-62 ~]$ sudo cat /var/spool/cron/ec2-user
*/1 * * * * /home/ec2-user/[mem.sh](http://mem.sh)
[assistant] The script is still broken with the same `decimal.ConversionSyntax` bug – looks like the fix never made it into /home/ec2-user/mem.sh.
The two problem lines are still there. Let's surgically fix them.
```

****Open the file:****

```
```bash
nano /home/ec2-user/mem.sh
```
```

****Find this line:****

```
```bash
NODE_COUNT=$(pgrep -c node || echo 0)
```
```

****Replace it with these two lines:****

```
```bash
NODE_COUNT=$(pgrep -c node 2>/dev/null)
NODE_COUNT=${NODE_COUNT:-0}
```
```

****Then find this line:****

```
```bash
```

```
REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
...`
```

**\*\*Replace it with these two lines (keep the same indentation):\*\***

```
```bash
REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null)
REQ_LAST_MIN=${REQ_LAST_MIN:-0}
...`
```

Save (Ctrl+O, Enter, Ctrl+X).

****Re-test:****

```
```bash
/home/ec2-user/mem.sh &&& echo "MANUAL RUN OK"
...`
```

You should now see `MANUAL RUN OK` with no `decimal.ConversionSyntax` errors.

**\*\*Why this fixes it (one-liner for your future memory):\*\***

`pgrep -c` always prints the count to stdout, even when it's `0`. Combined with `|| echo 0`, you'd get the count printed *\*twice\** when no processes match (once from pgrep, once from echo), producing ``0\n0`` – which AWS CLI rejects as a malformed decimal. Capturing once and letting `\${VAR:-0}` handle empties gives you exactly one clean number.

Once `MANUAL RUN OK` prints, wait 2 minutes and check CloudWatch – the cron-driven measurements should resume cleanly.

[user] how to delete old metrics from claud watch

[assistant] Short answer: you can't directly delete CloudWatch metrics – AWS doesn't expose a delete API for them. They age out automatically based on the granularity you used:

- **\*\*1-minute\*\*** data points (what your script uses): kept for **\*\*15 days\*\***, then automatically removed
- 5-minute: 63 days
- 1-hour: 455 days

So any old instance IDs or test runs from your earlier VPC will simply disappear from the console after 15 days, no action needed.

**\*\*Practical options for "tidying up" right now:\*\***

The metrics list will keep showing every InstanceId you ever pushed under `Custom`, even for terminated instances. To make the console cleaner *\*for viewing\**, you have a few choices:

1. **\*\*Filter the view\*\*** – when you're in CloudWatch → Metrics → Custom → Instance, the search box at the top lets you type an InstanceId. Type your *\*current\** instance's ID to see only its metrics. The old ones still exist in the data store but stay out of view.
2. **\*\*Build a CloudWatch dashboard\*\*** with just the instances and metric names you care about (Add to dashboard → pick the metrics → save). The dashboard is the screenshot you'll actually use in the report; the cluttered "All metrics" page never needs to be opened again.
3. **\*\*Switch namespace if you want a clean slate.\*\*** Edit `mem.sh` to push to `"PlaceMark/Custom"` (or any new namespace) instead of `"Custom"`. The old namespace still exists with its 15-day-aging data, but a fresh one starts empty. This is the closest thing to "delete and start over."

```
```bash
# In mem.sh, change every:
--namespace "Custom"
# to:
--namespace "PlaceMark/Custom"
```

...

After you re-run, check the console for the new namespace – only fresh data points appear there. The old `Custom` entries will quietly age out over the next two weeks.

****For the report:****

When you take screenshots, just zoom into the instances you care about and the time window of your demo. Old terminated-instance lines won't show up in a screenshot if you set the time range to "Last 1 hour" and only tick the boxes for your current ASG instances. The marker won't see the clutter.

[user] i want to change the period to 1 min

[assistant] Two ways to change the period – quick fix and the bulk fix.

****Quickest – change all at once via the time range:****

Look at the ****time range buttons**** at the top-right of the graph (1h, 3h, 12h, 1d, 3d, 1w, custom). Click ****1h****.

Then on the right side of the same toolbar, look for a ****gear icon**** (Graph options) or a "Period" dropdown at the graph level – depending on the view it's labeled differently. Set that to ****1 Minute****. This applies to every metric on the graph in one go.

****Or – change each row individually:****

In the table you're showing, each row has its own "Period" dropdown (currently `5 minu...`). Click it on each row and pick ****1 Minute****. With 8 metrics that's 8 clicks, but it works.

****Heads-up about period and time range:****

CloudWatch enforces a relationship between ****time range**** and ****minimum period****:

- Last 1 hour → minimum period is 1 second (1 Minute is fine)
- Last 3 hours → minimum 1 minute
- Last 1 day → minimum 5 minutes (1 Minute will be greyed out)
- Last 3+ days → minimum 1 hour

So if you don't see "1 Minute" available, ****shorten the time range first**** (1h or 3h), then change the period.

****Why this matters for your data:****

Your `mem.sh` pushes one data point per minute. If you graph at 5-minute period, CloudWatch averages every 5 of your data points into one – visually smoother, but you can't see minute-by-minute spikes from a load test. At 1-minute period over a 1-hour window, you see every individual data point – which is what you want for the load-test screenshot in your report.

****Save this for next time:****

Click ****Actions → Save dashboard**** (or Add to dashboard) once you have a graph configured the way you like – period, time range, metrics. That bookmarks the view so you don't have to re-set the period every visit.

[user] Last login: Sun May 3 14:24:11 2026

[ec2-user@ip-10-0-31-134 ~]\$ crontab -l

*/1 * * * * /home/ec2-user/mem.sh

[ec2-user@ip-10-0-31-134 ~]\$ sudo systemctl is-active crond
active

[ec2-user@ip-10-0-31-134 ~]\$ sudo journalctl -u crond -f

May 03 14:24:09 ip-10-0-31-134.ec2.internal systemd[1]: Started crond.service - Command Scheduler.

May 03 14:24:09 ip-10-0-31-134.ec2.internal crond[2150]: (CRON) STARTUP (1.5.7)

May 03 14:24:09 ip-10-0-31-134.ec2.internal crond[2150]: (CRON) INFO (Syslog will be used instead of sendmail.)

May 03 14:24:09 ip-10-0-31-134.ec2.internal crond[2150]: (CRON) INFO (RANDOM_DELAY will be scaled with factor 65% if used.)

May 03 14:24:09 ip-10-0-31-134.ec2.internal crond[2150]: (CRON) INFO (running with inotify support)

^C

```
[ec2-user@ip-10-0-31-134 ~]$
```

[assistant] You logged in at 14:24:11 and Ctrl+C'd almost immediately, so cron hasn't had a chance to fire yet (the next minute boundary was 14:25:00). Two ways to confirm it's working:

****Wait 2-3 minutes, then re-check:****

```
```bash
sudo journalctl -u crond --since "5 minutes ago"
```
```

You should see lines like `(ec2-user) CMD (/home/ec2-user/mem.sh)` for each minute that's passed – though as we saw before, AL2023 sometimes hides user-level cron entries from journalctl, so don't worry if you don't see them.

****Faster – verify the script works manually first:****

```
```bash
/home/ec2-user/mem.sh &&& echo "OK"
```
```

`OK` means script + AWS creds + script value parsing all good. Anything else (especially `ExpiredToken` or `ConversionSyntax`) tells us what to fix on this fresh instance.

****Most reliable check – go straight to CloudWatch:****

AWS console → CloudWatch → Metrics → Custom → Instance dimension → look for **this instance's ID** (the one starting with `i-...` matching `10.0.31.134`). If you see fresh data points within 2-3 minutes of the instance booting, everything works.

Run the manual script and paste the output if it errors – otherwise you're good to leave it to cron and go look at CloudWatch.

[user] metrics are not there,

[assistant] Let's diagnose. Run these in order and paste the output:

```
```bash
1. Does the script work right now, manually?
/home/ec2-user/mem.sh &&& echo "OK"
```
```

If you see `ExpiredToken` errors → lab session expired, refresh in Moodle.

If you see `decimal.ConversionSyntax` → script still has the unfixed bug.

If you see `OK` → script works, then it's a CloudWatch view issue (wrong region, wrong instance ID filter).

```
```bash
2. Are credentials actually working?
aws sts get-caller-identity
```
```

Should return an ARN. If `ExpiredToken`, that's the issue.

```
```bash
3. Stale credentials file from the AMI?
ls -la ~/.aws/
```
```

If `~/.aws/credentials` exists, it's overriding the IAM role:

```
```bash
```

```
rm -f ~/.aws/credentials
```
```

Then re-run step 1.

```
```bash
4. Get this instance's ID so you can find it in CloudWatch
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id
```
```

Note the `i-...` ID. Then in the AWS console make sure you're in the `us-east-1` (N. Virginia) region, go to CloudWatch → Metrics → All metrics → Custom → Instance, and search for that exact ID.

Run those four and tell me what each one prints – that'll pinpoint it within one round.

```
[user] [ec2-user@ip-10-0-31-134 ~]$ nano ~/.aws/credentials
[ec2-user@ip-10-0-31-134 ~]$ /home/ec2-user/[mem.sh](http://mem.sh) && echo "OK"
usage: aws [options] <command> [<subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:
```

```
aws help
aws <command> help
aws <command> <subcommand> help
Unknown options: 0
[ec2-user@ip-10-0-31-134 ~]$ rm -f ~/.aws/credentials
[ec2-user@ip-10-0-31-134 ~]$ /home/ec2-user/[mem.sh](http://mem.sh) && echo "OK"
Unable to locate credentials. You can configure credentials by running "aws login".
Unable to locate credentials. You can configure credentials by running "aws login".
Unable to locate credentials. You can configure credentials by running "aws login".
Unable to locate credentials. You can configure credentials by running "aws login".
Unable to locate credentials. You can configure credentials by running "aws login".
Unable to locate credentials. You can configure credentials by running "aws login".
Unable to locate credentials. You can configure credentials by running "aws login".
usage: aws [options] <command> [<subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:
```

```
aws help
aws <command> help
aws <command> <subcommand> help
Unknown options: 0
```

```
[ec2-user@ip-10-0-31-134 ~]$ ^C
[ec2-user@ip-10-0-31-134 ~]$
```

[assistant] Two distinct problems on this instance – let's fix both.

****Problem 1 – No IAM role attached.****

The "Unable to locate credentials" tells us the launch template didn't attach `LabInstanceProfile` (or you launched this instance directly without setting it). The AWS CLI has nothing to authenticate with, so every metric call fails.

Fix: in the AWS console → EC2 → Instances → select this instance (the one with IP `10.0.31.134`) → **Actions** → Security → **Modify IAM role** → pick **LabInstanceProfile** → **Update IAM role**.

The change takes effect within ~60 seconds (no instance restart needed). Verify:

```
```bash
aws sts get-caller-identity
```
```

Should return an ARN containing `LabRole` or similar.

****Problem 2 – The script still has the `pgrep / grep` bug.****

This instance was launched from an AMI baked *before* you applied the fix, so the broken version of `mem.sh` is in the AMI. Fix it on this instance:

```
```bash
nano /home/ec2-user/mem.sh
```
```

Find these lines:

```
```bash
NODE_COUNT=$(pgrep -c node || echo 0)
```
```

Replace with:

```
```bash
NODE_COUNT=$(pgrep -c node 2>/dev/null)
NODE_COUNT=${NODE_COUNT:-0}
```
```

And find:

```
```bash
REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
```
```

Replace with:

```
```bash
REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null)
REQ_LAST_MIN=${REQ_LAST_MIN:-0}
```
```

Save (Ctrl+O, Enter, Ctrl+X), then test:

```
```bash
/home/ec2-user/mem.sh &&& echo "OK"
```
```

You should see `OK` cleanly. CloudWatch should pick up the metrics within a minute.

****Once it works on this instance, you have two choices for the AMI:****

****Option A (quick) – re-bake the AMI now**** so the next ASG instance inherits the fix:

```
```bash
pkill node 2>/dev/null
rm -f ~/.aws/credentials # don't bake personal keys
```
```

Then in the console: stop instance → Create image → name `PlaceMark-master-v2` → wait for `available` → update the launch template to point at v2.

****Option B (defer) – leave the AMI broken for now****, accept that test ASG instances will have the script bug until you re-bake. Fine if you're under

time pressure and just need the master/single-instance demo to work.

****Also for the launch template**** – go check it now and confirm "IAM instance profile" under Advanced details is set to ``LabInstanceProfile``. If it's empty, every instance the ASG launches will hit the same "no credentials" wall you just hit. Fix that once and every future instance will inherit the role.

[user] is there a way to set the IAM role automatically ?

[assistant] Yes – that's exactly what the ****launch template**** is for. Set the IAM role on the launch template once, and every instance launched from it (manually or by the ASG) inherits it automatically. You never have to attach it by hand again.

****Where to set it:****

EC2 console → ****Launch Templates**** (left menu) → click your launch template → ****Actions → Modify template (Create new version)****.

Launch templates are versioned – you can't edit an existing version, only create a new one. That's a feature, not a bug: it lets you roll back if something goes wrong.

In the new version's form:

1. Name the version description: e.g., ``add-iam-role``.
2. Scroll to ****Advanced details**** at the bottom.
3. Find ****IAM instance profile**** → pick ****`LabInstanceProfile`**** from the dropdown.
4. Leave everything else as-is (the form pre-fills from the previous version).
5. Click ****Create template version****.

****Make the new version the default:****

After creation, on the launch template's page, click ****Actions → Set default version**** → pick the new version. Now any ASG using "default version" will launch instances with the role attached.

****If you have an ASG attached:****

The ASG references a specific version of the launch template. To pick up the change:

EC2 → Auto Scaling Groups → select your ASG → ****Edit**** → under "Launch template," set Version to ****Default**** (or specifically the new version number) → Update.

The ASG won't replace existing instances automatically (those keep their current config), but every **new** instance it launches from now on will use the updated launch template – meaning IAM role attached on first boot. To force a refresh of existing instances, use ****Actions → Start instance refresh**** on the ASG.

****Verify the new behaviour:****

Launch one test instance from the launch template (Actions → Launch instance from template → 1 instance). SSH into it and run:

```
``bash
aws sts get-caller-identity
``
```

If it returns an ARN without you doing anything, the role is now wired in for good. CloudWatch metrics should start flowing automatically within 1-2 minutes – no more "Unable to locate credentials" errors.

****One small thing for the report:****

This is a clean "infrastructure as code" talking point: the launch template is your **single source of truth** for instance configuration. IAM role, AMI, instance type, security groups, user data – all locked into one versioned object. New ASG instances are guaranteed to be configured identically because they all derive from the same template version. Worth a sentence in the architecture section.

[user] but every time it is a different lab, does it matter?

[assistant] Good question, and the answer is yes – the launch template setting persists across lab sessions, so you only have to set it **once**. Here's why:

What's actually happening with the Learner Lab:

Every time you "start" the lab in Moodle, you're not getting a **new** AWS account. You're resuming the **same** account with the same persistent state. What gets refreshed is just the temporary session credentials issued by the IAM roles – the underlying account, all your resources (EC2 instances, AMIs, launch templates, VPCs, security groups), and IAM roles like `LabInstanceProfile` are all sitting there waiting for you when you log back in.

So the launch template you create today will still be there tomorrow, with `LabInstanceProfile` attached, and any instance the ASG launches will inherit it correctly.

What does change between lab sessions:

The temporary credentials your instances get **via** the IAM role – those rotate every few hours. But the instance picks up fresh credentials automatically from the metadata service the moment you start the lab again. No manual intervention needed.

What also changes: any **personal CLI access keys** you might have copied from "AWS Details → AWS CLI" into a `~/.aws/credentials` file – those expire when the lab ends and are reissued when it starts. (Yet another reason not to bake `~/.aws/credentials` into the AMI.)

The thing that genuinely is a risk:

The assignment PDF warns about this: "AWS Academy accounts have no durability guarantees... your account could be wiped at any time." This isn't routine behaviour – it usually only happens if you trip an alarm (e.g., running too many instances simultaneously). If it does happen, you lose **everything** at once: instances, AMIs, launch templates, VPCs, the lot. There's no per-session wipe, but there is a "the whole lab account got nuked" risk.

This is why the additional functionality option of **automating with Python/boto3 or CloudFormation** is so valuable for this assignment – if the account ever does get reset, you can recreate the entire architecture in minutes from a script. Worth a short paragraph in your report on disaster recovery, even if you never have to use it.

Practical answer to "does it matter?":

For day-to-day work between lab sessions – no, it doesn't matter. Set the IAM role on the launch template once and it stays. Your AMI, ASG, VPC, and security groups all persist. Just refresh the lab in Moodle when credentials expire and the running instances pick up the new tokens.

For account-wipe protection – that's where automation/CloudFormation earns its 20% additional-functionality marks.

```
[user] [ec2-user@ip-10-0-31-134 ~]$ /home/ec2-user/[mem.sh](http://mem.sh) &&& echo "OK"
```

```
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
```

To see help text, you can run:

```
aws help
```

```
aws <command> help
```

```
aws <command> <subcommand> help
```

Unknown options: 0

```
[ec2-user@ip-10-0-31-134 ~]$
```

[assistant] Auth is working now (no more "Unable to locate credentials") – that's progress. The remaining error is the script bug from before that still hasn't been edited on this instance.

Open the script:

```
``bash
```

```
nano /home/ec2-user/mem.sh
```

```
``
```

Find this line and change it:

```
Currently:
```bash
NODE_COUNT=$(pgrep -c node || echo 0)
```
```

```
Change to:
```bash
NODE_COUNT=$(pgrep -c node 2>/dev/null)
NODE_COUNT=${NODE_COUNT:-0}
```
```

****Find this line and change it too:****

```
Currently (inside the `if` block):
```bash
REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null || echo 0)
```
```

```
Change to:
```bash
REQ_LAST_MIN=$(grep -c "$ONE_MIN_AGO" "$APP_LOG" 2>/dev/null)
REQ_LAST_MIN=${REQ_LAST_MIN:-0}
```
```

Save: Ctrl+O, Enter, Ctrl+X.

****Test:****

```
```bash
/home/ec2-user/mem.sh && echo "OK"
```
```

You should see a clean `OK` with no errors.

****Why the existing version breaks:****

`pgrep -c` and `grep -c` *always print the count to stdout* (even `0`), but they exit with status 1 when there's no match. The `|| echo 0` fallback then runs because of the non-zero exit, *also* printing `0`. The shell captures both – your variable becomes `0\n0` (two lines), AWS CLI sees it as a malformed decimal, and the script dies with `Unknown options: 0`.

The fix removes the `|| echo 0` and instead uses `\${VAR:-0}` (parameter expansion), which only substitutes if the variable is genuinely empty – so you get exactly one clean number.

Once `OK` prints, wait 1-2 minutes and CloudWatch will start receiving metrics from this instance.

[user] how to test the increase of tcp connection on port 300

[assistant] Three ways depending on what you have on hand. Run any of these from your laptop, bastion, or another EC2 instance – anywhere that can reach port 3000 on the web server.

****Quick method – curl in a loop:****

```
```bash
for i in {1..100}; do curl -s http://10.0.31.134:3000 >/dev/null; done
```
```

Hits the app 100 times sequentially. Quick and dirty – proves connections are happening but the connections close fast, so the metric may only tick up briefly.

****Sustained load – keep connections open with curl:****

```
```bash
for i in {1..50}; do curl -s --max-time 30 http://10.0.31.134:3000 > /dev/null & & done
wait
```
```

50 parallel curls held open for up to 30 seconds – much better for seeing the TCP count actually rise. The `&` backgrounds each one, `wait` blocks until they all finish.

****Best – Apache Bench (real load test):****

```
```bash
sudo dnf install -y httpd-tools # one-time install
ab -n 1000 -c 50 http://10.0.31.134:3000/
```
```

`-n 1000` = total requests, `-c 50` = 50 concurrent. While it runs, the `TCP\_connection\_on\_port\_3000` metric should spike. ab also prints useful stats (requests per second, latency) which is great for the report.

****While the test runs, watch the metric tick:****

In another terminal on the web server (or the bastion), run:

```
```bash
watch -n 1 'netstat -an | grep :3000 | wc -l'
```
```

You'll see the count update once per second. Should jump from a low single digit to dozens (or whatever your `-c` was) during the test, then return to baseline after.

****See it on CloudWatch:****

After 2–3 minutes, refresh ****CloudWatch → Custom → Instance → TCP_connection_on_port_3000**** with period set to ****1 Minute**** and time range ****Last 1 hour****. You should see a clear bump aligned with when you ran the test. That's screenshot-worthy for the report – proves the metric reflects real traffic.

****For the actual spec #6 evidence later:****

Run `ab` against the ****ALB DNS**** rather than a single instance:

```
```bash
ab -n 1000 -c 50 http://&alb-dns-name&;/
```
```

The ALB will distribute connections across all ASG instances, and you'll see *every* instance's `TCP\_connection\_on\_port\_3000` metric rise simultaneously – different lines on the same graph. That's exactly the "load distributed across multiple web servers" picture the spec asks for.

[user] how to generate traffic in my web app

[assistant] Three tools, picked by what you want to test.

****For a quick smoke test – `curl`:****

```
```bash
curl http://10.0.31.134:3000/ # one request, see the response
curl http://10.0.31.134:3000/test1b # if you added the route, returns instance ID
```
```

Useful to confirm the app is actually serving. Not enough load to trigger anything.

****For real traffic – `ab` (Apache Bench):****

```
```bash
sudo dnf install -y httpd-tools # one-time install on the box you run from
ab -n 5000 -c 50 http://10.0.31.134:3000/
```
```

`-n 5000` total requests, `-c 50` concurrent. Sustained load – this is the one that drives CPU up and triggers your scaling policy. Run it for several minutes (`-n 50000` or run in a loop) so the alarm period elapses and the ASG actually scales.

****For longer / programmatic traffic – `for` loop with curl:****

```
```bash
for i in {1..1000}; do curl -s http://10.0.31.134:3000/testlb; echo; done
```
```

If you added a `/testlb` route that returns the instance ID, this gives you the ****distribution evidence**** for spec #6 – you'll see different instance IDs in the responses once the ASG has multiple instances behind the ALB.

****Where to run from:****

- ****From your laptop**** – fine for a quick test if port 3000 is reachable. If you closed it down to BastionSG only, you can't.
- ****From the bastion**** – fast, simple, always works (it's inside the VPC).
- ****From a separate EC2 instance dedicated to load generation**** – best for the report, because the load doesn't compete with the app server's resources and you can run sustained tests without your laptop being involved.

****What to watch while it runs:****

Open three terminals side-by-side:

1. ****CloudWatch**** → custom metrics graph at 1-minute period → see TCP connections, memory, CPU, requests/min all rising.
2. ****App server's log****:

```
```
tail -f /var/log/placemark.log
```
```

Live stream of requests landing on each instance.
3. ****`ab` itself**** – prints throughput, average latency, success rate when it finishes. Those numbers are great for the report ("the system handled X req/s before scaling kicked in").

****For the assignment specifically:****

- Generating traffic against a single instance proves your app and metrics work.
- Generating traffic against the ****ALB DNS**** (after you set it up) proves the load balancer distributes work across the ASG – that's spec #6's main evidence.

Run `ab` against the ALB DNS, watch CPU climb past your alarm threshold, see ASG scale-out trigger, see `/testlb` start returning multiple different instance IDs, then stop the load and watch scale-in. Three screenshots from that one test cover spec items #4, #5, and #6.

[user] to increase the tcp connection on port 3000

[assistant] To make `TCP\_connection\_on\_port\_3000` spike noticeably, you need ****many concurrent connections held open at once****, not quick fire-and-close requests. Three tools, ranked by effectiveness:

****Best – `ab` with high concurrency:****

```
```bash
```



```
sudo dnf install -y httpd-tools # if not installed
ab -n 10000 -c 200 http://10.0.31.134:3000/
```
```

`-c 200` = 200 simultaneous connections held open while the test runs. `netstat -an | grep :3000 | wc -l` will jump to ~200+ during the test. Run it for a few minutes by upping `-n`.

****Even better – `wrk` (modern, sustained load):****

```
```bash
sudo dnf install -y wrk # may need EPEL or build from source
wrk -t 4 -c 300 -d 60s http://10.0.31.134:3000/
```
```

4 threads, 300 connections, for 60 seconds. Better than `ab` for sustained traffic – connections stay actually open rather than churning.

****No-install option – parallel `curl` in background:****

```
```bash
for i in {1..200}; do
 curl -s --max-time 60 http://10.0.31.134:3000/ &> /dev/null &
done
```
```

Spawns 200 background curls, each held open up to 60 seconds. Then `wait` to keep your shell from exiting:

```
```bash
wait
```
```

Stop early with Ctrl+C if needed.

****Trick to inflate the metric without sustained load – TIME_WAIT entries:****

`netstat -an | grep :3000 | wc -l` counts ****all**** connection states, including the `TIME\_WAIT` lingering ones (which stick around ~60 seconds after the connection closes). So even a fast burst leaves a trail:

```
```bash
for i in {1..2000}; do curl -s http://10.0.31.134:3000/ &> /dev/null; done
```
```

Run that and immediately check:

```
```bash
netstat -an | grep :3000 | wc -l
```
```

You'll see thousands of `TIME\_WAIT` entries – the metric will spike for the next minute even though no real load is currently active.

****Watch the metric live (separate terminal on the web server or bastion):****

```
```bash
watch -n 1 'netstat -an | grep :3000 | wc -l'
```
```

Updates once per second. Run any of the above traffic generators in another terminal and you'll see the count climb in real time.

****For CloudWatch:****

Cron pushes the value once per minute. So:

1. Start the traffic.
2. Wait at least 60 seconds for the next cron tick.
3. CloudWatch → metric `TCP\_connection\_on\_port\_3000` at 1-minute period → fresh data point shows the spike.

If you're testing at minute 12:00:30 and cron last fired at 12:00:00, the next push is 12:01:00 – so always wait for the next minute mark before refreshing CloudWatch.

[user] curl to this dns

[assistant] You'll need to paste in the actual DNS – it'll look something like `placemark-alb-1234567890.us-east-1.elb.amazonaws.com`. Find it in AWS console → EC2 → Load Balancers → click your ALB → "DNS name" field at the top.

Once you have it:

```
```bash
Single request – sanity check
curl -i http://your-alb-1234567890.us-east-1.elb.amazonaws.com/

50 quick requests – see different instance IDs in the responses
for i in {1..50}; do curl -s http://your-alb-1234567890.us-east-1.elb.amazonaws.com/testlb; echo; done

Sustained load – drives ALB+ASG metrics
ab -n 5000 -c 100 http://your-alb-1234567890.us-east-1.elb.amazonaws.com/
```
```

Paste the actual DNS and I'll give you ready-to-run commands.

****One note about the URL format:****

The ALB serves on port 80 by default – no `:3000` needed. The ALB listens on 80 (or 443), and forwards to port 3000 on your instances behind the scenes (defined by the target group). So users see clean URLs, and your app port stays internal. If you've kept everything default, just `http://<alb-dns>` works.

[user] PlacemarkLB-762126245.us-east-1.elb.amazonaws.com

[assistant] No response requested.

[user] http://placemarklb-762126245.us-east-1.elb.amazonaws.com/

[assistant] Ready-to-run commands using your ALB DNS:

****Sanity check – does the ALB reach your app?****

```
```bash
curl -i http://placemarklb-762126245.us-east-1.elb.amazonaws.com/
```
```

Should return PlaceMark's login page or a redirect (200 / 302). If you get `502 Bad Gateway`, the ALB couldn't reach a healthy target – check the target group health.

****See traffic distributing across instances (spec #6 evidence):****

```
```bash
for i in {1..50}; do curl -s http://placemarklb-762126245.us-east-1.elb.amazonaws.com/testlb; echo; done
```
```

You'll see a mix of different instance IDs in the responses – proof the ALB is round-robinning across your ASG instances. ****Take a screenshot of this terminal output**** for the report.

****Sustained load – drives CPU above scale-out threshold:****

```
```bash
sudo dnf install -y httpd-tools # if ab isn't installed
ab -n 5000 -c 100 http://placemaklb-762126245.us-east-1.elb.amazonaws.com/
```
```

5000 requests, 100 concurrent. Run it for a few minutes (rerun with ``-n 50000`` if you want it longer). While it runs, watch:

- ****CloudWatch → CPU utilization**** on the ASG – should climb past your alarm threshold.
- ****CloudWatch → custom metrics**** – TCP connections and request count both spike across all instances.
- ****EC2 → Auto Scaling Groups → Activity history**** – you'll see "Launching" events when scaling out, "Terminating" when scaling in.

****Heavier – wrk for sustained connections:****

```
```bash
wrk -t 4 -c 200 -d 120s http://placemaklb-762126245.us-east-1.elb.amazonaws.com/
```
```

If you have wrk available – 4 threads, 200 connections, 2 minutes. Better than ``ab`` at sustaining real connection counts.

****Practical run order for the demo / report screenshots:****

1. Open three browser tabs: CloudWatch metrics graph, EC2 Auto Scaling Activity tab, and the ALB target group's Targets tab.
2. Run the ``for`` loop with ``/testlb`` first – capture mixed instance IDs.
3. Start ``ab`` with high ``-n``.
4. Watch CPU climb → alarm fires → ASG launches new instances → target group goes from 2 healthy to 3 or 4.
5. Stop traffic → CPU falls → ASG terminates extras → target group drops back.
6. Screenshot the CloudWatch graph showing the full curve: ramp-up, plateau, ramp-down.

That single run gives you evidence for spec items #4 (ASG launches), #5 (scaling triggered by alarm), and #6 (load distributed across instances).

[user] how to know if cloud watch is running, and crono

[assistant] Two quick check sequences – one for each.

****Is cron firing the script?****

```
```bash
1. Daemon running?
sudo systemctl is-active crond
Expected: active

2. Your entry is there?
crontab -l
Expected: */1 * * * * /home/ec2-user/mem.sh

3. Add a log redirect if you don't have one yet
env EDITOR=nano crontab -e
change line to: */1 * * * * /home/ec2-user/mem.sh &&& /home/ec2-user/cron.log 2&&&1
save &&& exit

4. Wait 2 minutes, then check the log file
ls -la /home/ec2-user/cron.log
tail /home/ec2-user/cron.log
```
```

If ``cron.log``'s timestamp updates within 2 minutes, cron is firing. If the file has error text, that's your debug clue. If the file doesn't appear at

all, cron isn't running your job (rare – most likely something deleted the crontab).

****Is CloudWatch receiving the metrics?****

Three ways, in order of reliability.

****Console (most visual):****

AWS console → CloudWatch → Metrics → All metrics → Custom → Instance → look for this instance's ID. If you see your metrics listed and they have data points within the last 1-2 minutes, everything works.

****CLI (most direct):****

```
```bash
aws cloudwatch list-metrics --namespace Custom \
 --dimensions Name=Instance,Value=$(curl -s -H "X-aws-ec2-metadata-token: $(curl -s -X PUT http://169.254.169.254/latest/api/token -H 'X-aws-ec2-metadata-token-ttl-seconds: 60')" http://169.254.169.254/latest/meta-data/instance-id)
```
```

Returns the list of metric names registered for this specific instance. If the list is empty, no metrics are being pushed.

****Get the latest datapoint for one metric:****

```
```bash
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $(curl -s -X PUT http://169.254.169.254/latest/api/token -H 'X-aws-ec2-metadata-token-ttl-seconds: 60')" http://169.254.169.254/latest/meta-data/instance-id)
```

```
aws cloudwatch get-metric-statistics \
 --namespace Custom \
 --metric-name memory-usage \
 --dimensions Name=Instance,Value=$INSTANCE_ID \
 --start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%S) \
 --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
 --period 60 \
 --statistics Average
```
```

If `Datapoints` is empty, metrics aren't arriving. If it lists a recent timestamp + value, they are.

****One-shot health check (copy-paste this whole block):****

```
```bash
echo "=== Cron daemon ==="; sudo systemctl is-active crond
echo "=== Crontab ==="; crontab -l
echo "=== Last script output ==="; tail -3 /home/ec2-user/cron.log 2>& /dev/null || echo "(no cron.log yet)"
echo "=== Manual run ==="; /home/ec2-user/mem.sh && echo "OK" || echo "FAIL"
echo "=== AWS auth ==="; aws sts get-caller-identity --output text
```
```

Five lines tell you everything: daemon active, schedule set, recent output exists, script works manually, AWS credentials valid. If all five say OK, you're done – go look at CloudWatch for confirmation.